THE DRAWING(S) AND THE INFORMATION CONTAINED HEREIN ARE THE PROPERTY OF **WOOJIN INDUSTRIAL SYSTEMS CO., LTD.** THEY SHALL NOT BE DISCLOSED, REPRODUCED OR USED, IN ANY MANNER WITHOUT THE PRIOR WRITTEN CONSENT OF **WOOJIN INDUSTRIAL SYSTEMS CO., LTD.**

DENVER RTD EAGLE P3

Communication System

Software Design Description

DISTRIBUTION	
TOTAL	

REVISION							
0	Jan. 10, 2012	First Issued	D.B. LEE	D.S. JANG	B.J. JONE	E.K. LEE
No.	ISSUED DATE	REVISION NOTE	DESIGNED	CHECKED	APPROVED	REVIEWED
			ENGINEERING			PM

PM	REVIEWED	E.K.LEE	CUSTOMER : **HYUNDAI-ROTEM COMPANY**
ENGINEERING	APPROVED	B.J.JONE	
	CHECKED	D.S.JANG	
	DESIGNED	D.B.LEE	PROJECT : **DENVER RTD COMMUNICATION SYSTEM**
ISSUED DATE	January 10, 2012		
ISSUED BY	R & D CENTER		
PROJECT NO.	P2011-020		
ORDER NO.	P110701149		
DOCUMENT NO.		REVISION	WOOJIN Industrial Systems Co., Ltd.
WJ-RTD-COMM-SDD-V0		**0**	

REVISION RECORD

Rev. No.	PAGE	Details		Date
		FROM	TO	
0		Issued		Jan. 10, 2012

Table of Contents

1.0 Introduction 4
1.1 Purpose 4
1.2 Scope 4
1.3 Acronyms and Abbreviations. 4
2.0 Reference 5
3.0 Decomposition description 6
3.1 PAC Description 6
3.1.1 Process Decomposition 6
3.1.2 Data Composition 14
3.2 CCP Description 15
3.2.1 Process Decomposition 15
3.2.2 Data Decomposition 20
3.3 CNCS Description 24
3.3.1 Process Decomposition 24
3.3.2 Data Decomposition 35
3.4 LEDC Description 42
3.4.1 Process Decomposition 42
3.4.2 Data Composition 45
3.5 LIC Description 47
3.5.1 Process Decomposition 47
3.5.2 Data Decomposition 48
4.0 Dependency Description 51
4.1 PAC Description 51
4.1.1 Inter-process dependency – Data Flow Diagram 51
4.1.2 Interprocess Dependency-Structure Chart 63
4.2 CCP Description 71
4.2.1 Interprocess Dependency-Data Flow Diagram 71
4.2.2 Class Diagram 79
4.3 CNCS Description 97
4.3.1 Interprocess Dependency-Data Flow Diagram 97
4.3.2 Class Daigram 118
4.4 LEDC Description 138
4.4.1 Interprocess Dependency-Data Flow Diagram 138
4.4.2 Interprocess Dependency-Structure Chart 148
4.5 LIC Description 153
4.5.1 Interprocess Dependency-Data Flow Diagram 153
5.0 Interface Description 162
5.1 PAC Interface 162
5.1.1 External Interface 162
5.2 CCP Interface 167
5.2.1 External Interface 167
5.3 CNCS Interface 168

5.3.1 External Interface 168
5.4 LEDC Interface 173
5.4.1 External Interface 173
5.4.2 Internal Interface 174
5.5 LIC Interface 175
5.5.1 External Interface 175
6.0 Detailed Description 175
6.1 PAC Description 175
6.1.1 Module Detailed Design 175
6.1.2 Data Detailed Design 180
6.2 CCP Description 182
6.2.1 Module Detailed Design 182
6.2.2 Data Detailed Design 187
6.3 CNCS Description 193
6.3.1 Module Detailed Design 193
6.4 LEDC Description 226
6.4.1 Module Detailed Design 226
6.4.2 Data Detailed Design 228
6.5 LIC 231
6.5.1 Module Detailed Design 231

1. Introduction

1.1. Purpose

This document (software Design Description, here-in-after called ad 'SDD') to the RTD Silverliner V Communication System Configuration Unit for detailed design of the software and configure the Unit the narrative work together through the interface between implementation and system integration capabilities are intended.

This document shall be used by project Manager, configuration manager, system designer, architect and software developer through all over the software development lifecycle.

1.2. Scope

This document will be used for project team to prove that the system is functioning right, and all design specifications defined in this document are leading if there is no statement separately.

This document shall be used by project manager, configuration manager, system designer, and architect and software developer through all over the software development lifecycle. If there is no separate mention of all the design issues are defined by this article shall have precedence.

1.3. Acronyms and Abbreviations.

Acronyms & Abbreviations	Description
PID	LCD Passenger Information Display
LEDC	LED(FDI/SDI/PII) Control Board
PAC	Public Address Controller
CNCS	Content & Control Server
FDI	Front Destination Indicator
SDI	Side Destination Indicator
PII	LED Passenger Information Indicator
CCP	Communication Control Panel
MDS	~~Central Diagnostic Terminal~~
CRA	Communication Rack A
CS	Communication System

DPO	Driver PA Operation Box
GPS	Global Positioning System
LIC	LonWorks Interface Card
PA	Public Announcement
PEI	Passenger Emergency Intercom
PID	Passenger Information Display
TRS	Train Radio System
TX	Transmit
VOIP	

2. Reference

2.1. The following documents are referenced.

Publisher	Document Number	Title
DENVER	18FH012	DENVER RTD TS (Confirmed Version, March 2010) Part C Rolling Stock Requirements
IEEE	1016-1998	IEEE Recommended Practice for Software Design Descriptions
Rotem	REDU 106113	Procement Technical Specification For DENVER RTD EMU Communication System
WOOJIN	WJ-RTD-COMM-SRS-V1.0	DENVER RTD EAGLE P3(Communication System Software Requirements Specification)

3. Decomposition description

This section describes the division of the software into design entities. Each design entity is further divided into a distinct grouping of source code modules at the detailed design level.f

3.1. PAC Description

The CS includes two distinct PA Controllers which are housed in both CRA. These devices provide for the physical aspects of managing and distributing both Audio and Control Data within a specific vehicle, as well as amongst the various vehicles making up a Train Consist. The PAC's process and distribute all audio messaging in accordance with the parameters specified within the AVL messaging system, the Intercom system, and the PA system. PAC Control software provides the following functions.

3.1.1. Process Decomposition

PAC software is decomposed into 2 major software entities. Software entities are logical sub-groupings of software functions. Some entities are invoked by interrupt.

Figure 5 shows the software entities that comprise PAC software. The entities are arranged in 2 major groups and theses groups consist of several sub groups.

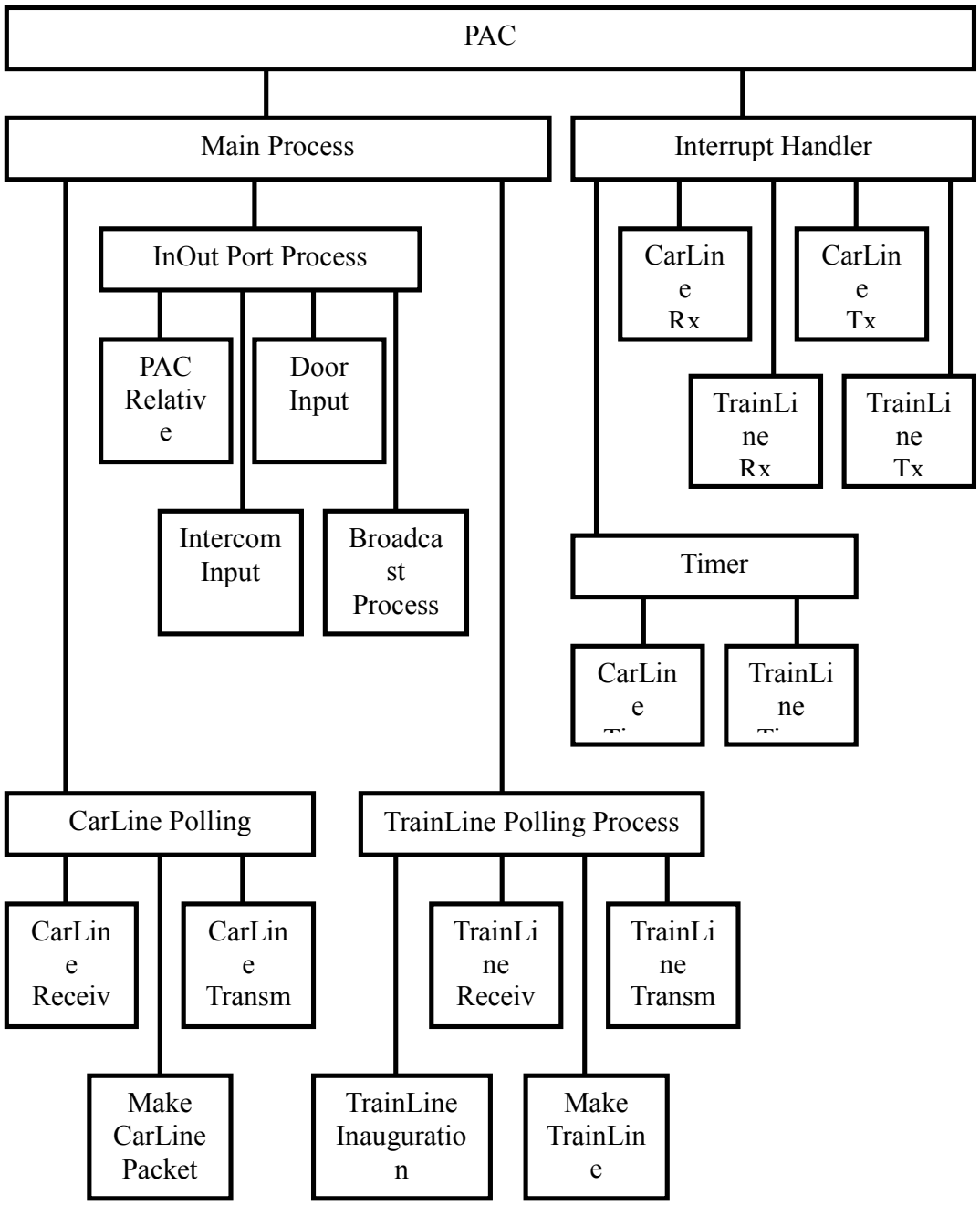


Figure 1. PAC Software Decomposition Diagram

3.1.1.1. Main Process

Identifier	PAC.Main.F
Function	Main Process initialize SW and HW and communicates with external input port. It also processes data to communicate between RS485 and each communication device. It constructs double headed train and communicates with each train. It processes DPO, PEI and auto announcement for PA.
Sub Entities	PAC.InOut.F PAC.CarLinePolling.F PAC.TrainLinePolling.F

3.1.1.1.1. InOut Process

Identifier	PAC.InOut.F
Function	Main process also saves the signals from married-car intercom port single car intercom, doors and train related from an incoming train and calls speakers port signal to check the functionality of announcement and priority signal to broadcast.
Sub Entities	PAC.RelativeInput.F PAC.DoorInput.F PAC.IntercomInput.F PAC.Broadcast.F

3.1.1.1.1.1. PAC Relative Input

Identifier	PAC.RelativeInput.F
Function	Train generated from PAC_ID, CAB, Conductor, Forward, Reverse, Uncouple signals are stored and read.
Sub Entities	

3.1.1.1.1.2. Door Input

Identifier	PAC.DoorInput.F
Function	Rea door status and resore.
Sub Entities	

3.1.1.1.1.3. Single Intercom Input

Identifier	PAC.SingleIntercomInput.F
Function	Read live and address signal of car intercom and restore.
Sub Entities	

3.1.1.1.1.4. Broadcast Process

Identifier	PAC.Broadcast.F
Function	Based on the evaluation of tone from intercom card and broadcasting status from port, it determines intercom, PEI and auto-announcement status.
Sub Entities	

3.1.1.1.2. CarLine Polling Process

Identifier	PAC.CarLinePolling.F
Function	Main PAC communicates each device connected via RS485 by polling method.
Sub Entities	PAC.CarLineReceive.F PAC.CarLineTransmit.F PAC.MakeCarLinePacket.F

3.1.1.1.2.1. CarLine Receive

Identifier	PAC.CarLineReceive.F
Function	It saves response packet sent from each device on protocol type after method.
Sub Entities	

3.1.1.1.2.2. CarLine Transmit

Identifier	PAC.CarLineTransmit.F
Function	Send packet to each device by polling sequence.
Sub Entities	

3.1.1.1.2.3. Make CarLine Packet

Identifier	PAC.MakeCarLinePacket.F
Function	Make packet for each devices by defined protocol based on restored information.
Sub Entities	

3.1.1.1.3. TrainLine Polling Process

Identifier	PAC.TrainLinePolling.F
Function	It reads each train information via RS485 and the polling method is used to control properly. The train information connected by double heading is sent to all double headed trains.
Sub Entities	PAC.Inauguration.F PAC.TrainLineReceive.F PAC.TrainLineTransmit.F PAC.MakeTrainLinePacket.F

3.1.1.1.3.1. Inauguration

Identifier	PAC.Inauguration.F
Function	Collect information of connected train.
Sub Entities	

3.1.1.1.3.2. TrainLine Receive

Identifier	PAC.TrainLineReceive.F
Function	The response packet sent to each double headed train is saved in protocol type after parsing.
Sub Entities	

3.1.1.1.3.3. TrainLine Transmit

Identifier	PAC.TrainLineTransmit.F
Function	Send packet to each multi-coupled train by polling sequence.
Sub Entities	

3.1.1.1.3.4. Make TrainLine Packet

Identifier	PAC.MakeTrainLinePacket.F
Function	Make packet for each devices by defined protocol based on restored information.
Sub Entities	

3.1.1.2. Interrupt Handler

Identifier	PAC.InterruptHandler.F
Function	By two RS485 carline transmitting and receiving and trainline transmitting and receiving are done. The transmitting and receiving is done in byte with RS485 communication for multi-tasking. Carling timer and trainline timer are used respectively to evaluate multi-tasking and polling.
Sub Entities	PAC.CarLineRxPacket.F PAC.TrainLineRxPacket.F PAC.CarLineTxPacket.F PAC.TrainLineTxPacket.F PAC.Timer.F

3.1.1.2.1. CarLine Rx Packet

Identifier	PAC.CarLineRxPacket.F
Function	Restore byte from CarLine RS485 in accurate location.
Sub Entities	

3.1.1.2.2. TrainLine Rx Packet

Identifier	PAC.TrainLineRxPacket.F
Function	The byte received by the TrainLine RS485 is stored in the exact location.
Sub Entities	

3.1.1.2.3. CarLine Tx Packet

Identifier	PAC.CarLineTxPacket.F
Function	Read the accurate byte from the packet stored in the CarLine transmission buffer and transmit.
Sub Entities	

3.1.1.2.4. TrainLine Tx Packet

Identifier	PAC.TrainLineTxPacket.F
Function	**Read t**he accurate byte from the packet stored in the TrainLine transmission buffer and transmit.
Sub Entities	

3.1.1.2.5. Timer Process

Identifier	PAC.Timer.F
Function	Create accurate polling time. Determine irresponsive device or train.
Sub Entities	PAC.CarLineTimer.F PAC.TrainLineTimer.F

3.1.1.2.5.1. CarLine Timer

Identifier	PAC.CarLineTimer.F
Function	It creates polling timing for CarLine RS 485 communication and discriminates un-responded device.
Sub Entities	

3.1.1.2.5.2. TrainLine Timer

Identifier	PAC.TrainLineTimer.F
Function	It creates polling timing for TrainLine RS 485 communication and discriminates un-responded train
Sub Entities	

3.1.2. Data Composition

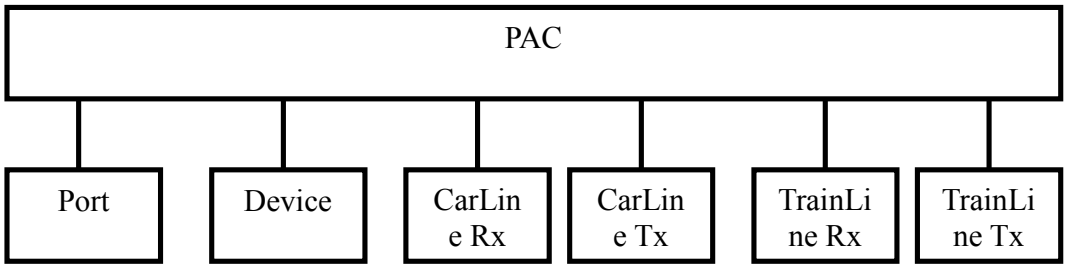


Figure 6. PAC software data decomposition diagram

3.1.2.1. Port

Identifier	PAC.PortInOut.D
Type	Global Variables
Purpose	Restore port signal of train relay, door, intercom signal.

3.1.2.2. Device

Identifier	PAC.DeviceData.D
Type	Global Variables
Purpose	Restore status information of devices and multi-coupled cat information.

3.1.2.3. CarLine Rx

Identifier	PAC.CarLineRx.D
Type	Global Variables
Purpose	CarLine Receive communication variable using interrupt.

3.1.2.4. CarLine Tx

Identifier	PAC.CarLineTx.D
Type	Global Variables
Purpose	CarLine 485 Transmit communication variable using interrupt.

3.1.2.5. TrainLine Rx

Identifier	PAC.TrainLineRx.D
Type	Global Variables
Purpose	TrainLine 485 Receive communication variable using interrupt.

3.1.2.6. TrainLine Tx

Identifier	PAC.TrainLineTx.D
Type	Global Variables
Purpose	TrainLine 485 Transmit communication variable using interrupt.

3.2. CCP Description

3.2.1. Process Decomposition

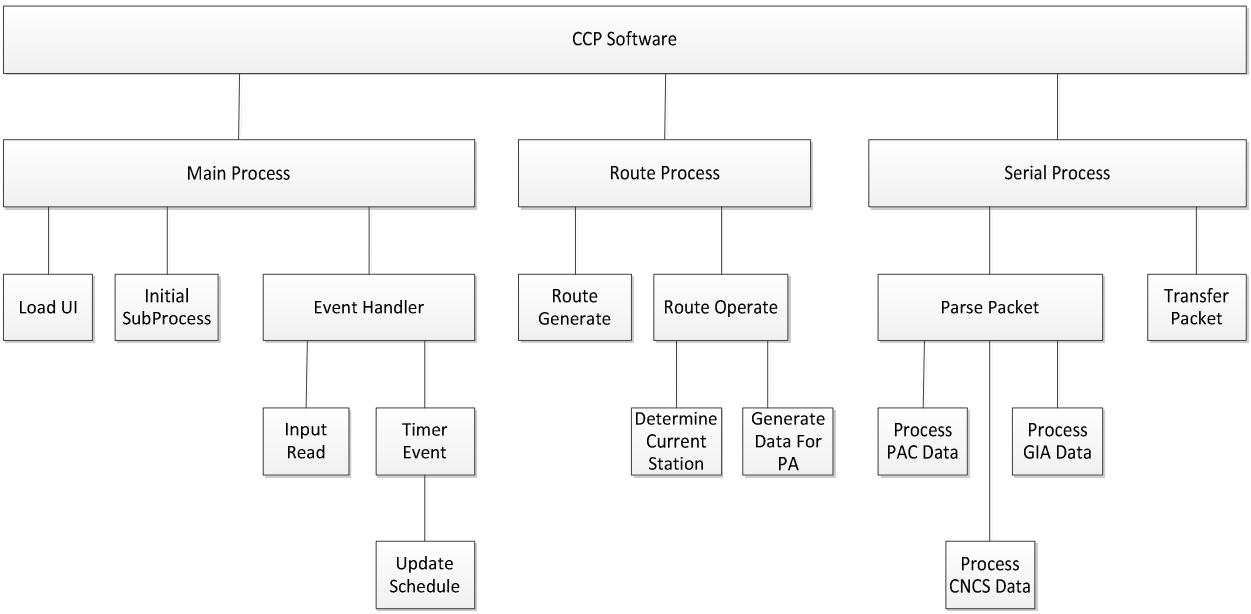


Figure2. CCP software decomposition diagram

3.2.1.1. Main Process

Identifier	CCP.MainProcess.F

Function	MainProcesss loads resources related to UI of CCP system, and runs sub-process.
Sub Entities	CCP.LoadUI.F CCP.InitialSubProcess.F CCP.EventHandler.F

3.2.1.1.1. LoadUI

Identifier	CCP.LoadUI.F
Function	It loads image resources for UI output, creates UI control, and fills up graphic buffer.
Sub Entities	None

3.2.1.1.2. Initial Sub Process

Identifier	CCP.InitailSubProcess.F
Function	It runs sub-process that will be runs against MainProcess.
Sub Entities	

3.2.1.1.3. EventHandler

Identifier	CCP.EventHandler.F
Function	It performs user input and related event function.
Sub Entities	CCP.InputRead.F CCP.TimerEvent.F

3.2.1.1.4. Input Read

Identifier	CCP.InputRead.F
Function	It performs for user to input TrainNumber and CrewID via UI.
Sub Entities	

3.2.1.1.5. Timer Event

Identifier	CCP.TimerEvent.F
Function	It processes the events occured every regularly scheduled time.

Sub Entities	CCP.UpdateSchedule.F

3.2.1.1.6. Update Schedule

Identifier	CCP.UpdateSchedule.F
Function	It updates the schedule information which is set on Route Process every regularly scheduled time.
Sub Entities	

3.2.1.2. Route Process

Identifier	CCP.Route.Process.F
Function	It performs router setting and operation function.
Sub Entities	CCP.RouteGenerate.F CCP.RouteOperate.F

3.2.1.2.1. Route Generate

Identifier	CCP.RouteGenerate.F
Function	It creates the path for TrainNumber entered by loading schedule DB.
Sub Entities	

3.2.1.2.2. Route Operate

Identifier	CCP.RouteOperate.F
Function	It searches the train station based on the designated path correspond to GPS location.
Sub Entities	CCP.DetermineCurrentStation.F CCP.GenerateDataForPA.F

3.2.1.2.3. Determine Current Station

Identifier	CCP.DetermineCurrentStation.F
Function	It searches current station information based on GPS coordinate.
Sub Entities	

3.2.1.2.4. Generate Data for PA

Identifier	CCP.GenerateDataForPA.F
Function	It creates information for PA based on current station information.
Sub Entities	

3.2.1.3. Serial Process

Identifier	CCP.SerialProcess.F
Function	It process serial data received from external devices.
Sub Entities	CCP.ParsePacket.F CCP.TransferPakcket.F

3.2.1.3.1. Parse Packet

Identifier	CCP.ParsePacket.F
Function	It structuralizes to classify packets received as trasmission devices.
Sub Entities	CCP.ProcessPACData.F CCP.ProcessGIA_DATAData.F CCP.ProcessCNCSData.F

3.2.1.3.2. Process PAC Data

Identifier	CCP.ProcessPACData.F
Function	It handles packet received from PAC.
Sub Entities	

3.2.1.3.3. Process CNCS Data

Identifier	CCP.ProcessCNCSData.F
Function	It handles packet received form CNCS.
Sub Entities	

3.2.1.3.4. Transfer Packet

Identifier	CCP.TransferPacket.F
Function	It transmits packet to external system.
Sub Entities	

3.2.2. Data Decomposition

Data used by CCP software is decomposed into 4 major data entities as figure 3.

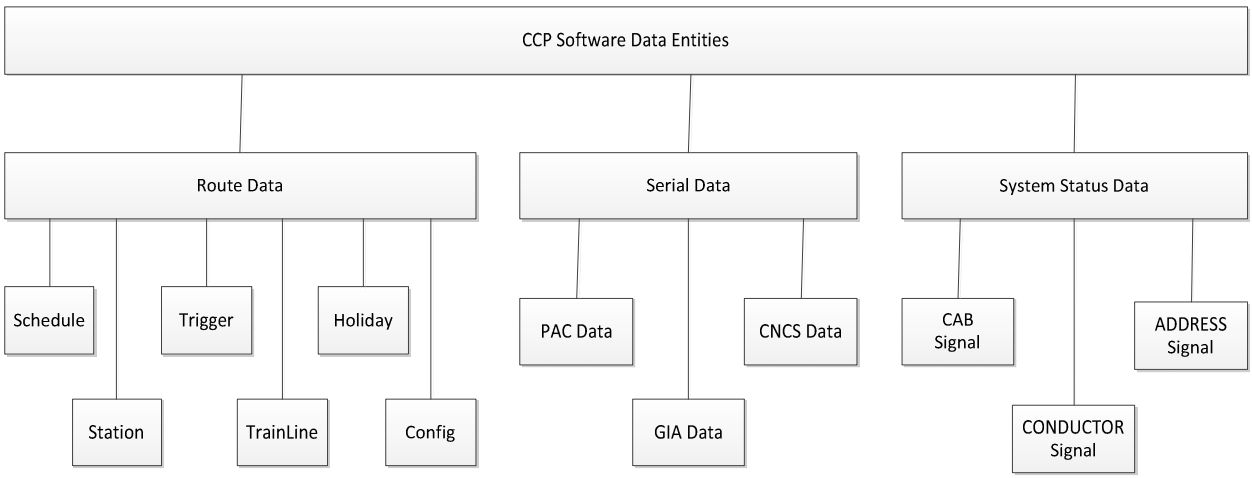


Figure 3. CCP software data decomposition diagram

3.2.2.1. Route Data

Identifier	CCP.RouteData.D
Type	Class
Purpose	The data which setting for train operation route

3.2.2.1.1. Schedule

Identifier	CCP.Schedule.D
Type	Class
Purpose	The data which is stored all train operation information

3.2.2.1.2. Station

Identifier	CCP.Station.D
Type	Class
Purpose	The data which is stored all train station information

3.2.2.1.3. Trigger

Identifier	CCP.Trigger.D

Type	Class
Purpose	The data which is stored all location coordinate of station

3.2.2.1.4. TrainLine

Identifier	CCP.TrainLine.D
Type	Class
Purpose	The data which is stored route information

3.2.2.1.5. Holiday

Identifier	CCP.Holiday.D
Type	Class
Purpose	The data which is stored holiday route alternation information

3.2.2.1.6. Config

Identifier	CCP.Config.D
Type	Class
Purpose	The data which is stored announcement information of special train number

3.2.2.2. Serial Data

Identifier	CCP.SerialData.D
Type	Class
Purpose	The data for packet communication

3.2.2.2.1. PAC Data

Identifier	CCP.PACData.D
Type	Class
Purpose	The data for communication with PAC

3.2.2.2.2. CNCS Data

Identifier	CCP.CNCSData.D
Type	Class
Purpose	The data for communication with CNCS

3.2.2.2.3. GIA_DATA Data

Identifier	CCP.RouteData.D
Type	Class
Purpose	The data for communicate with GIA_DATA

3.2.2.2.4. System Status Data

Identifier	CCP.SystemStatus.Data
Type	Class
Purpose	The data for status information of system

3.2.2.2.5. Cab Signal

Identifier	CCP.CabSignal.D
Type	Class
Purpose	The data to save Cab signal information

3.2.2.2.6. Conductor Signal

Identifier	CCP.ConductorSignal.D
Type	Class
Purpose	The data to save Conductor signal information

3.2.2.2.7. Address Signal

Identifier	CCP.AddressSignal.D

Type	Class
Purpose	The data to save Address signal information

3.3. CNCS Description

CNCS operation program can manage both display and data of the information of the stations via the interface between the devices and has a fuction to store MDS data and Wayside's contents data into the vehicles and update sub-systems' contents.

3.3.1. Process Decomposition

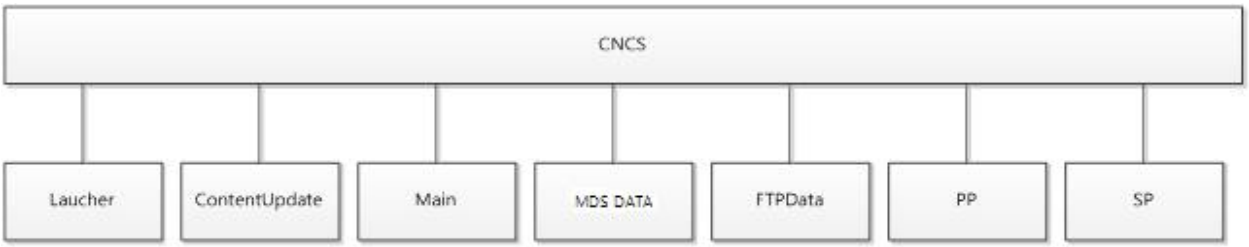


Figure 4. CNCS Software Decomposition Diagram

3.3.1.1. Launcher

3.3.1.1.1. Description

Launcher is responsible for updating on the operation program of CNCS system.

3.3.1.1.2. Launcher Process Decomposition

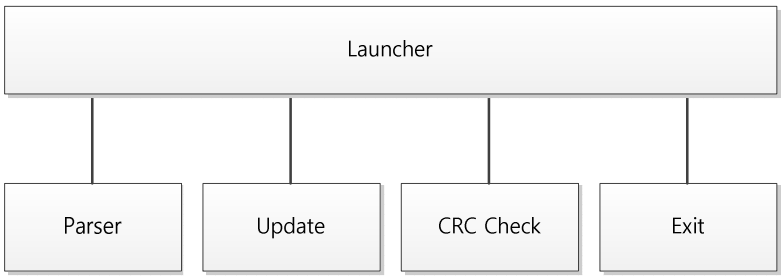


Figure 5. Launcher Process Decomposition Diagram

3.3.1.1.3. **Launcher**

Identifier	Launcher.F
Function	Launcher Process updates CNCS operating program when runs CNCS
Sub Entities	Parser.F Update.F CRC Check.F Exit.F

3.3.1.1.4. **Paresr**

Identifier	Launcher.Parser.F
Function	Parser updates Launcher.exe program of CNCS.
Sub Entities	None

3.3.1.1.5. **Update**

Identifier	Launcher.Update.F
Function	CNCS operational program update.
Sub Entities	None

3.3.1.1.6. Update

Identifier	Launcher.Update.F
Function	Parser updates Launcher.exe program of CNCS
Sub Entities	None

3.3.1.1.7. CRC Check

Identifier	Launcher.CRC Check.F
Function	Check if CRC of update data is normal condition.
Sub Entities	None

3.3.1.1.8. Exit Process

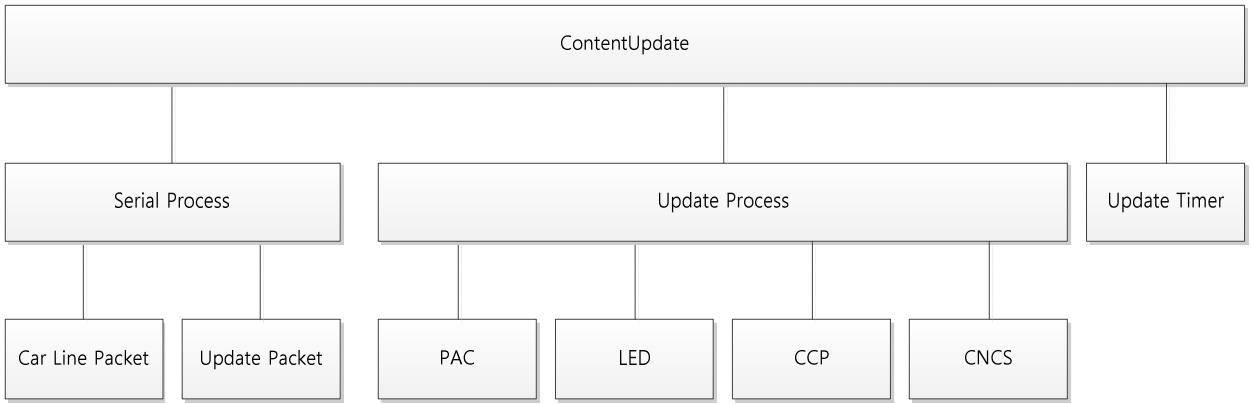
Identifier	Launcher.Exit.F
Function	Conduct ContentUpdate Processor after Launcher processor shutdown.
Sub Entities	None

3.3.1.2. Content Update

3.3.1.2.1. Description

Content Update administers each devices content update of CS.

3.3.1.2.2. Process Decomposition



F

igure 6. Content Update Process Decomposition Diagram

3.3.1.2.3. ContentUpdate

Identifier	ContentUpdate.F
Function	It downloads contents data to each sub-device which received from Wayside.
Sub Entities	Serial Process.F Update Process.F Update Timer.F

3.3.1.2.4. Serial Process

Identifier	Serial Process.F
Function	It acquires authority from PAC through Car Line and it downloads

	contents update data to each sub-device.
Sub Entities	Car Line Packet.F Update Packet.F

3.3.1.2.4.1. CarLine Packet Process

Identifier	Car Line Packet.F
Function	It checks automatic vehicular status using Car Line and gets CSNCS authority from PAC.
Sub Entities	None

3.3.1.2.4.2. Update Data Process

Identifier	Update Packet.F
Function	It updates contents data of sub-device using Car Line.
Sub Entities	None

3.3.1.2.5. Update Process

Identifier	Update Process.F
Function	It updates contents of each sub-device.
Sub Entities	PAC.F LED.F CCP.F CNCS.F

3.3.1.2.5.1. PAC

Identifier	PAC.F
Function	It updates audio contents of PAC.
Sub Entities	None

3.3.1.2.5.2. LED

Identifier	LED.F
Function	It updates LED display of FDI, SDI and PII contents.

Sub Entities	None

3.3.1.2.5.3. CCP

Identifier	CCP.F
Function	It updates CCP of operation scheduler, TP and station information contents.
Sub Entities	None

3.3.1.2.5.4. CNCS

Identifier	CNCS.F
Function	It updates PP and SP contents data of CNCS.
Sub Entities	None

3.3.1.2.6. Update Timer

Identifier	Update Timer.F
Function	It counts time necessary for the sub-system when the content update and occursTime Out when errors during the update.
Sub Entities	None

3.3.1.3. Main

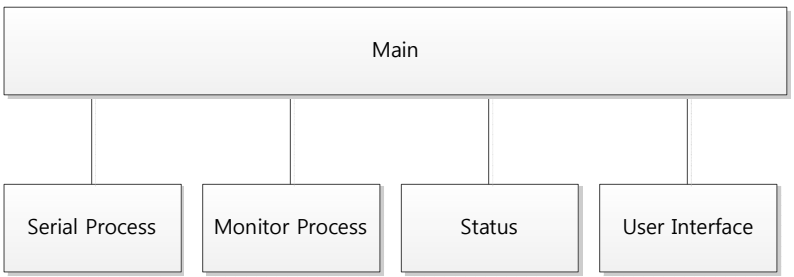


Figure 7. Main Process Decomposition Diagram

3.3.1.3.1. Main

Identifier	Main.F
Function	It is responsible for both CNCS Car Line communication and monitoring MDS Data, FTP Data, PP and SP program.

Sub Entities	Serial Process.F Monitor Process.F Status.F User Interface.F

3.3.1.3.2. Serial Process

Identifier	Serial Process.F
Function	It manages Car Line communication reception and responding.
Sub Entities	None

3.3.1.3.3. Monitor Process

Identifier	Monitor Porcess.F
Function	Manages sub-processor monitoring using UDP communication.
Sub Entities	None

3.3.1.3.4. Status

Identifier	Status.F
Function	Monitors status information using Car Line information.
Sub Entities	None

3.3.1.3.5. User Interface

Identifier	User Interface.F
Function	Provides interface between user and CNCS.
Sub Entities	None

3.3.1.4. MDSData

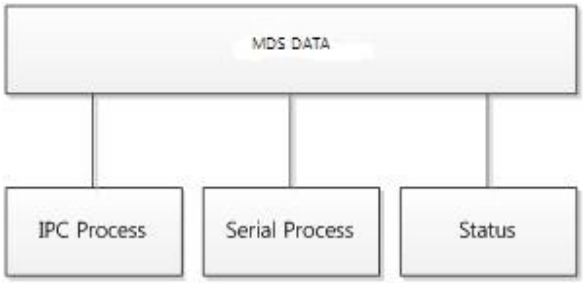


Figure 8. MDS Data Process Decomposition Diagram

3.3.1.4.1. MDSData

Identifier	MDSData.F
Function	Controls operation between MDS and LIC.
Sub Entities	IPC Process.F Serial Process.F Status.F

3.3.1.4.2. IPC Process

Identifier	IPC Process.F
Function	It receives train Wayside status information from Main Process or returns MDSData work state.
Sub Entities	None

3.3.1.4.3. Serial Process

Identifier	Serial Process.F
Function	Manages Serial communication between LIC and CNCS.
Sub Entities	None

3.3.1.4.4. Status

Identifier	Status.F
Function	Manages status information between LIC and CNCS.

Sub Entities	None

3.3.1.5. FTPData

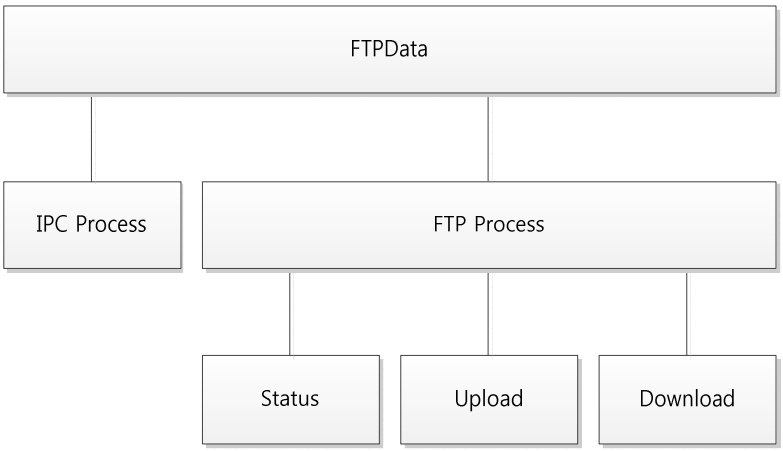


Figure 9. FTP Data Process Decomposition Diagram

3.3.1.5.1. FTP Data

Identifier	FTPData.F
Function	Manages E-NET communication between Wayside and CNCS.
Sub Entities	IPC Process FTP Process

3.3.1.5.2. IPC Process

Identifier	IPC Process.F
Function	It receives train Wayside status information from Main Process or returns MDSData work state.
Sub Entities	None

3.3.1.5.3. FTP Process

Identifier	FTP Process.F
Function	Controls operation between CNCS and FTP Server.
Sub Entities	Status.F Update.F

	Download.F

3.3.1.5.4. Status

Identifier	Status.F
Function	Manages connection status with FTP server.
Sub Entities	None

3.3.1.5.5. Upload

Identifier	Upload.F
Function	It uploads both MDS failure data and automatic updates log records collected by CNCS to FTP server.
Sub Entities	None

3.3.1.5.6. Download

Identifier	Downalod.F
Function	It downloads the contents data registered on FTP server with CNCS.
Sub Entities	None

3.3.1.6. PP

3.3.1.6.1. PP Process Decomposition.

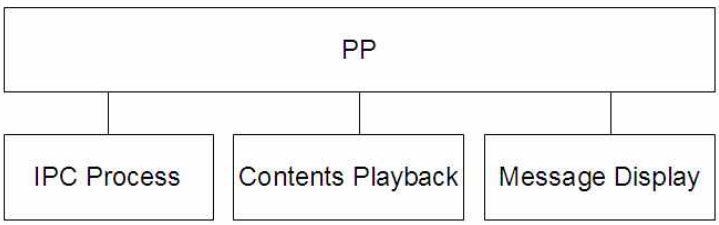


Figure 10. PP Process Decomposition Diagram

3.3.1.6.2. PP Process.

Identifier	PP.F

Function	It is responsible for a display of videos, still image contents' play, passenger guide words and optional information.
Sub Entities	IPC Process , Comtents Playback , Message Display

3.3.1.6.3. IPC Process.

Identifier	IPC Process.F
Function	It receives the information from the main process and transmits the state data.
Sub Entities	None

3.3.1.6.4. Contents Playback

Identifier	Comtents Playback.F
Function	It is responsible for playing of videos and still image contents.
Sub Entities	None

3.3.1.6.5. Message Display

Identifier	Message Display.F
Function	It is responsible for showing passenger guide words and optional information.
Sub Entities	None

3.3.1.7. SP

3.3.1.7.1. SP Process Decomposition

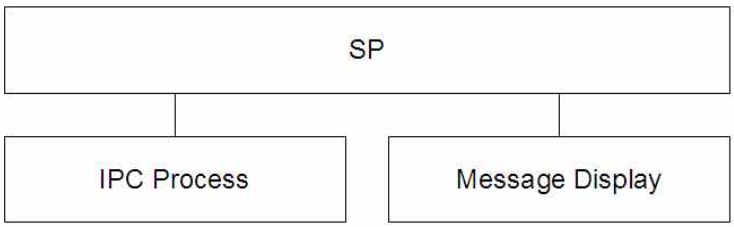


Figure 11. SP Process Decomposition Diagram

3.3.1.7.2. SP Process.

Identifier	SP.F
Function	It is responsible for a display of passenger guide message.
Sub Entities	IPC Process , Message Display

3.3.1.7.3. IPC Process

Identifier	IPC Process.F
Function	It receives the information from the main process and transmits the state data.
Sub Entities	None

3.3.1.7.4. Message Display

Identifier	Message Display.F
Function	It is responsible for showing passenger guide message.
Sub Entities	None

3.3.2. Data Decomposition

3.3.2.1. Launcher

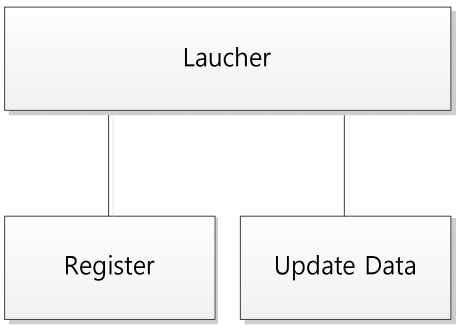


Figure 12. CNCS software data decomposition diagram

3.3.2.1.1. Register

Identifier	Register.D
Type	Register
Purpose	Information data to update CNCS's Launcher program.

3.3.2.1.2. Update Data

Identifier	Update Data.D
Type	Executable file
Purpose	Launcher.exe, ContentUpdate.exe, Main.exe, MDSData.exe, FTPData.exe, PP.Exe and SP.Exe data files to update the program being operated in CNCS.

3.3.2.2. Content Update

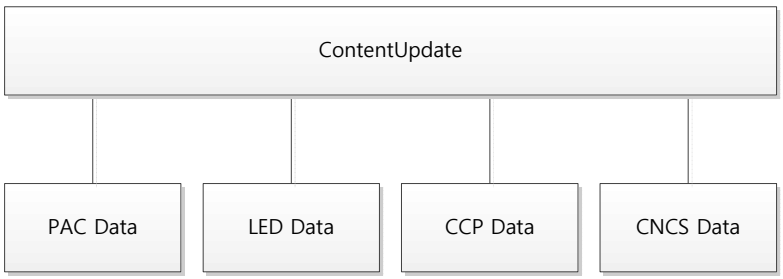


Figure 13. Content Update Process Data Decomposition Diagram

3.3.2.2.1. PAC Data

Identifier	PAC Data.D
Type	Mp3
Purpose	MP3 content file for PAC broadcasting.

3.3.2.2.2. LED Data

Identifier	LED Data.D
Type	Text
Purpose	Data file for FDI, SDI and PII content update.

3.3.2.2.3. CCP Data

Identifier	CCP Data.D
Type	Text
Purpose	Update file for CCP operation schedule information.

3.3.2.2.4. CNCS Data

Identifier	CNCS Data.D
Type	Text, Movie
Purpose	Data file to update CNCS's PP, SP and video contents.

3.3.2.3. Main

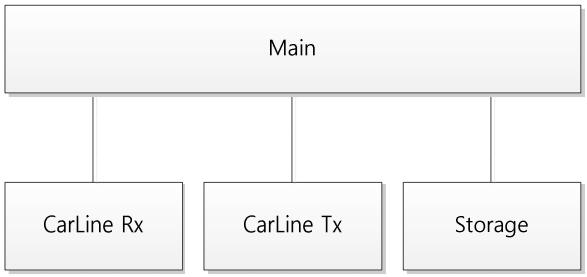


Figure 14. Main Process Data Decomposition Diagram

3.3.2.3.1. CarLine Rx

Identifier	CarLineRx.D
Type	Byte
Purpose	Each device data of Car Line

3.3.2.3.2. CarLine Tx

Identifier	CarLineTx.D
Type	Byte
Purpose	CNCS status data through Car Line

3.3.2.3.3. Storage

Identifier	Storage.D
Type	Global Variables
Purpose	Set point data which need for CNCS operation.

3.3.2.4. MDSData

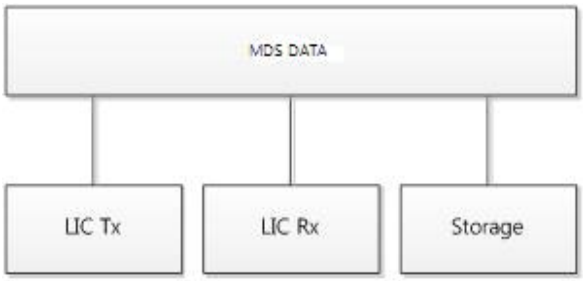


Figure 15. MDS Data Process Decomposition Diagram

3.3.2.4.1. LIC Tx

Identifier	LIC Tx.D
Type	Variable
Purpose	LIC requested data through RS232

3.3.2.4.2. LIC Rx

Identifier	LIC Rx.D
Type	Variable
Purpose	Response data that was requested from LIC through Rs232.

3.3.2.4.3. Storage

Identifier	Storage.D
Type	Binary File
Purpose	Stores MDS failure data which received from LIC.

3.3.2.5. FTPData

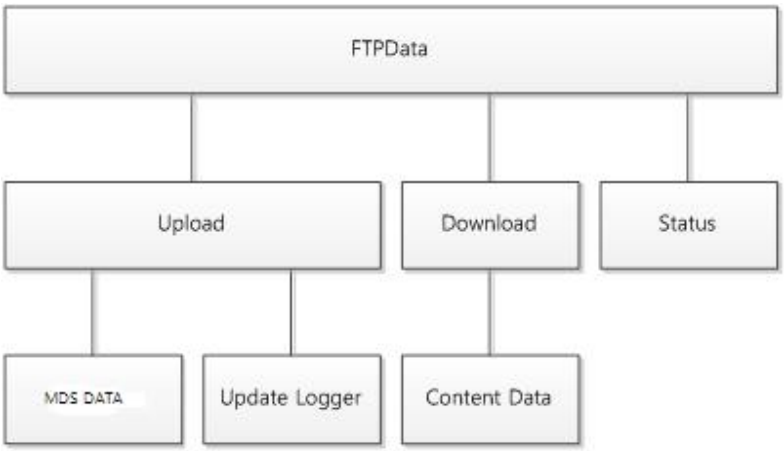


Figure 16. FTP Data Process data Decomposition Daigram

3.3.2.5.1. MDSData

Identifier	MDS Data.D
Type	Binary File
Purpose	Failure data file that is collected from MDS to transmit to FTP server.

3.3.2.5.2. Update Logger

Identifier	Update Logger.D
Type	Text File
Purpose	Automatic-updates log result file to transmit to FTP server.

3.3.2.5.3. Content Data

Identifier	Content Data.D
Type	Text, Movie File
Purpose	Data file downloaded by each sub-system as a contents file registered on FTP server,

3.3.2.5.4. Status

Identifier	Status.D

Type	Binanry
Purpose	Manages connection status between FTP Server and CNCS.

3.3.2.6. PP

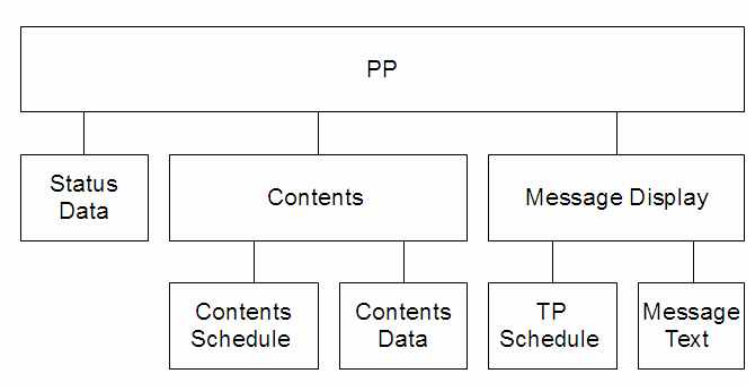


Figure 17. PP Process Decomposition Daigram

3.3.2.6.1. Contents Data

Identifier	Contents Data.D
Type	Movie File, Still Image File
Purpose	Video and still image file to be played.

3.3.2.6.2. Contents Schedule

Identifier	Contents Schedule .D
Type	Text File
Purpose	File that has a play list of contents to be played.

3.3.2.6.3. TP Schedule

Identifier	TP Schedule .D
Type	Text File
Purpose	File that has a display schedule for a relevant trigger point.

3.3.2.6.4. Message Text

Identifier	Message Text .D

Type	Text File.
Purpose	File that has a passenger guide message.

3.3.2.6.5. Status

Identifier	Status .D
Type	Binary.
Purpose	Data that both receives and transmits trigger points and other statuses.

3.3.2.7. SP

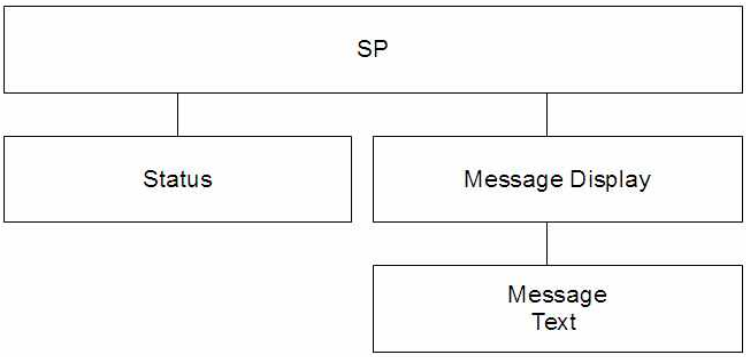


Figure 18. SP Process Decomposition Daigram

3.3.2.7.1. Message Text

Identifier	Message Text .D
Type	Text File
Purpose	It contains a guide message for a relevant trigger point.

3.3.2.7.2. Status

Identifier	Status .D
Type	Binary.
Purpose	It is data that both receives and transmits trigger points and other statuses.

3.4. LEDC Description

3.4.1. Process Decomposition

LEDC software is decomposed into 3 major software entities. Software entities are logical sub-groupings of software functions. Some entities are invoked by interrupt.

Figure 19 shows the software entities that comprise LEDC software. The entities are arranged in 3 major groups and theses groups consist of several sub groups.

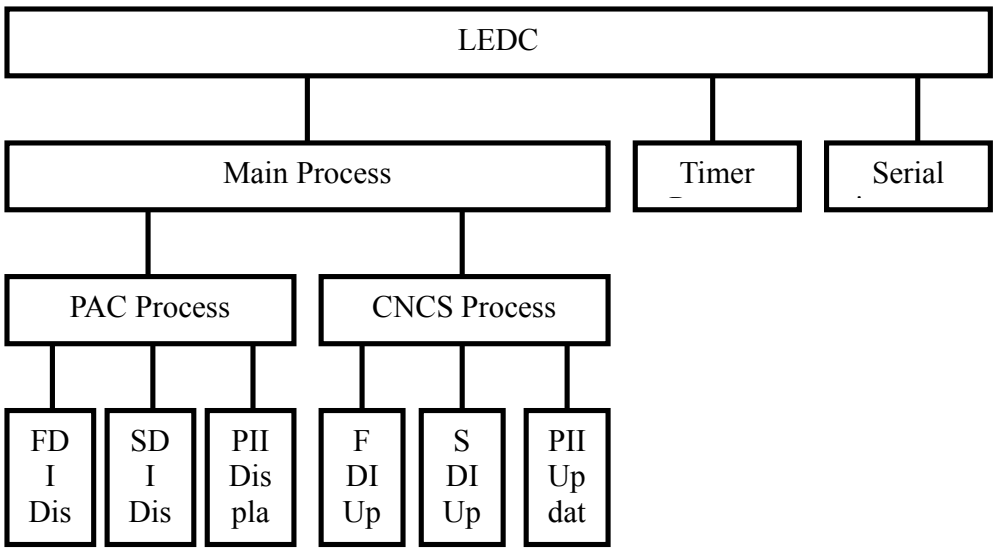


Figure 19. LEDC software decomposition diagram

3.4.1.1. Main Process

Identifier	LEDC.MainProcess.F
Function	The Main Process sets FDI, SDI, PII and PII of hardware and software initial value. And implements sub-PAC Process and CNCS Process.
Sub Entities	LEDC.PACProcess.F LEDC.CNCSProcess.F

3.4.1.1.1. PAC Process

Identifier	LEDC.PACProcess.F
Function	PAC Process interprets transmitted data from PAC Unit. The interpreted data by selection is sent to FDI display process, SDI display process, and PII display process by initial value set in main process.

Sub Entities	LEDC.FDIDisplayProcess.F LEDC.SDIDisplayProcess.F LEDC.PIIDisplayProcess.F

3.4.1.1.1.1. FDI Display Process

Identifier	LEDC.FDIDisplayProcess.F
Function	The FDI display process is that data is displayed on LED unit from image storage by using the data from PAC process.
Sub Entities	-

3.4.1.1.1.2. SDI Display Process

Identifier	LEDC.SDIDisplayProcess.F
Function	The DSI display process is that data is displayed on LED unit from image storage by using the data from PAC process.
Sub Entities	-

3.4.1.1.1.3. PII Process

Identifier	LEDC.PIIDisplayProcess.F
Function	The PII display process is that data is displayed on LED unit from image storage by using the data from PAC process.
Sub Entities	-

3.4.1.1.2. CNCS Process

Identifier	LEDC.CNCSProcess.F
Function	CNCS process interprets received data from CNCS unit. The interpreted data by selection is sent to FDI display process, SDI display process, and PII display process by initial value set in main process.
Sub Entities	LEDC.FDIUpdateProcess.F LEDC.SDIUpdateProcess.F LEDC.PIIUpdateProcess.F

3.4.1.1.2.1. FDI Update Process

Identifier	LEDC.FDIUpdateProcess.F
Function	FDI update process save received data from CNCS process in FDI

	image space of image storage.
Sub Entities	-

3.4.1.1.2.2. SDI Update Process

Identifier	LEDC.SDIUpdateProcess.F
Function	SDI update process save received data from CNCS process in SDI image space of image storage.
Sub Entities	-

3.4.1.1.2.3. PII Update Process

Identifier	LEDC.PIIUpdateProcess.F
Function	PII update process save received data from CNCS process in PII image space of image storage.
Sub Entities	-

3.4.1.2. Timer Process

Identifier	LEDC.TimerProcess.F
Function	The image scroll timer value is sent to PII display process by the timer interrupt event occurred in every 10ms on timer process.
Sub Entities	-

3.4.1.3. Serial Interrupt Process

Identifier	LEDC.SerialInterruptProcess.F
Function	Serial Interrupt process connects LEDC unit and CNCS unit. Saves received data to Rx buffer, and transmits TX buffer data.
Sub Entities	-

3.4.2. Data Composition

Data used by LEDC software is decomposed into 3 major data entities as figure 20.

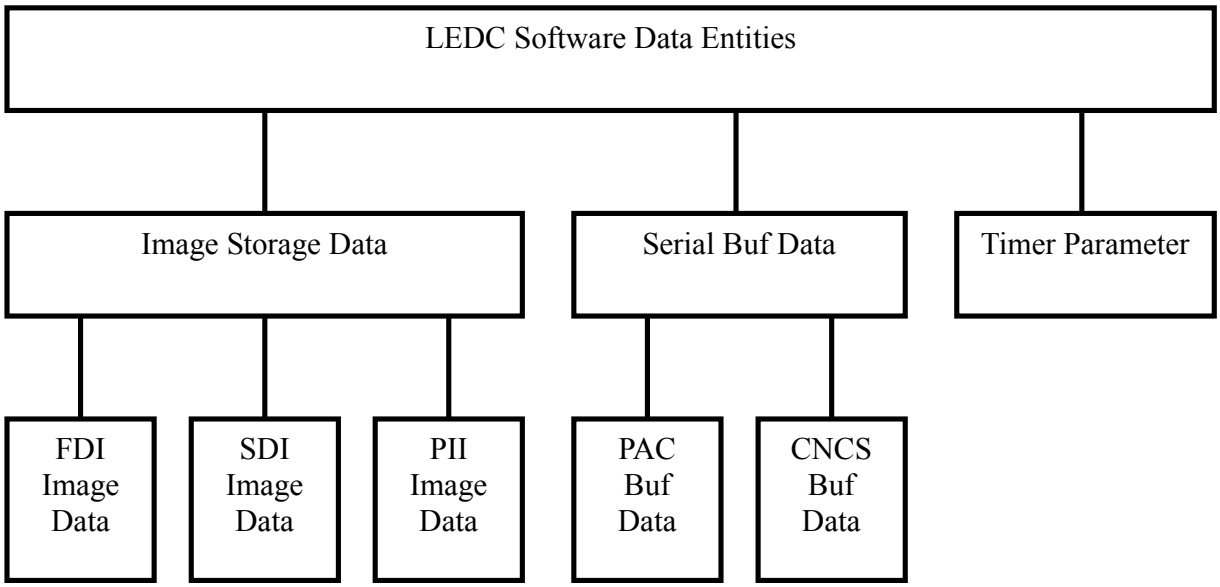


Figure 20. LEDC software data decomposition diagram

3.4.2.1. Image Storage Data

Identifier	ImageStorageData.D
Type	FLASH MEMORY
Purpose	In image storage data, FDI image data, SDI image data, and PII image data received from CNCS unit are saved and printed on FDI display circuit, SDI display circuit, and PII display circuit.

3.4.2.1.1. FDI Storage Data

Identifier	FDIImageData.D
Type	FLASH MEMORY
Purpose	FDI Image date received from CNCS Unit is saved in Image Data and is printed as FDI Display Circuit by FDI Display Process through receiving a command from PAC Unit.

3.4.2.1.2. SDI Image Data

Identifier	SDIImageData.D

Type	FLASH MEMORY
Purpose	In SDI image data, the SDI image data received from CNCS units us saved. It is printed on SDI display circuit by SDI display process commanded by PAC unit.

3.4.2.1.3. PII Image Data

Identifier	PIIImageData.D
Type	FLASH MEMORY
Purpose	In PII image data, the PII image data received from CNCS units us saved. It is printed on SDI display circuit by PII display process commanded by PAC unit.

3.4.2.2. Serial Buf Data

Identifier	SerialBufData.D
Type	Structure
Purpose	Received data and reception is stored when Serial Buf Data Structure is received data from PAC unit or CNCS unit. Transmitting data and transmission status data is stored when transmitting data.

3.4.2.2.1. PAC Buf Data

Identifier	PACBufData.D
Type	Structure
Purpose	Received data and reception is stored when PAC Buf Data Structure is received data from PAC unit. Transmitting data and transmission status data is stored when transmitting data.

3.4.2.2.2. CNCS Buf Data

Identifier	CNCSBufData.D
Type	Structure
Purpose	Received data and reception status is stored when CNCS Buf Data Structure is received data from CNCS unit. Transmitting data and transmission status data is stored when transmitting data.

3.4.2.3. Timer Parameter

Identifier	TimerParameter.D

Type	Structure
Purpose	The PII image scroll timer value is saved by the timer interrupt event occurred in every 10ms on timer process.

3.5. LIC Description

LIC operation program transferes device state information via MDS device and interface and receives vehicles failure date then transmit as CNCS.

3.5.1. Process Decomposition

LIC consists of a single process and each Process Decomposition is listed with the extra details

3.5.1.1. MAIN

3.5.1.1.1. Description.

Main is in charge of LIC opertaion program.

3.5.1.1.2. Main process decomposition

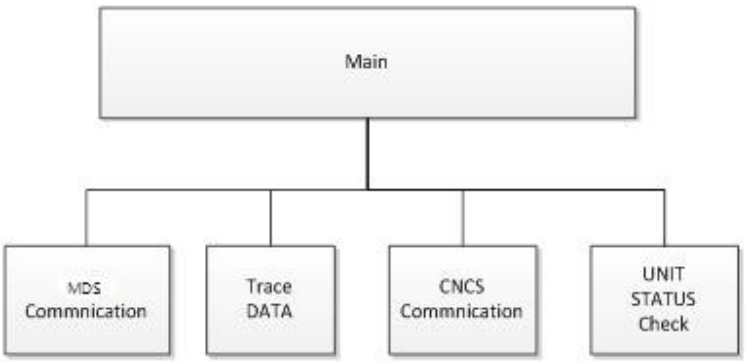


Figure 21. Main Process Decomposition Diagram

3.5.1.1.3. MDS Communication

Identifier	MDS.Commnication.F
Function	- MDS Communication regularly communicates MDS device. - It receives a failure data from MDS - It transmits vehicle device status to MDS
Sub Entities	None

3.5.1.1.4. Trace DATA

Identifier	TraceData.F
Function	- It stores the data received from MDS
Sub Entities	None

3.5.1.1.5. CNCS Communication

Identifier	TraceData.F
Function	- Trace Data LIC → CNCS - GPA time information → LIC
Sub Entities	None

3.5.1.1.6. UNIT State Communication

Identifier	State.Communication.F
Function	It receives device status information.
Sub Entities	None

3.5.2. Data Decomposition

3.5.2.1. MDS Data

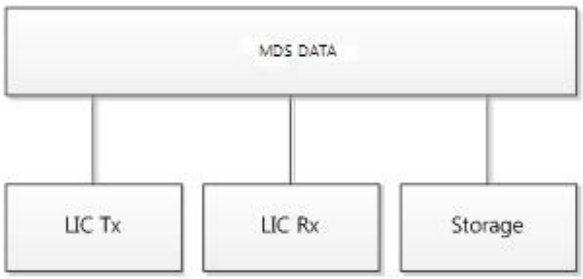


Figure 22. LIC Software Data Decomposition Diagram

3.5.2.1.1. MDS Tx

Identifier	MDS Tx.D
Type	Variable

Purpose	It transmits every device status via LonWorks communication.

3.5.2.1.2. MDS Rx

Identifier	MDS Rx.D
Type	Variable
Purpose	It receives vehicle failure data via LonWorks communication.

3.5.2.1.3. Storage

Identifier	Storage.D
Type	Binary File
Purpose	It stores MDS's failure data.

3.5.2.2. PAC Data

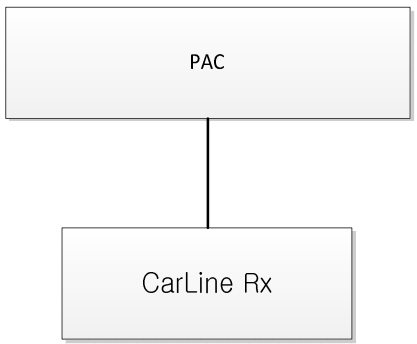


Figure 23. PAC Data Process Decomposition Daigram

3.5.2.2.1. PAC Rx.

Identifier	CarLineRx.D
Type	Byte
Purpose	It receives each device status data of Car Line.

3.5.2.3. CNCS Data

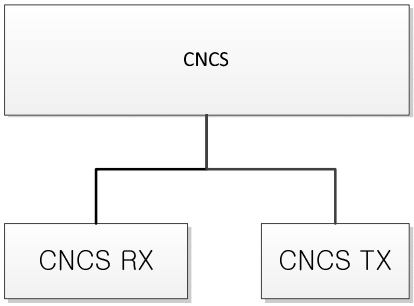


Figure 24. CNCS Data Decomposition Daigram

3.5.2.3.1. CNCS Tx

Identifier	CNCS Tx.D
Type	Variable
Purpose	To respond the data which receives LIC requests via Rs232.

3.5.2.3.2. CNCS Rx

Identifier	CNCS Rx.D
Type	Variable
Purpose	Response data of the data that receives requests from LIC via RS232.

4. Dependency Description

This section describes dependency of GIA_DATA_DATA software design entities using structure chart and data flow diagram.

4.1. PAC Description

This section describes dependency of PAC software design entities using structure chart and data flow diagram.

4.1.1. Inter-process dependency – Data Flow Diagram

In this subsection to describe relationship among entities DFD is used.

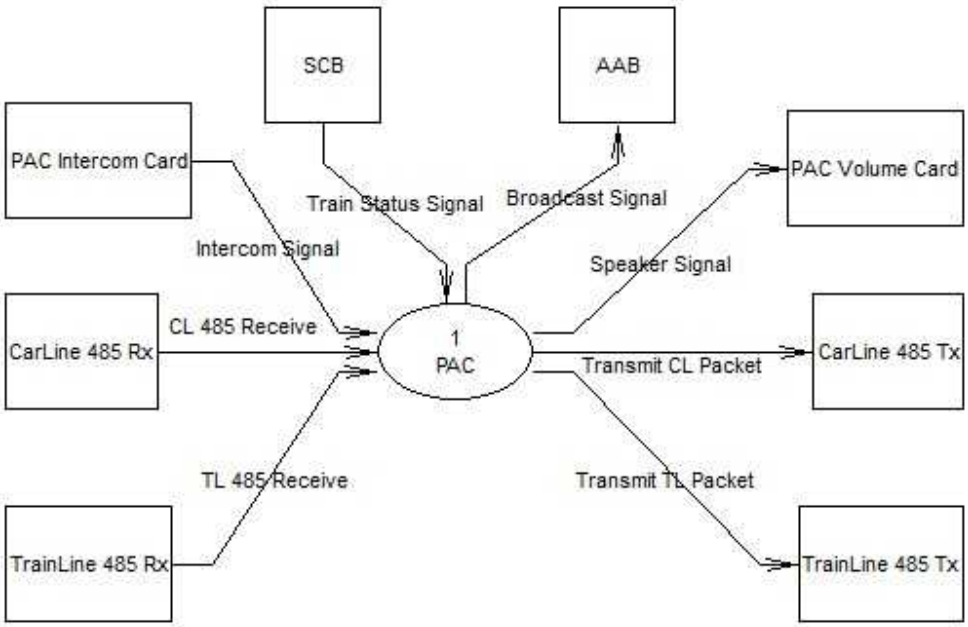


Figure 25. DFD level-0 (Context Diagram)

4.1.1.1. PAC

Identifier	PAC		
Purpose	It controls broadcasting by evaluating door and intercom signal occurred in the train works as a 485 communication master of carline. It communicates trainline 485 to process the information of double headed train and reads the device information at device storage to communicate with carline devices.		
Source	SCB	Data	Train Status Signal
	PAC Intercom Card		Intercom Signal

	CarLine 485 Rx		CL 485 Receive
	TrainLine 485 Rx		TL 485 Receive
Target	AAB	Data	Broadcast Signal
	PAC Volume Card		Speaker Signal
	CarLine 485 Tx		Transmit CL Packet
	TrainLine 485 Tx		Transmit TL Packet

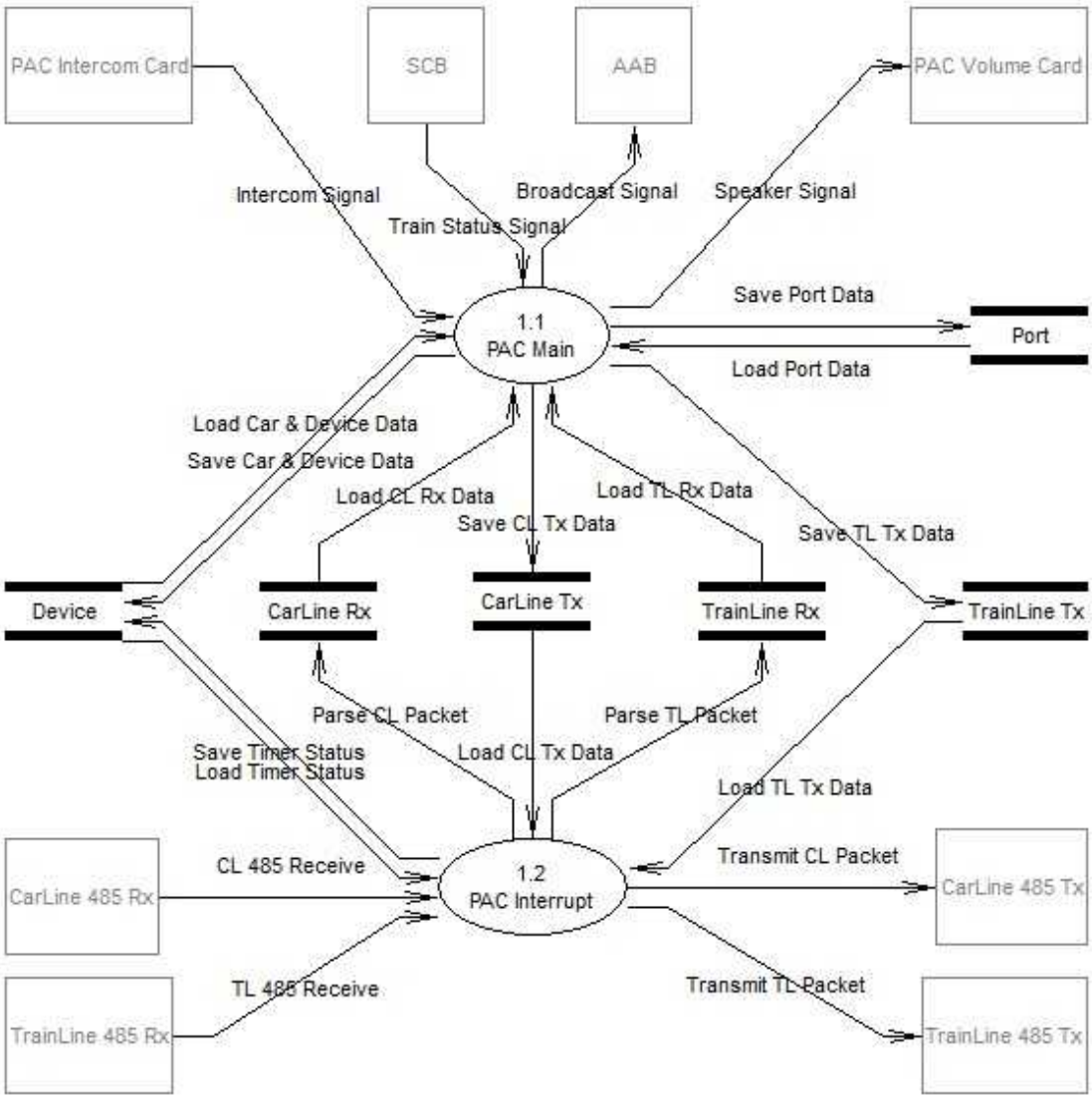


Figure 26. DFD level-1

4.1.1.2. PAC Main

Identifier	PAC Main		
Purpose	It polls train carline 485 communication. It processes double headed train process. It does train-line 485 communication.		
Source	SCB	Data	Train Status Signal
	PAC Intercom Card		Intercom Signal
	Port Storage		Load Port Data
	Device Storage		Load Car & Device Data
	CarLine Rx Storage		Load CL Rx Data
	TrainLine Rx Storage		Load TL Rx Data
Target	AAB	Data	Broadcast Signal
	PAC Volume Card		Speaker Signal
	Port Storage		Save Port Data
	Device Storage		Save Car & Device Data
	CarLine Tx Storage		Save CL Tx Data
	TrainLine Tx Storage		Save TL Tx Data

4.1.1.2.1. PAC Interrupt

Identifier	PAC Interrupt		
Purpose	It creates timing which is Car-Line, Train-Line, Train Polling and multi-coupled time for timing. It assesses device failure information by timer. It processes Car-Line and Train-Line 485 Rx, TX as interrupt.		
Source	Device Storage	Data	Load Timer Status
	CarLine Tx Storage		Load CL Tx Data
	TrainLine Tx Storage		Load TL Tx Data
	CarLine 485 Rx		CL 485 Receive
	TrainLine 485 Rx		TL 485 Receive

Target	Device Storage	Data	Save Timer Status
	CarLine Rx Storage		Parse CL Packet
	TrainLine Rx Storage		Parse TL Packet
	CarLine 485 Tx		Transmit CL Packet
	TrainLine 485 Tx		Transmit TL Packet

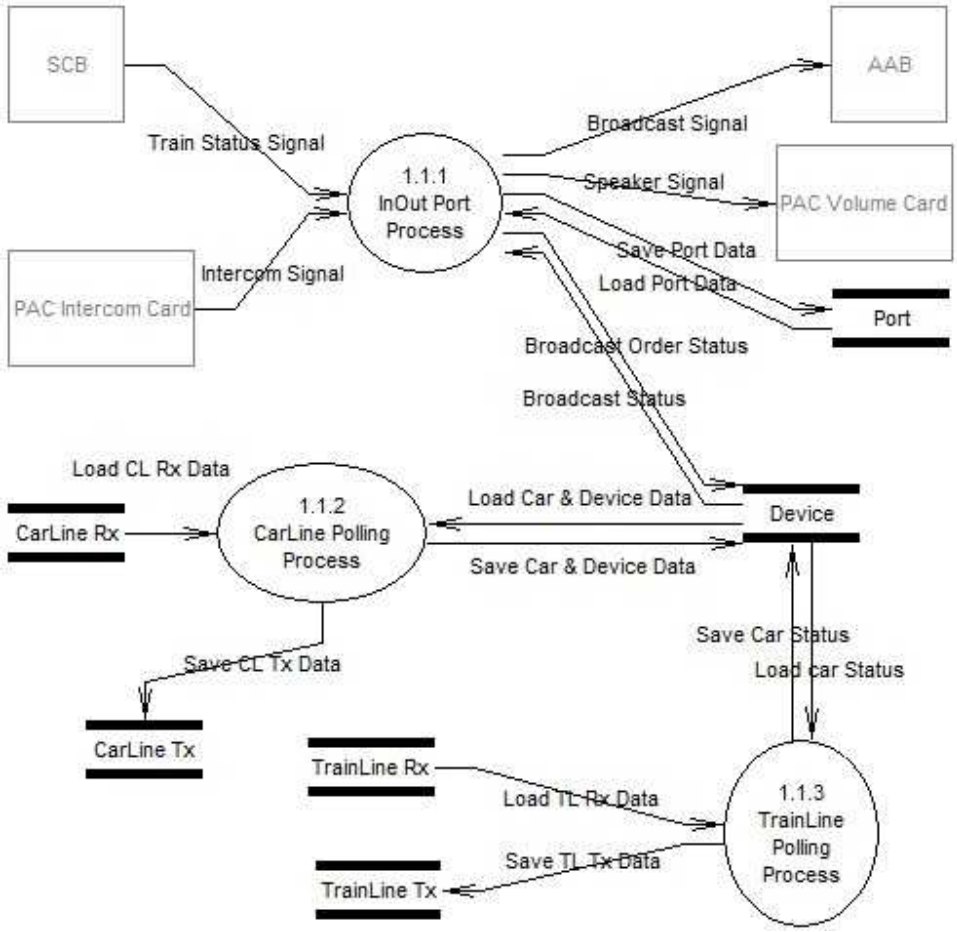


Figure 27. DFD level-2 Process 1.1

4.1.1.3. InOut Port Process

Identifier	InOut Port Process
Purpose	Read train of PAC_ID, CAB, Conductor, Forward, Reverse and Uncouple signals. Read train of door and intercom port signals.

Source	SCB	Data	Train Status Signal
	PAC Intercom Card		Intercom Signal
	Port Storage		Load Port Data
	Device Storage		Load Car & Device Data
Target	AAB	Data	Broadcast Signal
	PAC Volume Card		Speaker Signal
	Port Storage		Save Port Data
	Device Storage		Save Car & Device Data

4.1.1.3.1. CarLine Polling Process

Identifier	CarLine Polling Process		
Purpose	Make packet for device and restore CarLine Tx Storage.		
Source	Device Storage	Data	Load Car & Device Data
	CarLine Rx Storage		Load CL Rx Data
Target	Device Storage	Data	Save Car & Device Data
	CarLine Tx Storage		Save CL Tx Data

4.1.1.3.2. TrainLine Polling Process

Identifier	TrainLine Polling Process		
Purpose	It communicates thru train-line 485 by sending information of other train in train-line RX storage. It stores other train information received from communication in the train-line TX storage.		
Source	Device Storage	Data	Load Car Status
	TrainLine Rx Storage		Load TL Rx Data
Target	Device Storage	Data	Save Car Status
	TrainLine Tx Storage		Save TL Tx Data

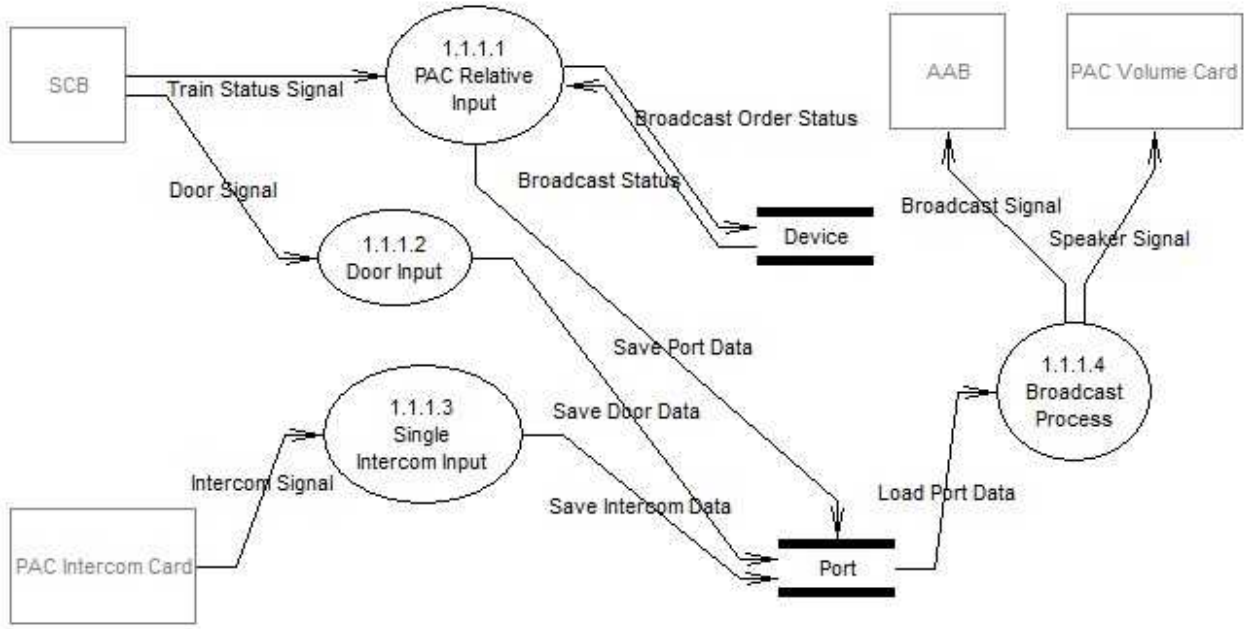


Figure 28. DFD level-3 Process 1.1.1

4.1.1.4. PAC Relative Input

Identifier	PAC Relative Input		
Purpose	It stores CAB, Conductor, Froward, Reverse and Port signal of the train in port storage. Broadcasting signal is saved in device storage after evaluating input signals.		
Source	SCB	Data	Train Status Signal
	Device Storage		Load Car & Device Data
Target	Port Storage	Data	Save Port Data
	Device Storage		Save Car & Device Data

4.1.1.4.1. Door Input

Identifier	Door Input		
Purpose	Detect train door open signal and restore Port Storage.		
Source	SCB	Data	Door Signal
Target	Port Storage		Save Door Data

4.1.1.4.2. Single Intercom Input

Identifier	Single Intercom Input		
Purpose	Read address signal and live DPO1, DPO2, PEI1, PEI2, PEI3, PEI4, PEI5, PEI6, PEI7, PEI8. Restore Port Storage.		
Source	PAC Intercom Card	Data	Intercom Signal
Target	Port Storage	Data	Save Single Intercom Data

4.1.1.4.3. Broadcast Process

Identifier	Broadcast Process		
Purpose	Port Storage reads the broadcast information to the source control and speaker control		
Source	Port Storage	Data	Load Port Data
Target	AAB	Data	Broadcast Signal
	PAC Volume Card		Speaker Signal

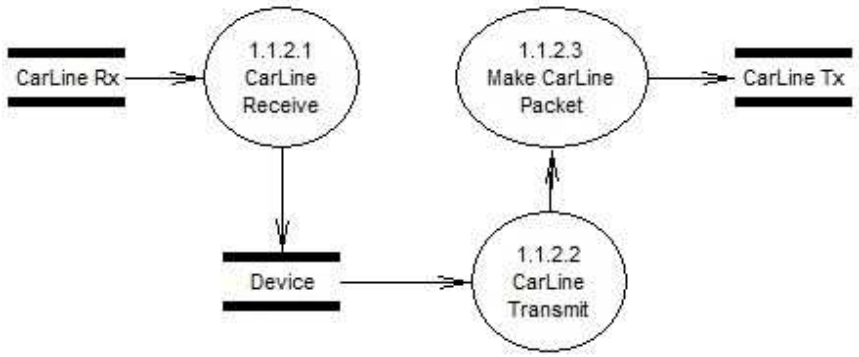


Figure 29. DFD level-3 Process 1.1.2

4.1.1.5. CarLine Receive

Identifier	CarLine Receive		
Purpose	It was read device information from Car-Line Rx Storage and saves in Device Storage.		
Source	CarLine Rx Storage	Data	Load CL Rx Data
Target	Device Storage	Data	Save Car & Device Data

4.1.1.5.1. CarLine Transmit

Identifier	CarLine Transmit		
Purpose	**Read** train status and device information from Device Storage.		
Source	Device Storage	Data	Load Car & Device Data
Target	Make CarLine Packet	Data	Make Carline Tx Packet

4.1.1.5.2. Make CarLine Packet

Identifier	Make CarLine Packet		
Purpose	It assesses information which comes from 'Car-Line Transmit' and creates packet for sending CarLine Device. Restore created packets in CarLine Tx Storage.		
Source	CarLine Transmit	Data	Make Carline Tx Packet
Target	CarLine Tx Storage	Data	Save CL Tx Data

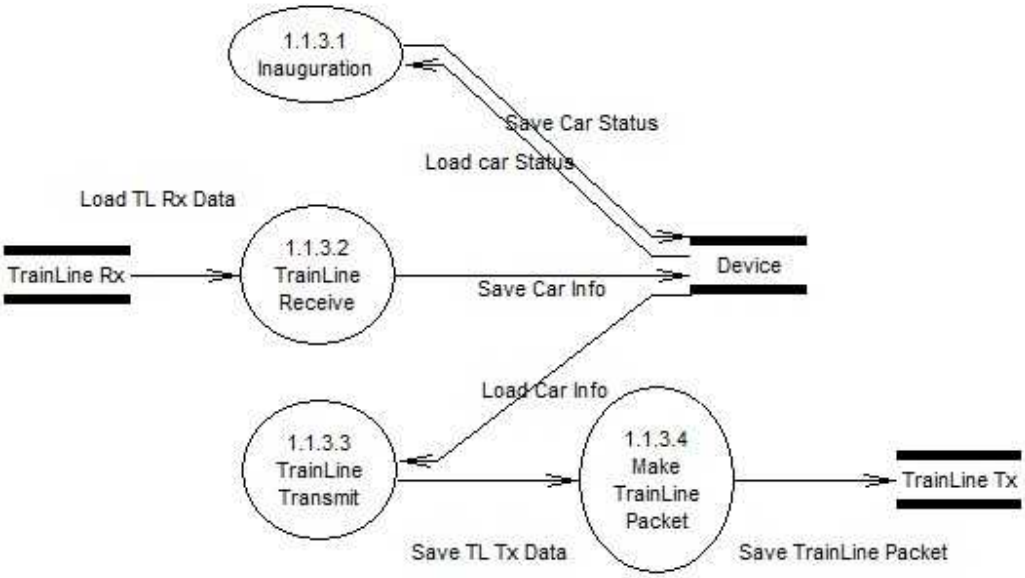


Figure 30. DFD level-3 Process 1.1.3

4.1.1.6. Inauguration

Identifier	Inauguration		
Purpose	It implements multi-coupled process when was read multi-coupled start-signal in Device Storage.		
Source	Device Storage	Data	Load Car Status
Target	Device Storage	Data	Save Car Status

4.1.1.6.1. TrainLine Receive

Identifier	TrainLine Receive		
Purpose	It saves multi-coupled train information from TrainLine Rx Storage.		
Source	TrainLine Rx Storage	Data	Load TL Rx Data
Target	Device Storage	Data	Save Car Info

4.1.1.6.2. TrainLine Transmit

Identifier	TrainLine Transmit		
Purpose	It bought multi-coupled train information from Device Storage.		
Source	Device Storage	Data	Load Car Info

Target	Make TrainLine Packet	Data	Save TL Tx Data

4.1.1.6.3. Make TrainLine Packet

Identifier	Make TrainLine Packet		
Purpose	By evaluating the information of double headed train, it stores the trainline packet in trainline Tx storage to be sent out to other double headed trains.		
Source	TrainLine Transmit	Data	Save TL Tx Data
Target	TrainLine Tx Storage	Data	Save TrainLine Packet

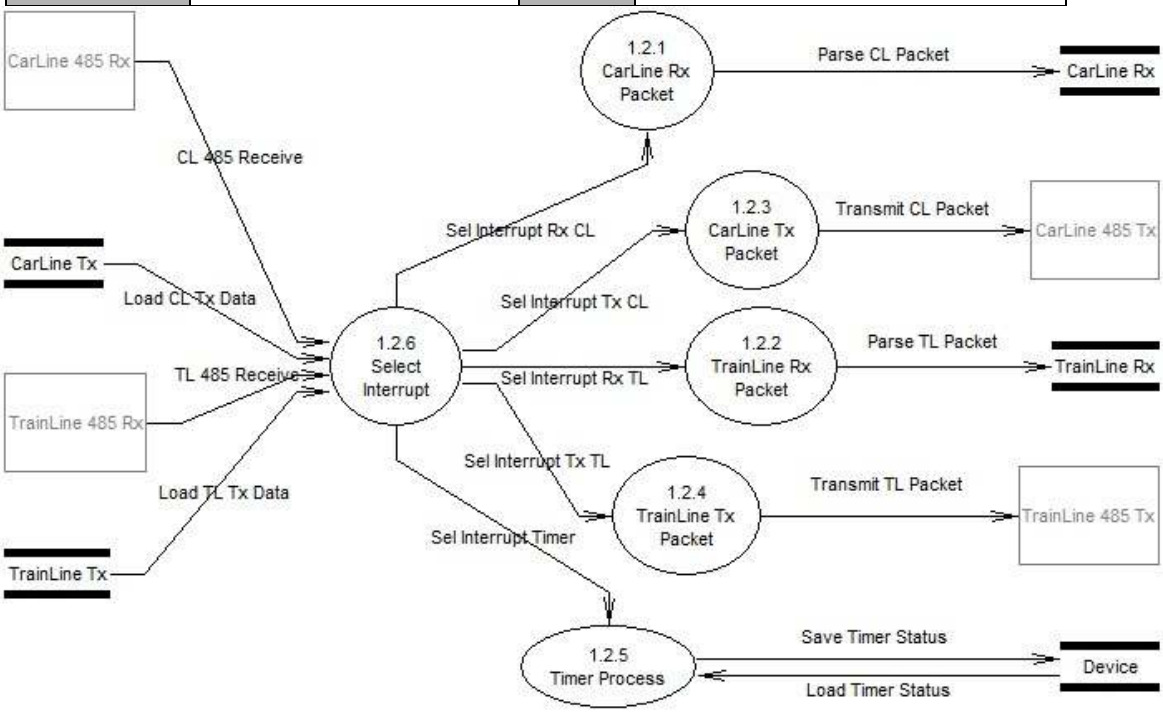


Figure 31. DFD level-2 Process 1.2

4.1.1.7. Select Interrupt

Identifier	Select Interrupt		
Purpose	It divides identified interrupt which caused during PAC process.		
Source	CarLine 485 Rx	Data	CL 485 Receive
	CarLine Tx Storage		Load CL Tx Data

	TrainLine 485 Rx		TL 485 Receive
	TrainLine Tx Storage		Load TL Tx Data
Target	CarLine Rx Packet	Data	Sel Interrupt Rx CL
	CarLine Tx Packet		Sel Interrupt Tx CL
	TrainLine Rx Packet		Sel Interrupt Rx TL
	TrainLine Tx Packet		Sel Interrupt Tx TL
	Timer Process		Sel Interrupt Timer

4.1.1.7.1. CarLine Rx Packet

Identifier	CarLine Rx Packet		
Purpose	One byte received by Rx interrupt in carline 485 communication is saved in CarLine Rx Storage.		
Source	Select Interrupt	Data	Sel Interrupt Rx CL
Target	CarLine Rx Storage	Data	Parse CL Packet

4.1.1.7.2. CarLine Tx Packet

Identifier	CarLine Tx Packet		
Purpose	It sends one byte which saved in CarLine Tx Storage to Car-line 485 Tx interrupt.		
Source	Select Interrupt	Data	Sel Interrupt Tx CL
Target	CarLine 485 Tx	Data	Transmit CL Packet

4.1.1.7.3. TrainLine Rx Packet

Identifier	TrainLine Rx Packet		
Purpose	One byte received by Rx interrupt in carline 485 communication is saved in CarLine Rx Storage.		
Source	Select Interrupt	Data	Sel Interrupt Rx TL
Target	TrainLine Rx Storage	Data	Parse TL Packet

4.1.1.7.4. TrainLine Tx Packet

Identifier	TrainLine Tx Packet		
Purpose	It sends 1 byte which saved in TranLine Tx Storage to TrainLine 485 Tx interrupt.		
Source	Select Interrupt	Data	Sel Interrupt Tx TL
Target	TrainLine 485 Tx	Data	Transmit TL Packet

4.1.1.7.5. Timer Process

Identifier	Timer Process		
Purpose	Create timing for CarLine, TrainLine train polling and multi-coupled time. Determine device fault information by timer.		
Source	Device Storage	Data	Load Timer Status
Target	Device Storage	Data	Save Timer Status

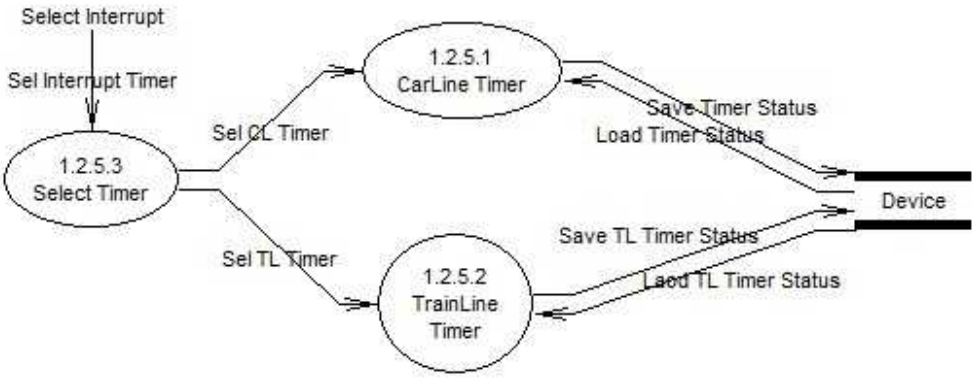


Figure 32. DFD level-3 Process 1.2.5

4.1.1.8. Select Timer

Identifier	Select Timer		
Purpose	Determine CarLine and TrainLine type and branches off.		
Source	Timer Process	Data	Sel Interrupt Timer
Target	CarLine Timer	Data	Sel CL Timer
	TrainLine Timer		Sel TL Timer

4.1.1.8.1. CarLine Timer

Identifier	CarLine Timer		
Purpose	Determine CarLine Device fault by using timer and it stores in the Device Storage. Set Polling timing.		
Source	Select Timer	Data	Sel CL Timer
	Device Storage		Load Timer Status
Target	Device Storage	Data	Save Timer Status

4.1.1.8.2. TrainLine Timer

Identifier	TrainLine Timer		
Purpose	Determine multi-coupled train communication malfunction by using timer and it stores in Device Storage.		
Source	Select Timer	Data	Sel TL Timer
	Device Storage		Load TL Timer Status
Target	Device Storage	Data	Save TL Timer Status

4.1.2. Interprocess Dependency-Structure Chart

In this subsection to describe relationship among entities structure chart is used.

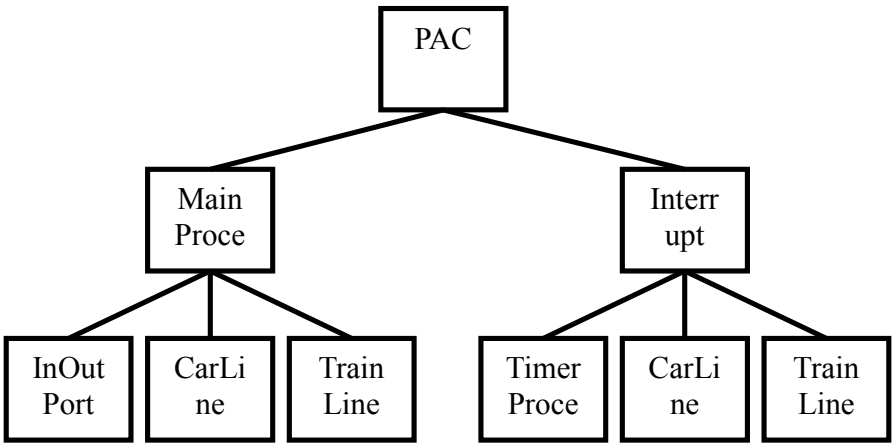


Figure 33. Structure Chart

4.1.2.1. PAC Software

ID	Process Name	Identifier
	PAC Software	PAC.Software.F
Relation	Call	Called
	- PAC.Main.F	-

	- PAC.InterruptHandler.F	
Parameter/Return	Parameter	Return
	-	-

4.1.2.1.1. Main Process

ID	Process Name	Identifier
	Main Process	PAC.Main.F
Relation	Call	Called
	- PAC.InOut.F - PAC.CarLinePolling.F - PAC.TrainLinePolling.F	- PAC.Software.F
Parameter/Return	Parameter	Return
	-	-

4.1.2.1.2. Interrupt Handler

ID	Process Name	Identifier
	Interrupt Handler	PAC.InterruptHandler.F
Relation	Call	Called
	- PAC.CarLineRxPacket.F - PAC.TrainLineRxPacket.F - PAC.CarLineTxPacket.F - PAC.TrainLinePacket.F - PAC.Timer.F	- PAC.Software.F
Parameter/Return	Parameter	Return
	-	

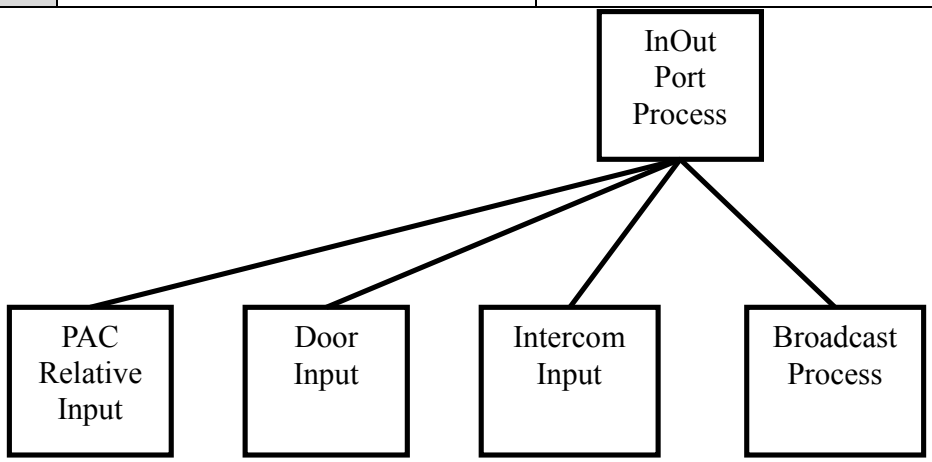


Figure 34. Structure Chart (Continue)

4.1.2.2. InOut Port Process

ID	Process Name	Identifier
	InOut Port Process	PAC.InOut.F
Relation	Call	Called
	- PAC.RelativeInput.F - PAC.DoorInput.F - PAC.IntercomInput.F - PAC.Broadcast.F	- PAC.Main.F
Parameter/Return	Parameter	Return

	-	-

4.1.2.2.1. PAC Relative Input

ID	Process Name	Identifier
	PAC Relative Input	PAC.RelativeInput.F
Relation	Call	Called
	-	- PAC.InOut.F
Parameter/Return	Parameter	Return
	-	- FORWARD - REVERSE - UNCOUPLE - CONDUCTOR - CAB - PAC_ID - Intercom_ISP - Intercom_ESP - Intercom_BOTH - Intercom_CANCEL

4.1.2.2.2. Door Input

ID	Process Name	Identifier
	Door Input	PAC.DoorInput.F
Relation	Call	Called
	-	- PAC.InOut.F
Parameter/Return	Parameter	Return
	-	- Door_RH - Door_RL - Door_LH - Door_LL

4.1.2.2.3. Single Intercom Input

ID	Process Name	Identifier
	Single Intercom Input	PAC.SingleIntercomInput.F
Relation	Call	Called
	-	- PAC.InOut.F
Parameter/Return	Parameter	Return
	-	- DPO1 or DPO2 : Live, Address - - DPO1 : Live, Address - DPO2 : Live, Address - PEI1 : Live, Address - PEI2 : Live, Address - PEI3 : Live, Address - PEI4 : Live, Address - PEI5 : Live, Address - PEI6 : Live, Address - PEI7 : Live, Address - PEI8 : Live, Address

4.1.2.2.4. Broadcast Process

ID	Process Name	Identifier
	Broadcast Process	PAC.Broadcast.F

Relation	Call	Called
	-	- PAC.InOut.F
Parameter/Return	Parameter	Return
	- Old_PA_Out_Switch - Select_INTERCOM - Select_MP3 - Select_Internal_Speaker - Select_External_Speaker_Left - Select_External_Speaker_Right	- MP3_Mute - Old_PA1_Status - Old_PA2_Status

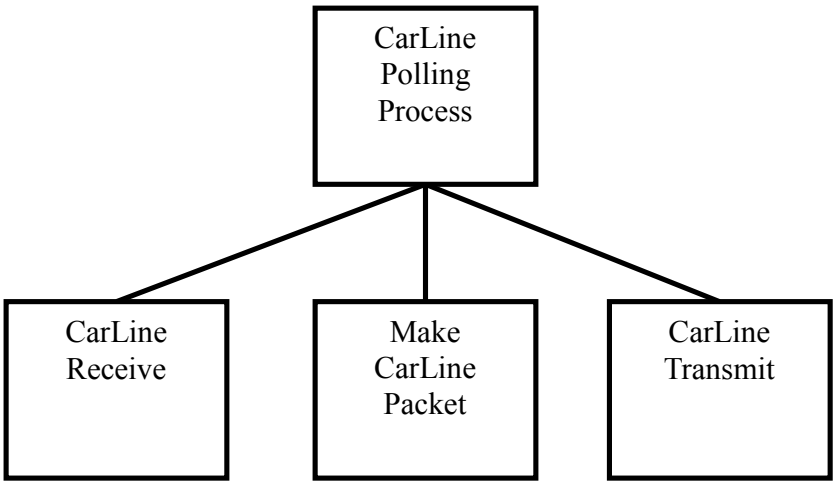


Figure 35. Structure Chart (Continue)

4.1.2.3. CarLine Polling Process

ID	Process Name	Identifier
	CarLine Polling Process	PAC.CarLinePolling.F
Relation	Call	Called
	- PAC.CarLineReceive.F - PAC.MakeCarLinePacket.F - PAC.CarLineTransmit.F	- PAC.Main.F
Parameter/Return	Parameter	Return
	-	-

4.1.2.3.1. CarLine Receive

ID	Process Name	Identifier
	CarLine Receive	PAC.CarLineReceive.F
Relation	Call	Called
	-	- PAC.CarLinePolling.F
Parameter/Return	Parameter	Return
	-	- Devices Data

4.1.2.3.2. Make CarLine Packet

ID	Process Name	Identifier
	Make CarLine Packet	PAC.MakeCarLinePacket.F
Relation	Call	Called
	-	- PAC.CarLinePolling.F

Parameter/Return	Parameter	Return
	- Device ID	-

4.1.2.3.3. CarLine Transmit

ID	Process Name	Identifier
	CarLine Transmit	PAC.CarLineTransmit.F
Relation	Call	Called
	-	- PAC.CarLinePolling.F
Parameter/Return	Parameter	Return
	- Device ID	-

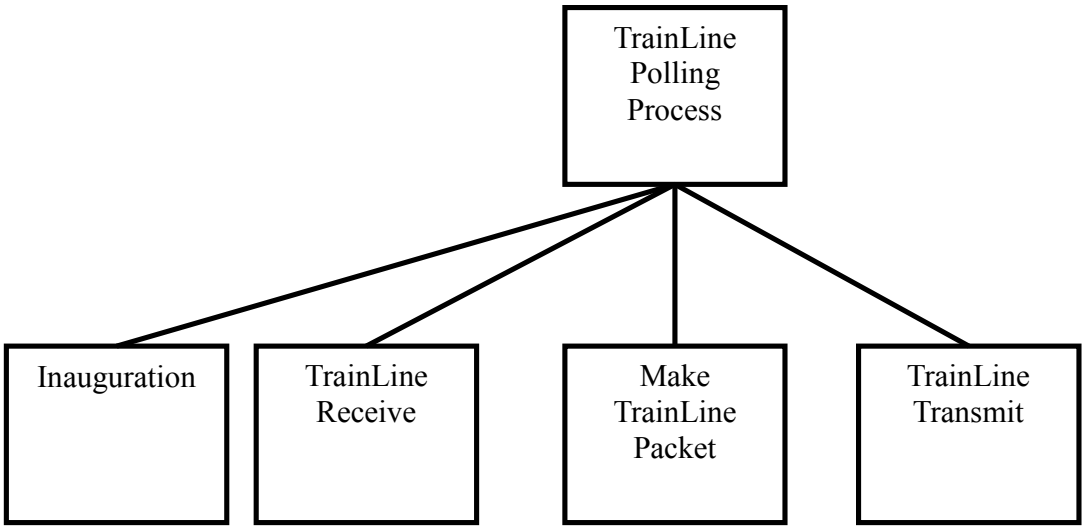


Figure 36. Structure Chart (Continue)

4.1.2.4. TrainLine Polling Process

ID	Process Name	Identifier
	TrainLine Polling Process	PAC.TrainLinePolling.F
Relation	Call	Called
	- PAC.Inauguration.F - PAC.TrainLineReceive.F - PAC.MakeTrainLinePacket.F - PAC.TrainLineTransmit.F	- PAC.Main.F
Parameter/Return	Parameter	Return
	-	-

4.1.2.4.1. Inauguration

ID	Process Name	Identifier
	Inauguration	PAC.Inauguration.F
Relation	Call	Called
	-	- PAC.TrainLinePolling.F
Parameter/Return	Parameter	Return
	-	-

4.1.2.4.2. TrainLine Receive

ID	Process Name	Identifier
	TrainLine Receive	PAC.TrainLineReceive.F
Relation	Call	Called
	-	- PAC.TrainLinePolling.F
Parameter/Return	Parameter	Return
	-	- Car Status

4.1.2.4.3. Make TrainLine Packet

ID	Process Name	Identifier
	Make TrainLine Packet	PAC.MakeTrainLinePacket.F
Relation	Call	Called
	-	- PAC.TrainLinePolling.F
Parameter/Return	Parameter	Return
	- Car Coupling ID	-

4.1.2.4.4. TrainLine Transmit

ID	Process Name	Identifier
	TrainLine Transmit	PAC.TrainLineTransmit.F
Relation	Call	Called
	-	- PAC.TrainLinePolling.F
Parameter/Return	Parameter	Return
	- Car Coupling ID	-

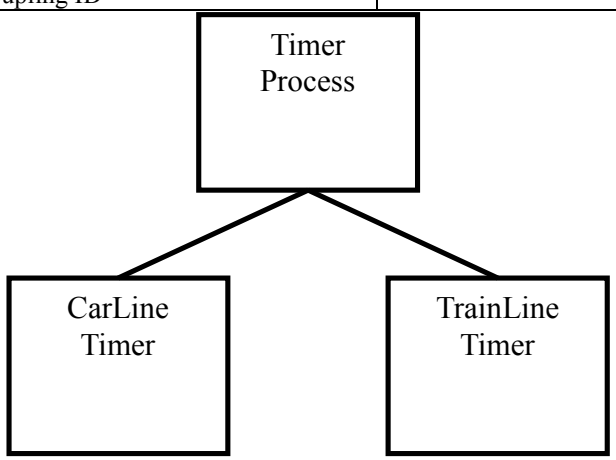


Figure 37. Structure Chart (Continue)

4.1.2.5. Timer Process

ID	Process Name	Identifier
	Timer Process	PAC.Timer.F
Relation	Call	Called
	- PAC.CarLineTimer.F - PAC.TrainLineTimer.F	- PAC.Interrupt.F
Parameter/Return	Parameter	Return
	-	-

4.1.2.5.1. CarLine Timer

ID	Process Name	Identifier
	CarLine Timer	PAC.CarLineTimer.F
Relation	Call	Called
	-	- PAC.Timer.F
Parameter/Return	Parameter	Return
	-	- Send CarLine Polling Timing - Device Error Count - Send Device ID

4.1.2.5.2. TrainLine Timer

ID	Process Name	Identifier
	TrainLine Timer	PAC.TrainLineTimer.F
Relation	Call	Called
	-	- PAC.Timer.F
Parameter/Return	Parameter	Return
	-	- Send TrainLine Polling Timing - Car Error Count - Send Car Coupling ID

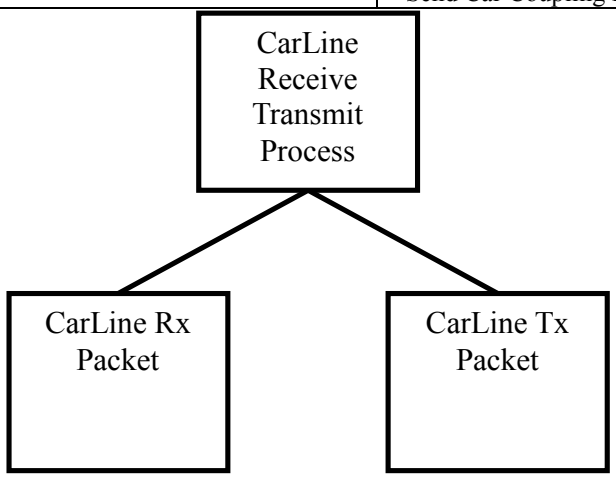


Figure 38. Structure Chart (Continue)

4.1.2.6. CarLine Interrupt Packet

ID	Process Name	Identifier
	CarLine Interrupt Packet	PAC.CarLineInt.F
Relation	Call	Called
	-	- PAC.Interrupt.F
Parameter/Return	Parameter	Return
	-	-

4.1.2.6.1. CarLine Rx Packet

ID	Process Name	Identifier
	CarLine Rx Packet	PAC.CarLineRxPacket.F
Relation	Call	Called
	-	- PAC.CarLineInt.F
Parameter/Return	Parameter	Return

	-	- CarLine 485 1 Byte

4.1.2.6.2. CarLine Tx Packet

ID	Process Name	Identifier
	CarLine Tx Packet	PAC.CarLineTxPacket.F
Relation	Call	Called
	-	- PAC.CarLineInt.F
Parameter/Return	Parameter	Return
	- CarLine 485 1 Byte	-

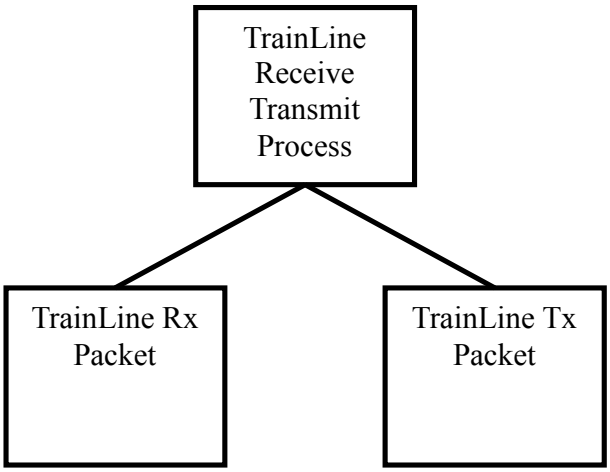


Figure 39. Structure Chart (Continue)

4.1.2.7. TrainLine Interrupt Packet

ID	Process Name	Identifier
	TrainLine Interrupt Packet	PAC.TrainLineInt.F
Relation	Call	Called
	-	- PAC.Interrupt.F
Parameter/Return	Parameter	Return
	-	-

4.1.2.7.1. TrainLine Rx Packet

ID	Process Name	Identifier
	TrainLine Rx Packet	PAC.TrainLineRxPacket.F
Relation	Call	Called
	-	- PAC.TrainLineInt.F
Parameter/Return	Parameter	Return
	-	- TrainLine 485 1 Byte

4.1.2.7.2. TrainLine Tx Packet

ID	Process Name	Identifier
	TrainLine Tx Packet	PAC.TrainLineTxPacket.F
Relation	Call	Called
	-	- PAC.TrainLineInt.F
Parameter/Return	Parameter	Return
	- TrainLine 485 1 Byte	-

4.2. CCP Description

4.2.1. Interprocess Dependency-Data Flow Diagram

In this subsection to describe relationship among entities DFD is used.

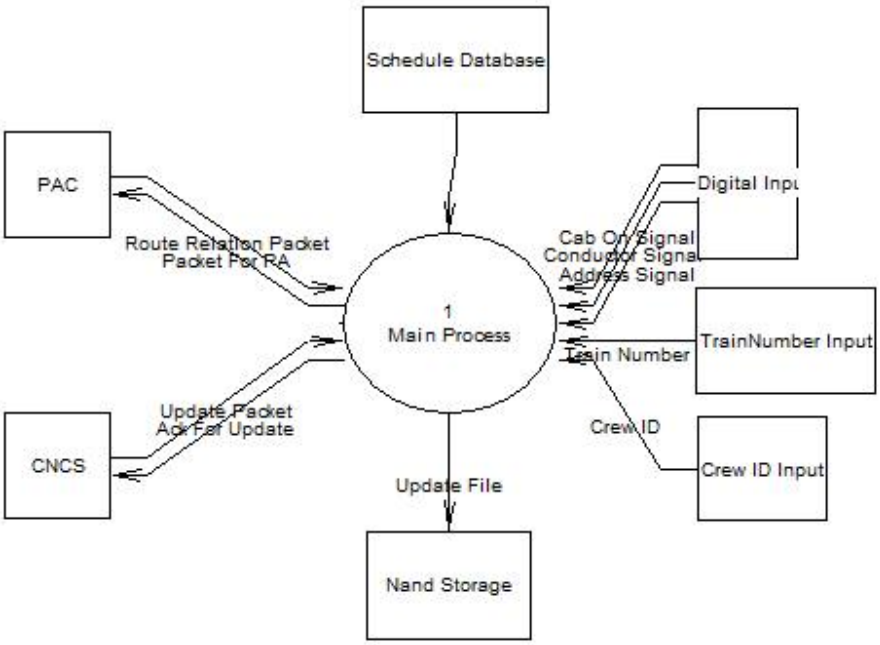


Figure 40. DFD level-0 (Context Diagram)

4.2.1.1. CCP.MainProcess.F

Identifier	CCP.MainProcess.F		
Purpose	CCP System Main Program		
Source	Nand Storage	Data	Schedule Database
	External Signal		Cab on Signal
	External Signal		Conductor Signal
	External Signal		Address Signal
	User Input		Train Number
	User Input		Crew ID
	PAC		Car Data and Route Data

	CNCS		Update Data
Target	PAC	Data	Route Data
	CNCNS		Update Result
	Nand Storage		Update Files

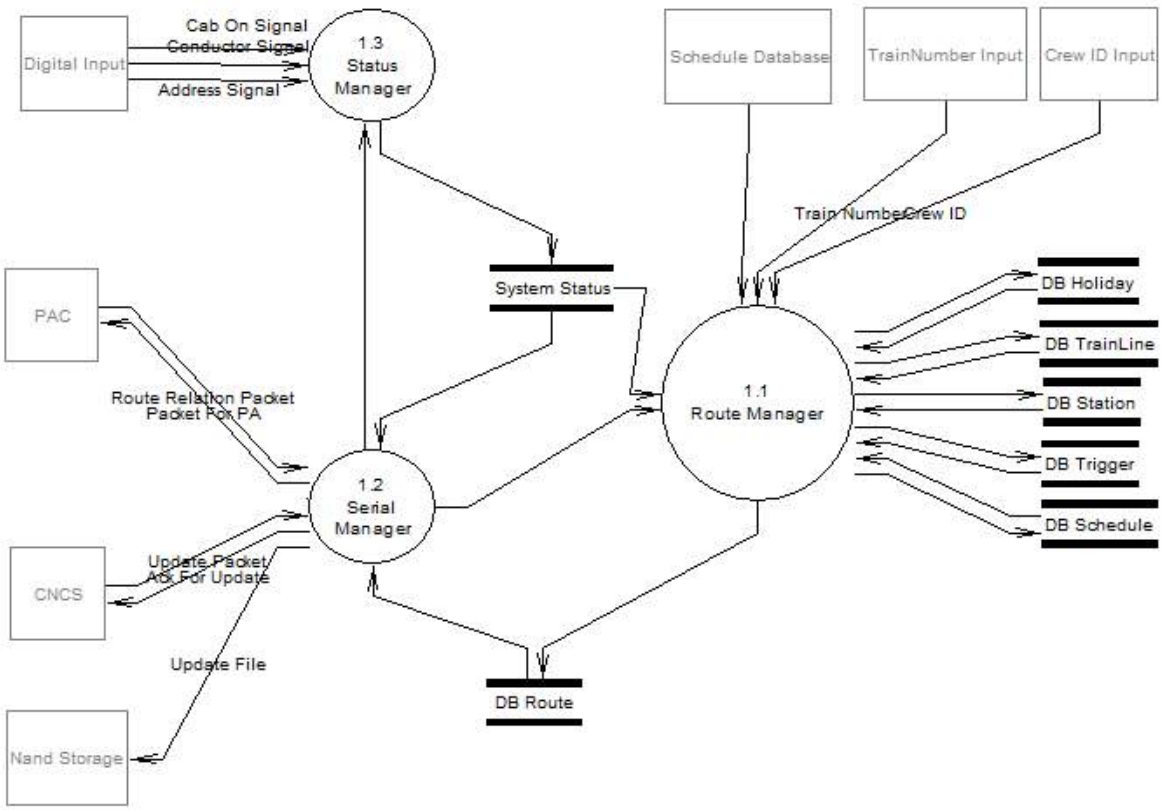


Figure 41. DFD level-1

4.2.1.2. CCP.RouteManager.F

Identifier	CCP.RouteManager.F		
Purpose	Management Module for CCP operation Route		
Source	Main Memory	Data	DB Holiday

	Main Memory		DB TrainLine
	Main Memory		DB Station
	Main Memory		DB Trigger
	Main Memory		DB Schedule
	Status Manager		System Status
Target	Main Memory	Data	DB Holiday
	Main Memory		DB TrainLine
	Main Memory		DB Station
	Main Memory		DB Trigger
	Main Memory		DB Schedule
	Main Memory		DB Route

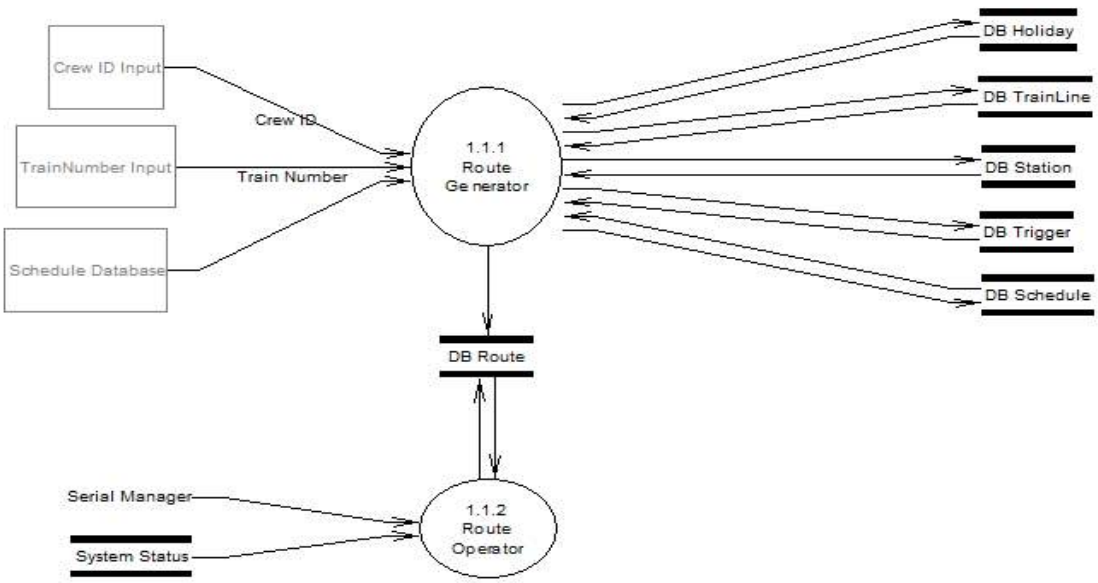


Figure 42. DFD level-2 RouteManager 1.1

4.2.1.3. CCP.RouteGenerator.F

Identifier	CCP.RouteGenerator.F		
Purpose	Module to create operation route		
Source	Main Memory	Data	DB Holiday
	Main Memory		DB TrainLine
	Main Memory		DB Station
	Main Memory		DB Trigger
	Main Memory		DB Schedule
	User Input		Train Number
	User Input		CrewID
	Nand Storage		Schedule Database
Target	Main Memory	Data	DB Holiday
	Main Memory		DB TrainLine
	Main Memory		DB Station
	Main Memory		DB Trigger
	Main Memory		DB Schedule
	Main Memory		DB Route

4.2.1.3.1. CCP.RouteOperator.F

Identifier	CCP.RouteOperator.F		
Purpose	The module to operate route		
Source	Main Memory	Data	DB Route
	Serial Manager		Car Data and Route Data
	System Status Manager		System Status
Target	Main Memory	Data	DB Route

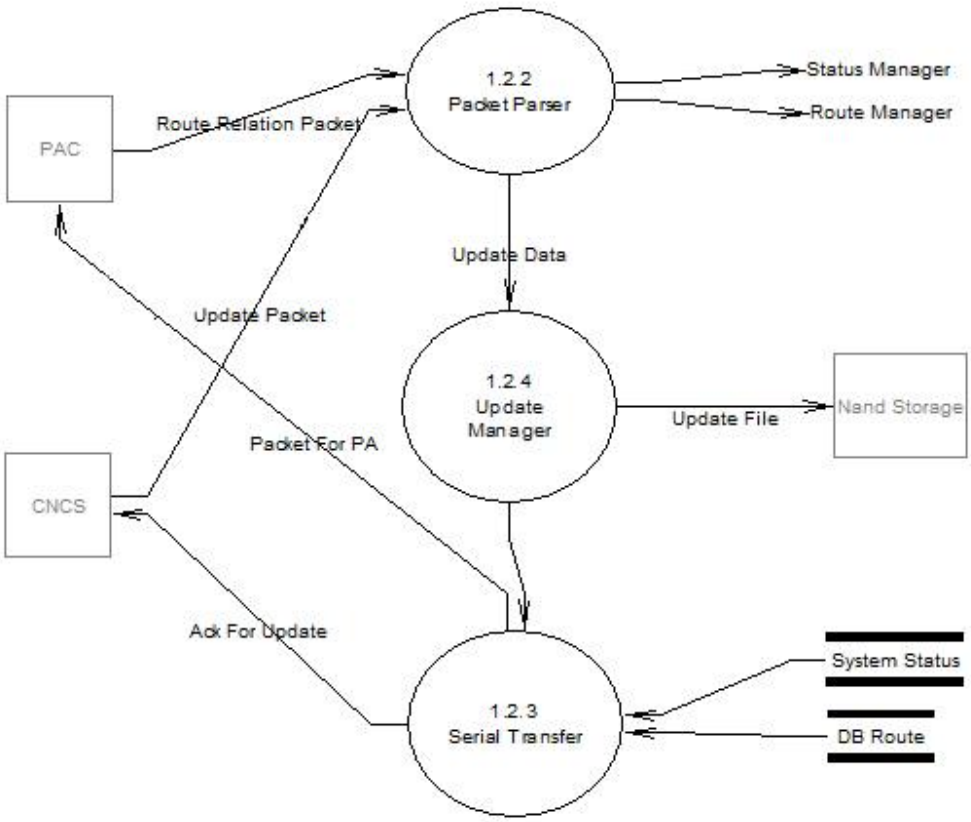


Figure 43. DFD level-2 SerialManager 1.2

4.2.1.4. CCP.PacketParser.F

Identifier	CCP.PacketParser.F		
Purpose	It structuralizes packets from external device by use.		
Source	PAC	Data	Car Data and Route Data
	CNCS		Update Data
Target	Status Manager	Data	Car Data
	Route Manager		Route Data
	Route Manager		Coordinate Data
	Update Manager		Update Data

4.2.1.4.1. CCP.UpdateManager.F

Identifier	CCP.UpdateManager.F		
Purpose	It process update data received from CNCS.		
Source	Packet Parser	Data	Update Data
Target	Nand Storage	Data	Update Files
	Serial Transfer		Update Result

4.2.1.4.2. CCP.SerialTransfer.F

Identifier	CCP.SerialTransfer.F		
Purpose	It transmits serial data on external device.		
Source	Update Manager	Data	Update Result
	Main Memory		System Status
	Main Memory		DB Route
Target	PAC	Data	Route Data
	CNCS		Update Result

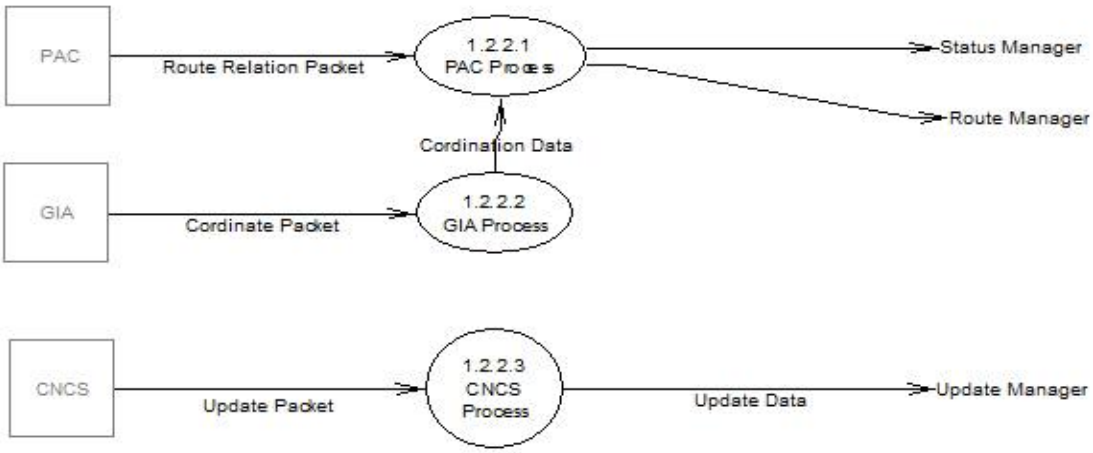


Figure 44. DFD level-3 Serial Parser 1.2.2 .

4.2.1.5. CCP.PACProcess.F

Identifier	CCP.PACProcess.F		
Purpose	The section that processes PAC data structured.		
Source	PAC	Data	Car Data and Route Data
Target	Status Manager	Data	Car Data
	Route Manager		Route Data

4.2.1.5.1. CCP.CNCSProcess.F

Identifier	CCP.CNCSProcess.F		
Purpose	The section that processes GIA_DATA_DATA data structured.		
Source	CNCS	Data	Update Packet
Target	Update Manager	Data	Update Data

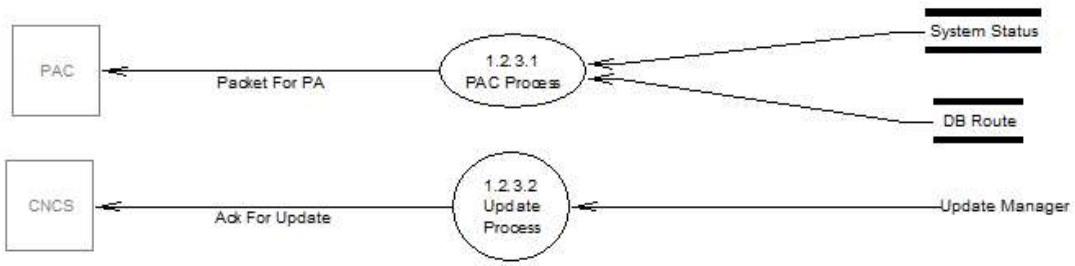


Figure 45. DFD level-3 Serial Transfer 1.2.3

4.2.1.6. CCP.PACProcess.F

Identifier	CCP.PACProcess.F		
Purpose	It transferes route information and vehicle status to PAC.		
Source	Main Memory	Data	System Status
	Main Memory		DB Route

Target	PAC	Data	Card Data
	PAC		PA Data

4.2.1.6.1. CCP.UpdateProcess.F

Identifier	CCP.UpdateProcess.F		
Purpose	It transferes update result data received from Update Manager to CNCS.		
Source	Update Manager	Data	Update Result
Target	CNCS	Data	Update ACK Packet

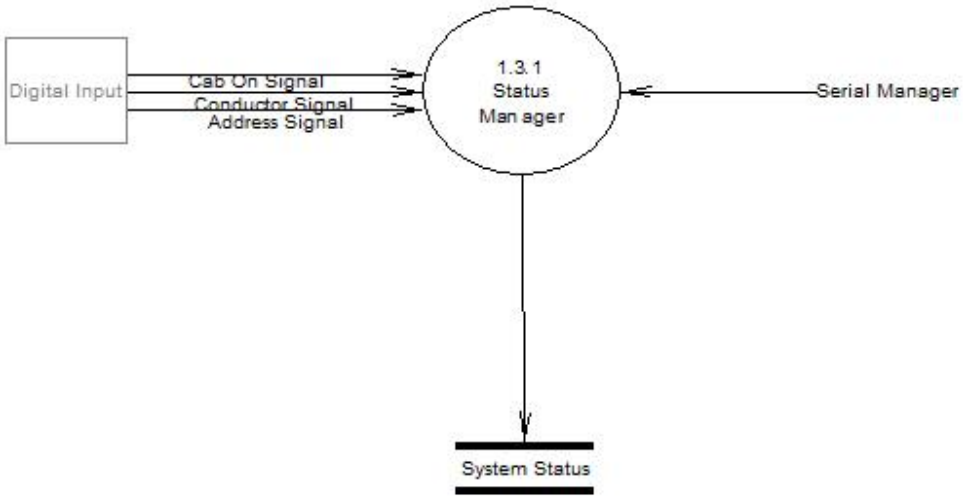


Figure 46. DFD level-2 Status Manager 1.3

4.2.1.7. CCP.StatusManager.F

Identifier	CCP.StatusManager.F		
Purpose	It manages the system status information by collecting.		
Source	External Signal	Data	Cab On Signal
	External Signal		Conductor Signal
	External Signal		Address Signal
	Serial Manager		Car Data
Target	Main Memory	Data	System Status

4.2.2. Class Diagram

In this paragraph, the class diagram, the class details and the sequence diagram for CCP realization function written in SRS are described.

When describing the overlap of the class diagram, text color to gray and the class details should be omitted.

4.2.2.1. Cabon Indicator function implementation

Figure 47. Class diagram

Class Name	CCP.CStatus.Variable.C		
Attributes	m_bSetCabonKey	Type	bool
Operations	SetCabonKey	Return	void
	GetCabonKey		void

Class Name	CCP.CCPDlg.C		
Attributes		Type	
Operations	UpdateStatus	Return	void

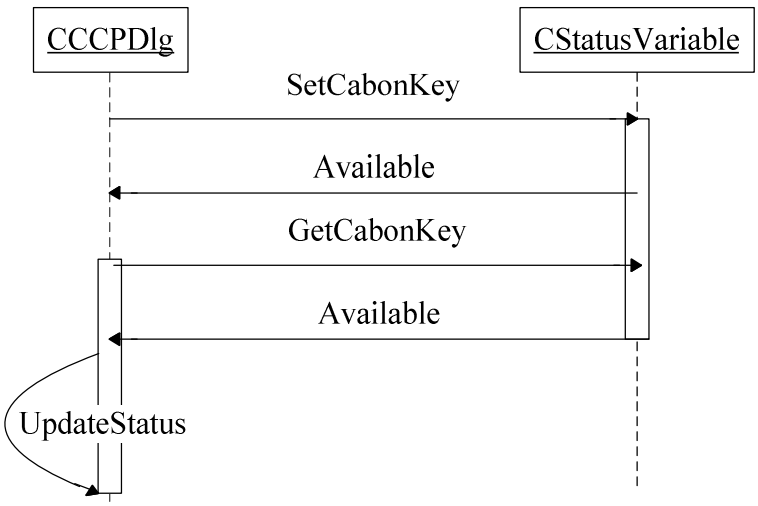


Figure 48. Sequence diagram

4.2.2.2. 485 Communication Operation Check

In the permitted status of Mascon input signal, It realizes the fucnion that displays PAC and the communicaton result.

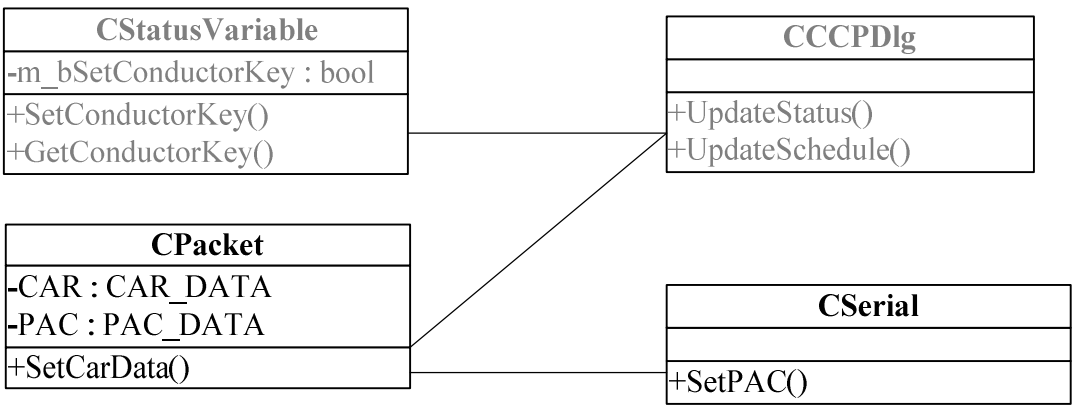


Figure 49. Class diagram

Class Name	CCP.CPacket.C		
Attributes	CAR	Type	CAR_DATA
	PAC		PAC_DATA
Operations	SetCarData	Return	void

Class Name	CCP.Serial.C		
Attributes		Type	
Operations	SetPAC	Return	void

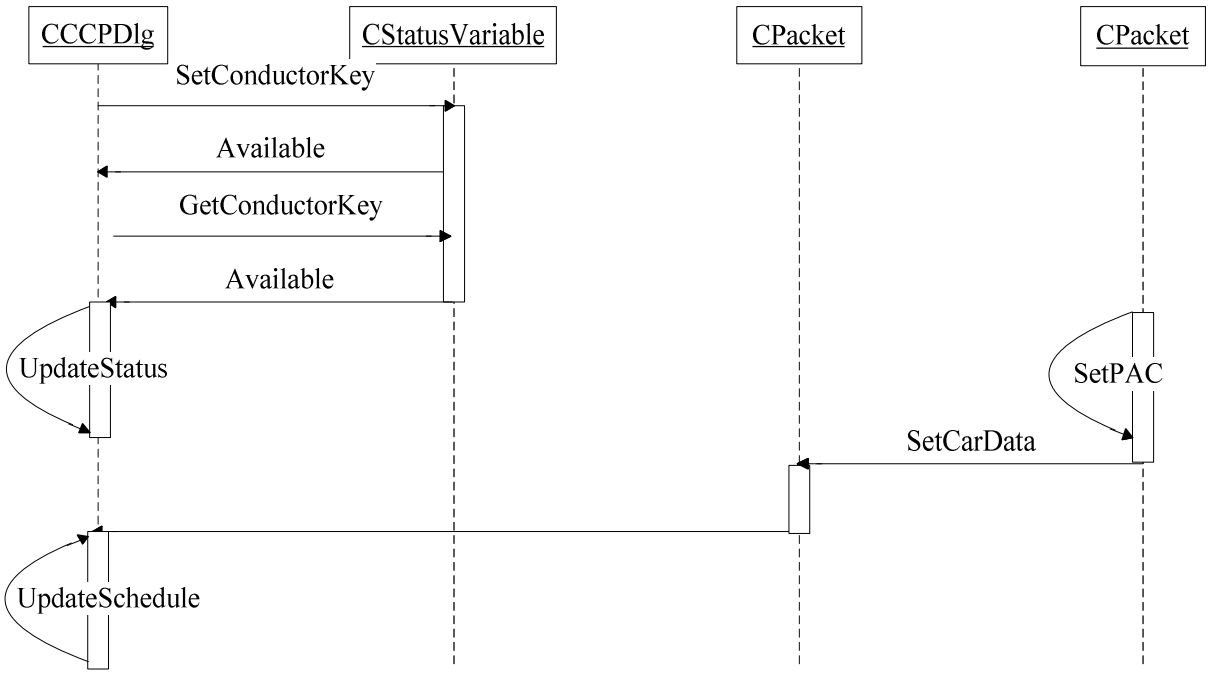


Figure 50. Sequence diagram

4.2.2.3. GUI interworking implementation

CPageNumber
-m_strNumber : CString
+OnClick_Num()

Figure 51. Class diagram

Class Name	CCP.CStatus.Variable.C		
Attributes	m_bSetCabonKey	Type	bool
Operations	SetCabonKey	Return	void
	GetCabonKey		void

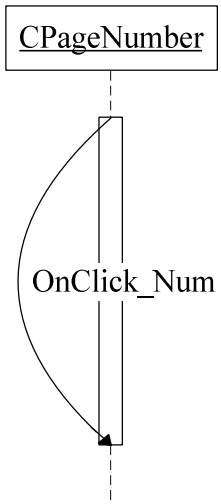


Figure 52. Sequence diagram

4.2.2.4. Implementation of Address Indicator Function

Figure 53. Class diagram

Class Name	CCP.CStatus.Variable.C		
Attributes	m_byCCPID	Type	unsigned char
Operations	SetCCPID	Return	void
	GetCCPID		void

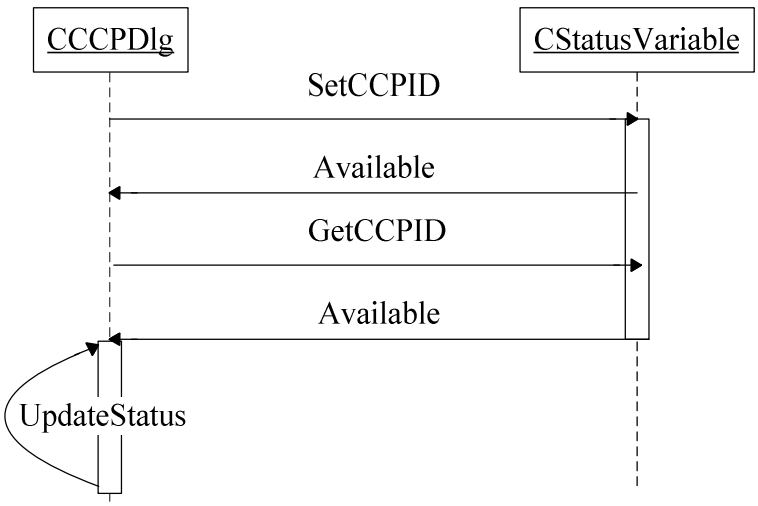


Figure 54. Sequence Diagram

4.2.2.5. Initialization Implementation

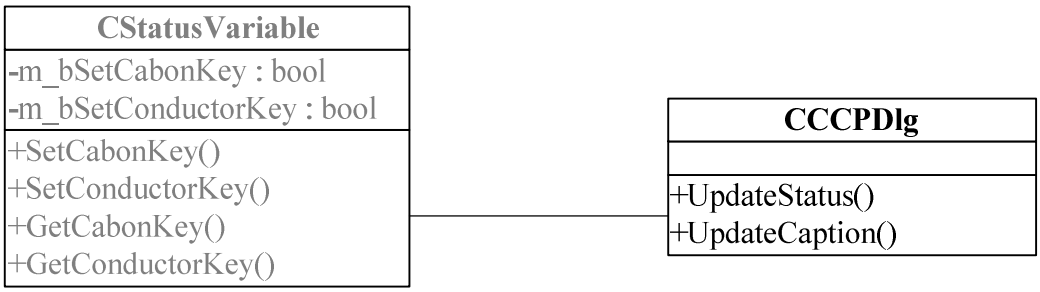


Figure 55. Class Diagram

Class Name	CCP.CCPDlg.C		
Attributes		Type	
Operations	UpdateStatus	Return	void
	UpdateCaption		void

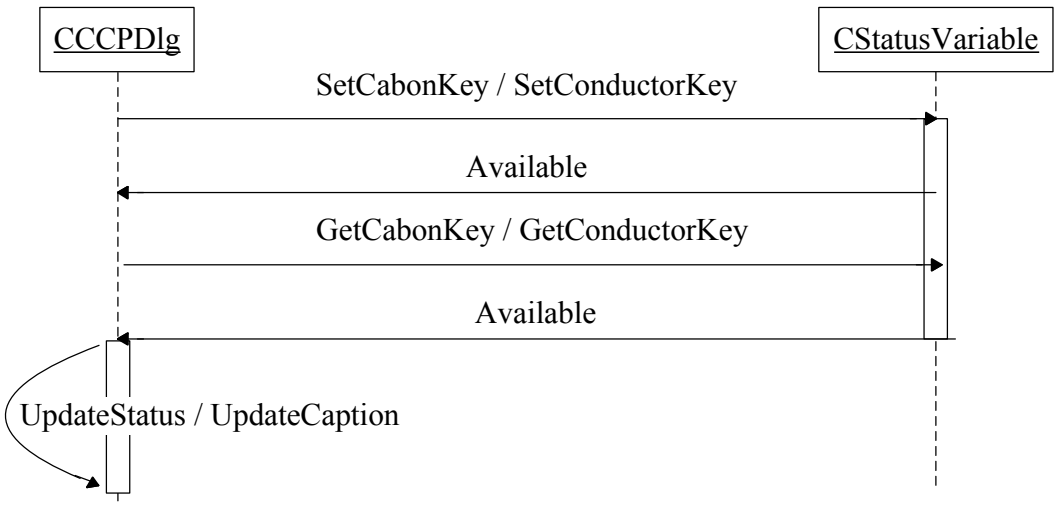


Figure 56. Sequence Diagram

4.2.2.6. Train Number input implementation

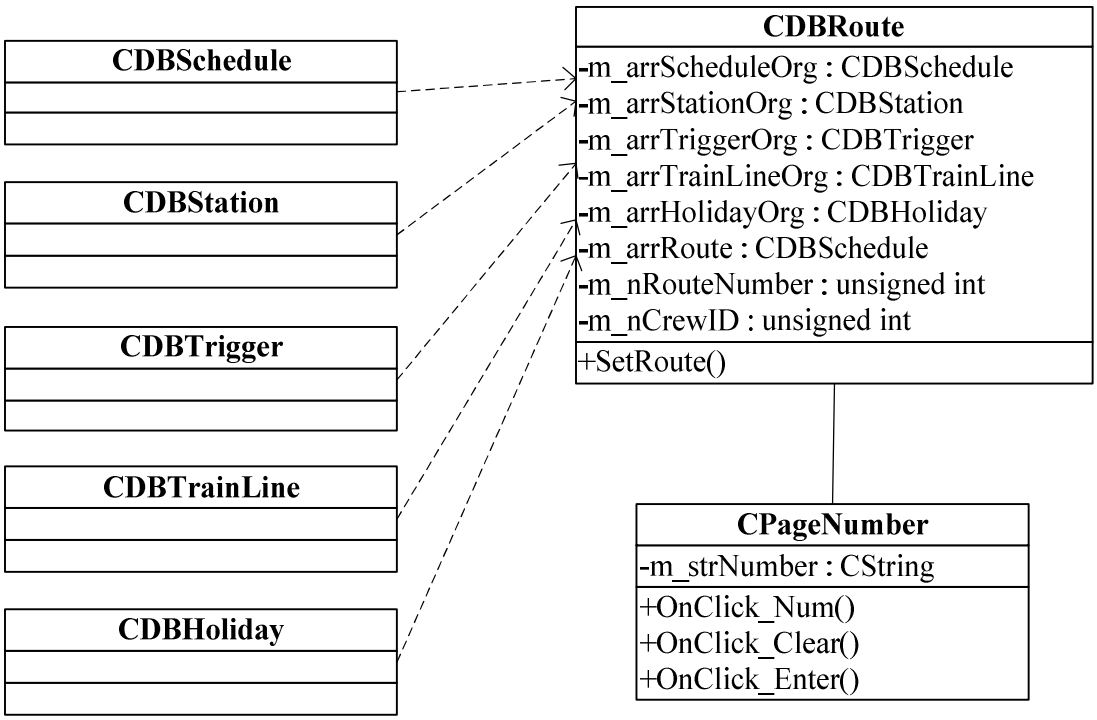


Figure 57. Class Diagram

Class Name	CCP.CDBRoute.C		
Attributes	m_arrScheduleOrg	Type	CDBSchedule
	m_arrStationOrg		CDBStation
	m_arrTriggerOrg		CDBTrigger
	m_arrTrainLineOrg		CDBTrainLine
	m_arrHolidayOrg		CDBHoliday
	m_arrRoute		CDBSchedule
	m_nRouteNumber		unsigned int
	m_nCrewID		unsigned int
Operations	SetRoute	Return	bool

	SetCrewID		void

Class Name	CCP.CPageNumber.C		
Attributes	m_strNumber	Type	CString
Operations	OnClick_Num	Return	void
	OnClick_Clear		void
	OnClick_Enter		void

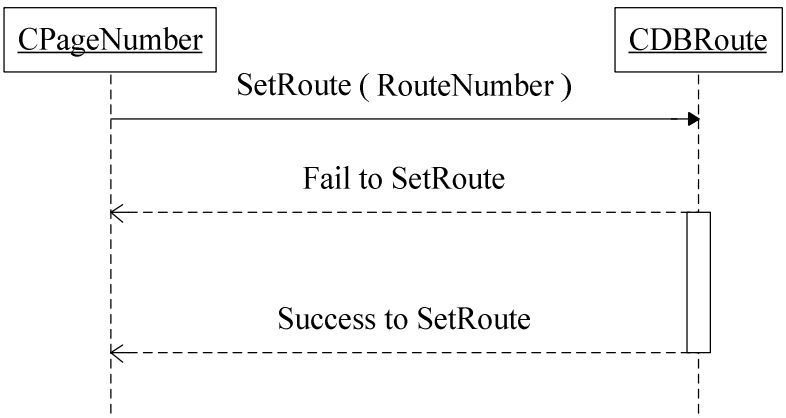


Figure 58. Sequence Diagram

4.2.2.7. Implementation of CREW ID input

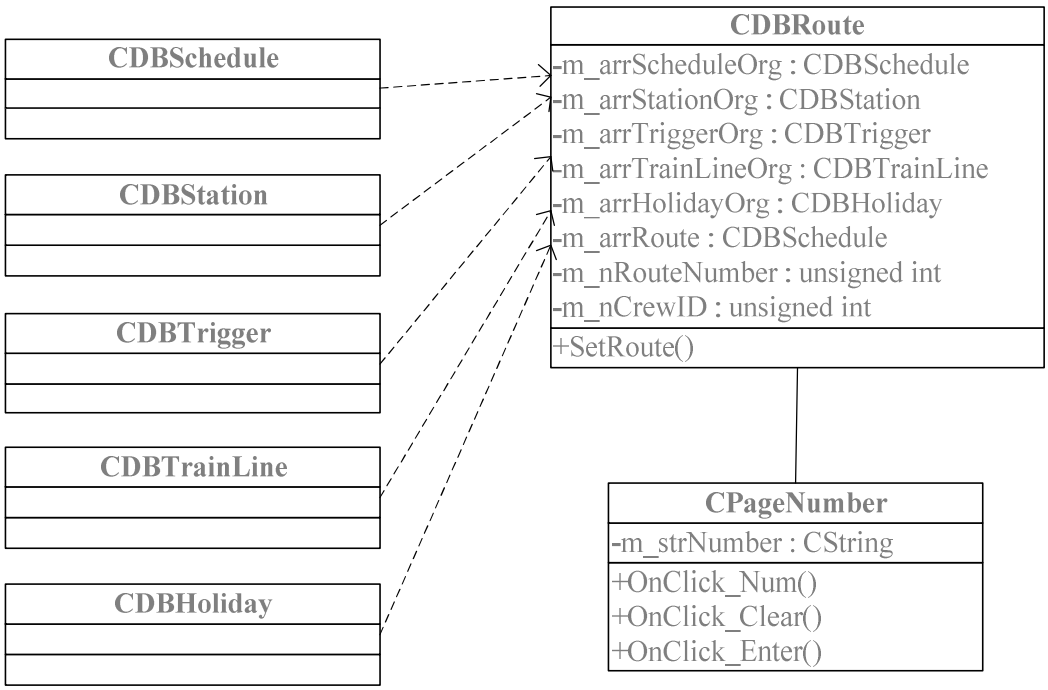


Figure 59. Class Diagram

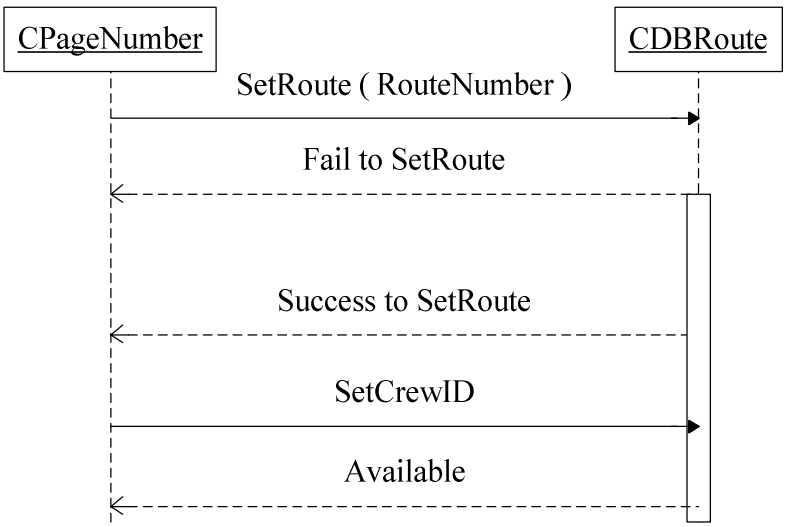


Figure 60. Sequence Diagram

4.2.2.8. Implementation of Route alteration

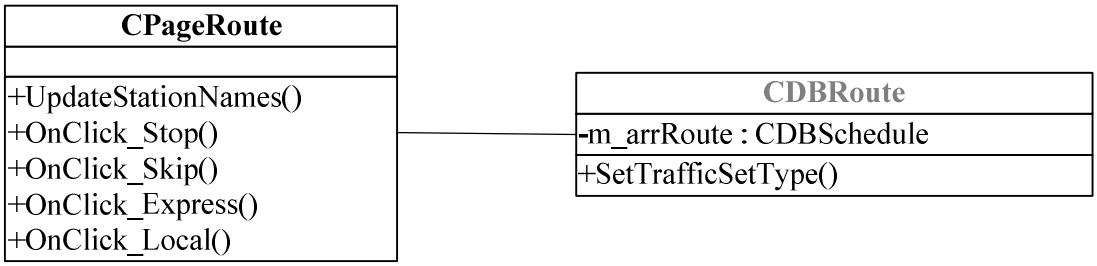


Figure 61. Class Diagram

Class Name	CCP.CPageRoute.C		
Attributes		Type	
Operations	UpdateStationNames	Return	void
	OnClick_Stop		void
	OnClick_Skip		void
	OnClick_Express		void
	OnClick_Local		void

Figure 62. Sequence Diagram

4.2.2.9. Implementation of Parameter value alteration

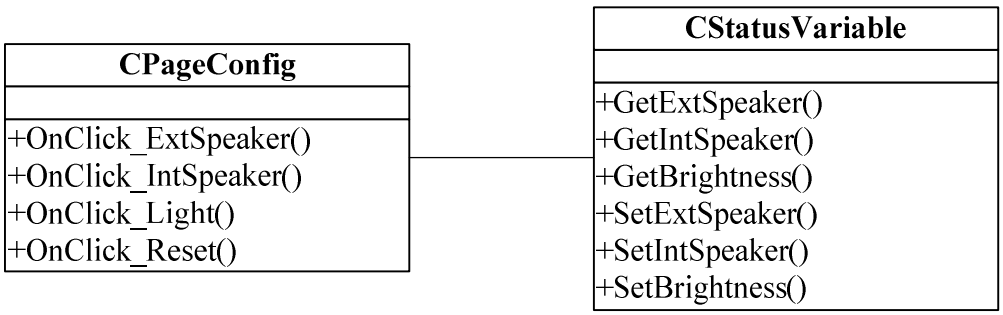


Figure 63. Class Diagram

Class Name	CCP.CPageConfig.C		
Attributes		Type	
Operations	UpdateStationNames	Return	void
	OnClick_ExtSpeaker		void
	OnClick_ IntSpeaker		void
	OnClick_Light		void
	OnClick_Reset		void

Class Name	CCP.CStatusVariable.C		
Attributes		Type	
Operations	GetExtSpeaker	Return	unsigned char
	GetIntSpeaker		unsigned char
	GetBrightness		unsigned char
	SetExtSpeaker		void
	SetIntSpeaker		void
	SetBrightness		void

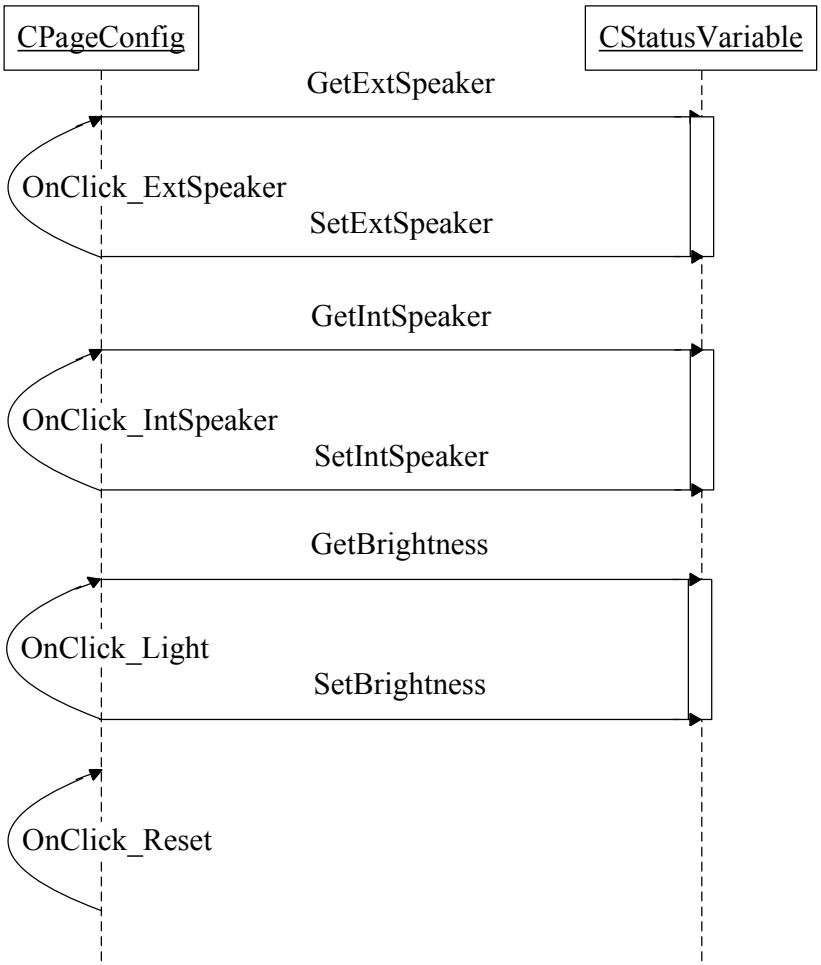


Figure 63. Sequence Diagram

4.2.2.10. Implementation of Train Information Indicator Function

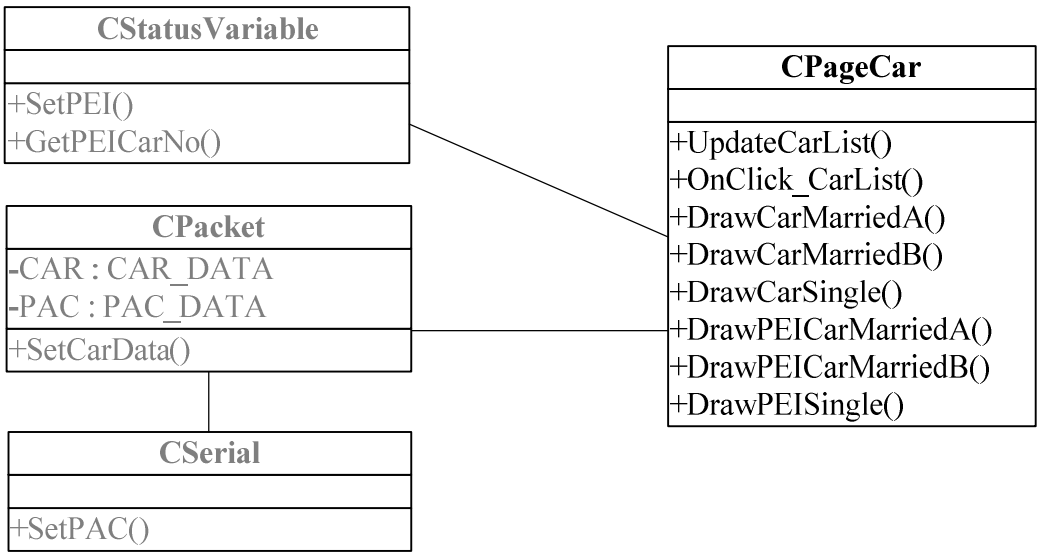


Figure 64. Class diagram

Class Name	CCP.CPageCar.C		
Attributes		Type	
Operations	UpdateCarList	Return	void
	OnClick_CarList		void
	DrawCarMarriedA		void
	DrawCarMarriedB		void
	DrawCarSignle		void
	DrawPEICarMarriedA		void
	DrawPEICarMarriedB		void
	DrawPEICarSingle		void

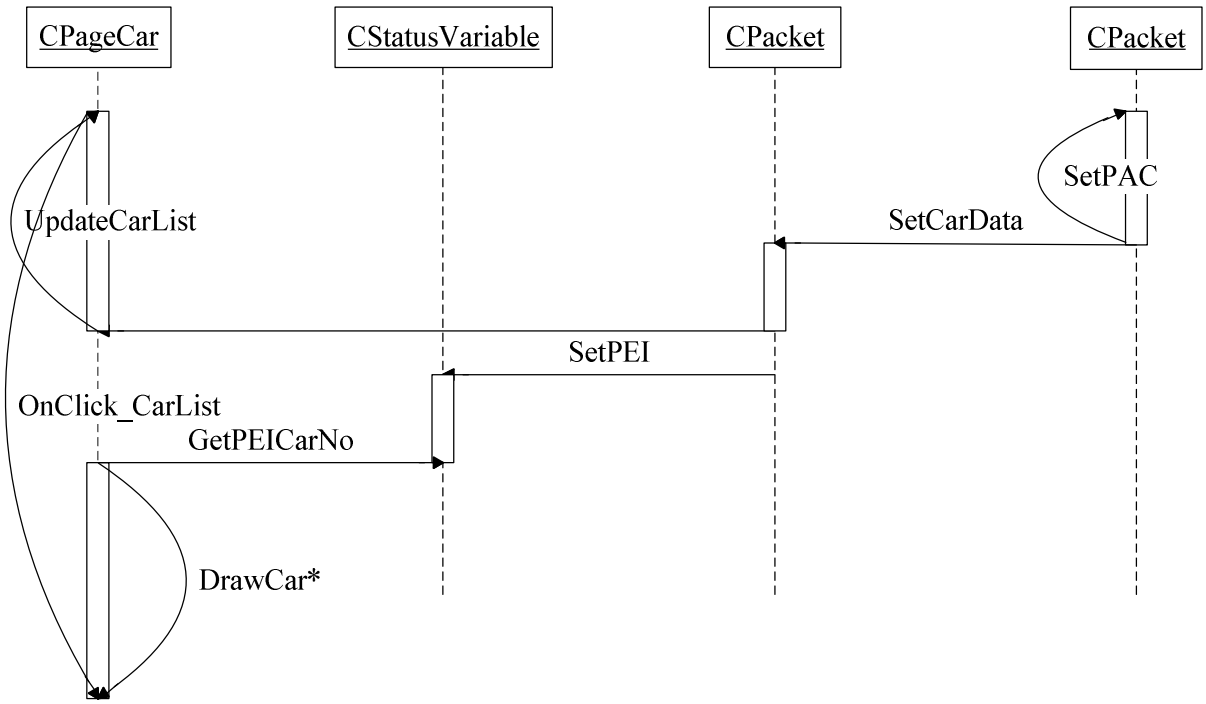


Figure 65. Sequence diagram

4.2.2.11. Implementation of HELP Indicator Function

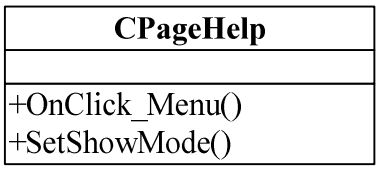


Figure 66. Class diagram

Class Name	CCP.CPageHelp.C		
Attributes		Type	
Operations	OnClick_Menu	Return	void
	SetShowMode		void

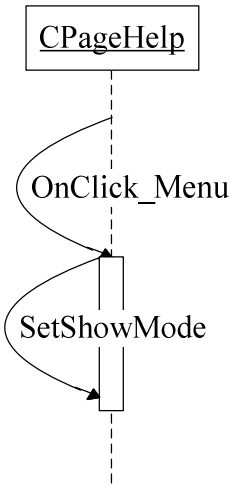


Figure 67. Sequence diagram

4.2.2.12. Implementation of PEI Indicator Function

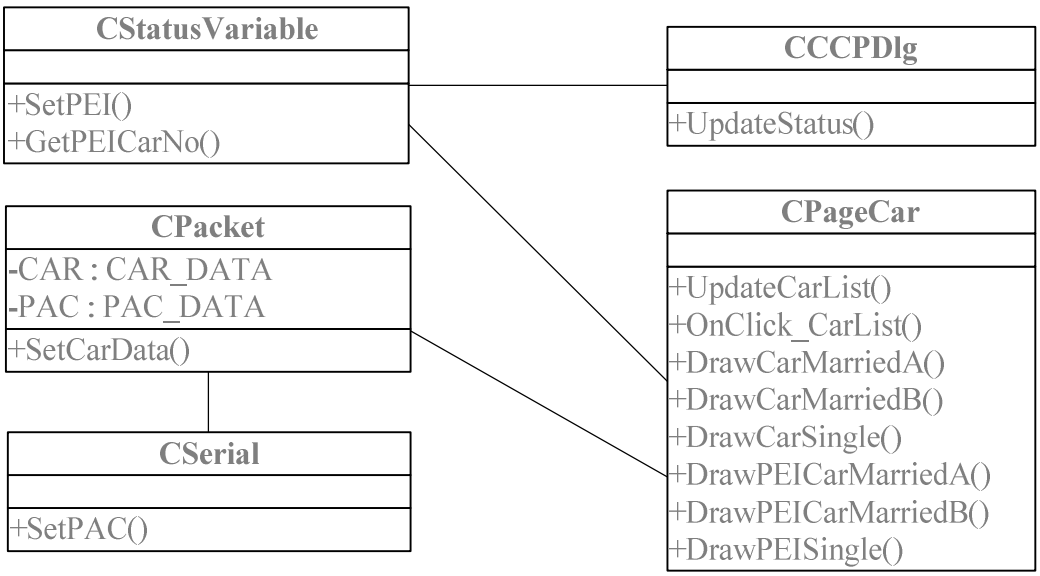


Figure 68. Class diagram

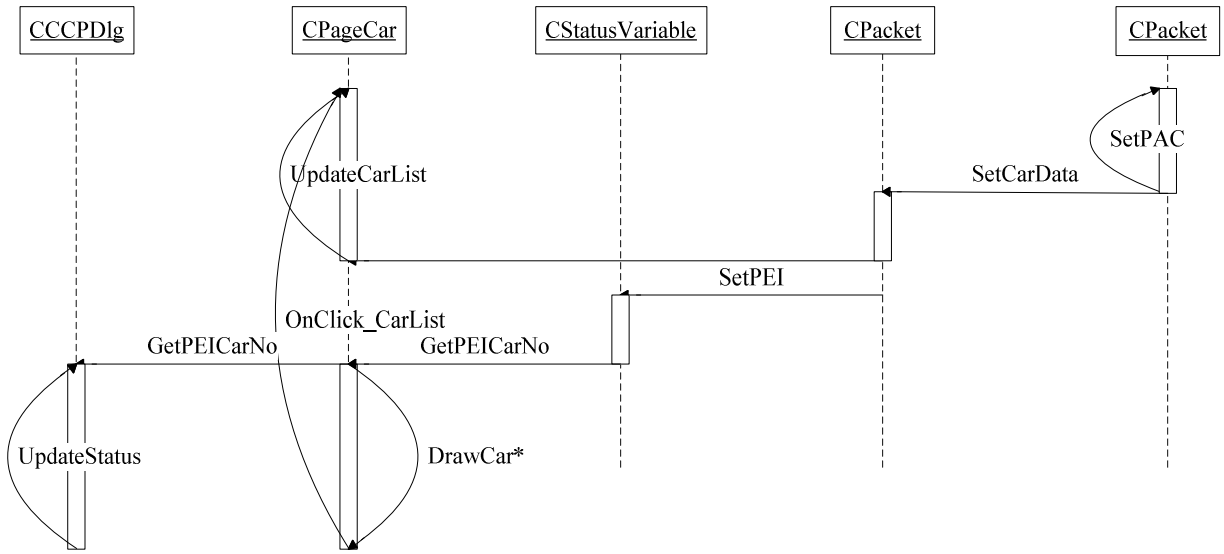


Figure 69. Sequence diagram

4.2.2.13. Route Information Indicator and Implementation of GPS Interworking

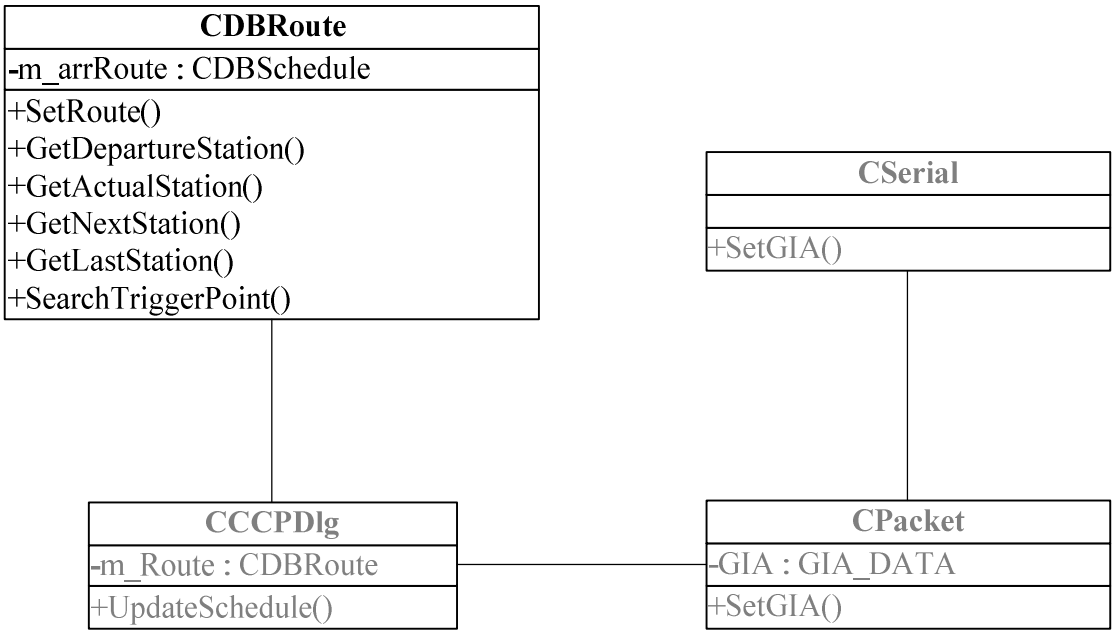


Figure 70. Class diagram

Class Name	CCP.CDBRoute.C		
Attributes	m_arrRoute	Type	CDBSchedule
Operations	SetRoute	Return	bool
	GetDepartureStation		unsigned int
	GetActualStation		unsigned int
	GetNextStation		unsigned int
	GetLastStation		unsigned int
	SearchTriggerPoint		unsigned int

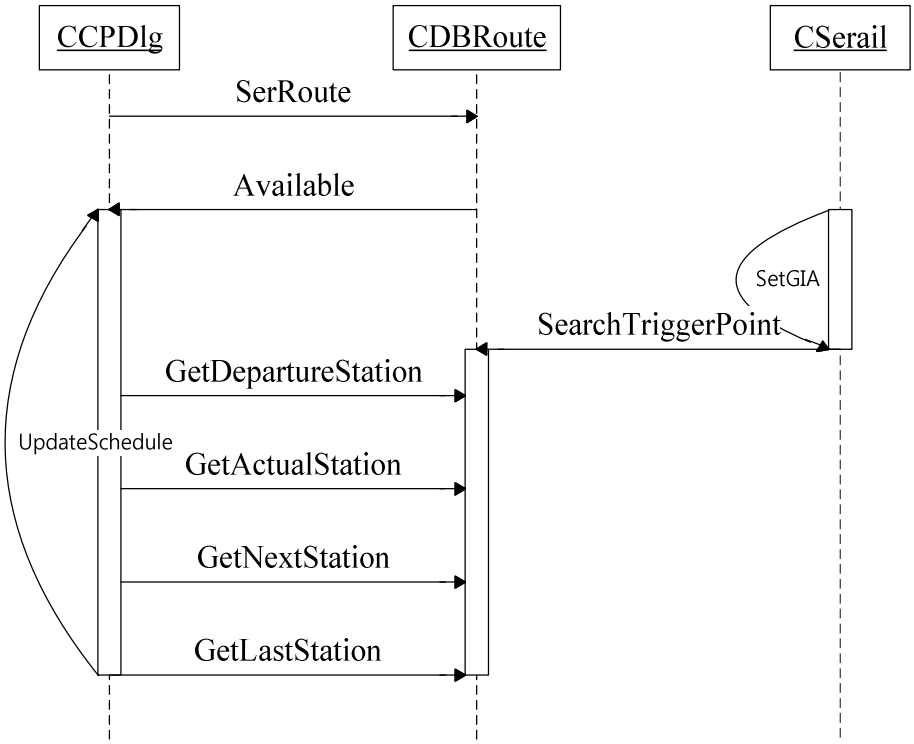


Figure 71. Sequence diagram

4.3. CNCS Description

4.3.1. Interprocess Dependency-Data Flow Diagram

4.3.1.1. CNCS (Context Diagram)

In this subsection to describe relationship among entities DFD is used.

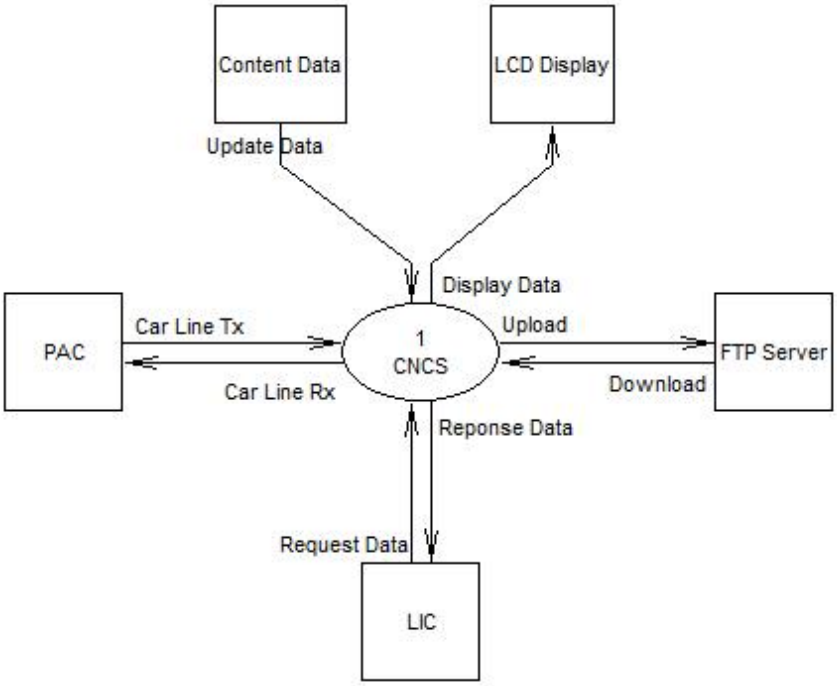


Figure 72. DFD level-0 (Context Diagram)

4.3.1.1.1. CNCS.F

Identifier	CNCS.F		
Purpose	It is responsible for controlling LCD Display broadcasting of CNCS and uploading/downloading ground Wayside and data and it also manages the contents data of PA system.		
Source	PAC	Data	Car Line Tx
	Content Data		Uptates contents data stored in HHDD of CNCS
	FTP Server		Updates contents data stored in FTP Server.
	LIC		MDS failure record information received from MDS

Target	PAC	Data	Car Line Rx
	LCD Display		LCD Display Data
	FTP Server		MDS's failure record and automatic uptates log record stored in CNCS.
	LIC		The status information of CNCS.

4.3.1.2. DFD Level-1

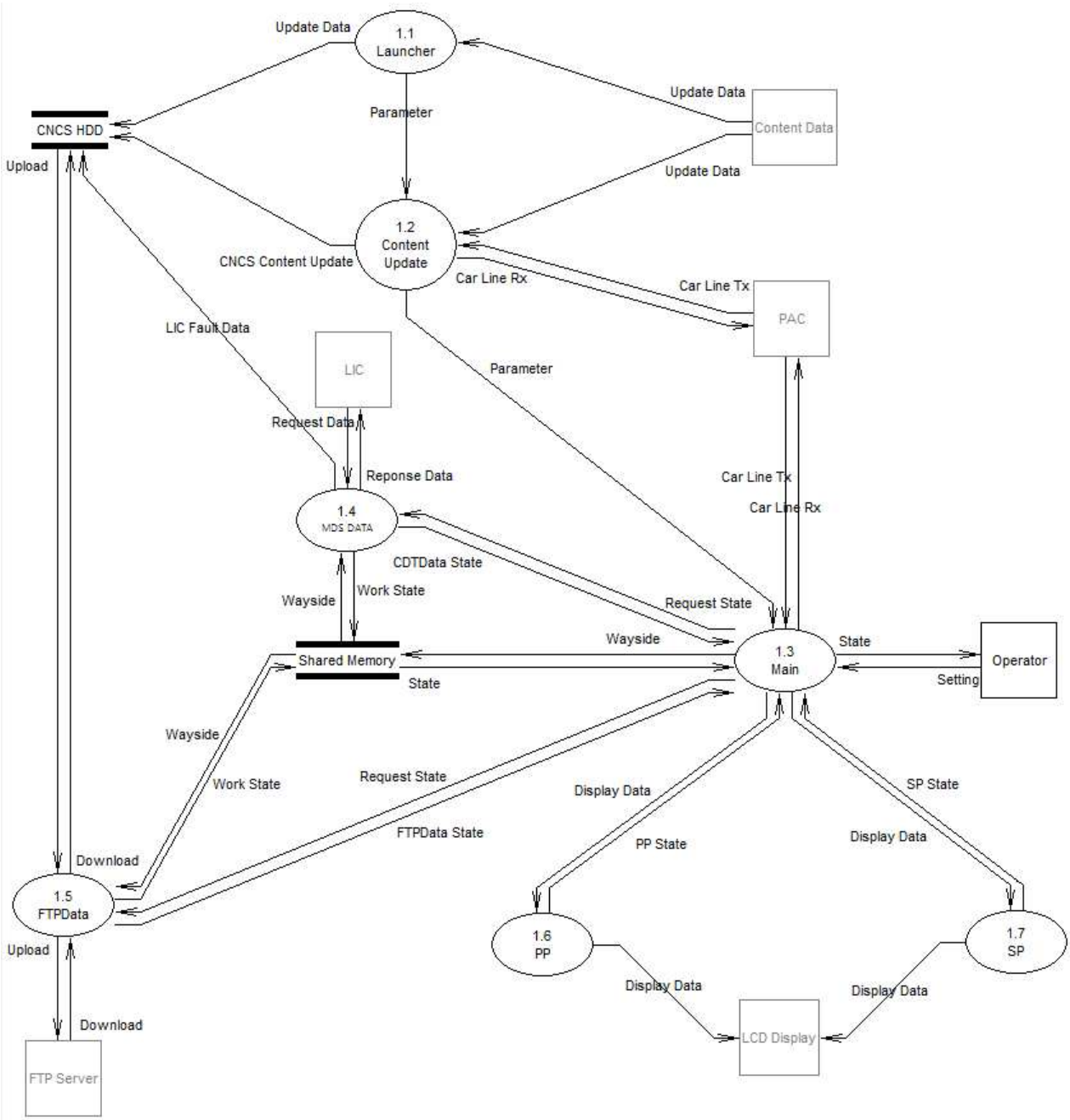


Figure 73. DFD level-1

4.3.1.2.1. Launcher.F

Identifier	Launcher.F		
Purpose	Updates CNCS operation program.		
Source	4.3.1.3. Content Data	Data	CNCS update data which receives from Wayside.
Target	CNCS HDD Storage	Data	Update CNCS operation program
	Content Update Process		After updating CNCS operation program, it transferes automatic updates Paremeter to Content Update.

4.3.1.3.1. Content Update.F

Identifier	Content Update.F		
Purpose	It updates the contents data of PA system.		
Source	4.3.1.4. Content Data	Data	Contents data that updates sub-PA system.
	Launcher		Condition Parameter running the automatic uptates of sub-system.
	PAC		PA System status information and the communication control authority via Car Line.
Target	CNCS HDD Storage	Data	CNCS content update data
	PAC		Communication control through Car Line and each sub-system content update data
	Main		It runs Main Process to control CNCS's PID and transferes condition Parameter.

4.3.1.4.1. Main.F

Identifier	Main.F		
Purpose	It is responsible for CNCS's main communication via Car Line. It controls LCD communication and checks Wayside's status to control MDS collection and FTP works.		
Source	Content Update	Data	Execution condition parameter of Main Process

	Operator		Receives broadcasting equipment set point from user.
	PAC		Receives communication data using Car Line.
	MDSData		MDSData Process status infromation
	FTPData		FTPData Process status information
	PP		PP Process status information
	SP		SP Process status information
	Shared Memory		Work state information of MDSData Process and FTPData Process
Target	Operator	Data	Broadcasting equipment status information
	PAC		Delivers broadcasting equipment set point using Car Line.
	MDSData		MDSData Process status information
	FTPData		FTPData Process status information
	PP		PP Process status information
	SP		SP Process status information
	SharedMemory		Wayside status information

4.3.1.4.2. MDSData.F

Identifier	MDSData.F		
Purpose	Manages communication and control with LIC.		
Source	Main	Data	MDSData Process status request
	Shared Memory		Wayside status information
	LIC		CNCS status request information of LIC
Target	Main	Data	MDSData Process status information
	Shared Memory		Work state information of MDS Data

			Proces
	LIC		MDSData status information
	CNCS HDD Storage		MDS failure recording data which collects from LIC.

4.3.1.4.3. FTPData.F

Identifier	FTPData.F		
Purpose	Controls Wayside and FTP communication.		
Source	Main	Data	Request FTPData Process's status
	Shared Memory		Wayside status information
	FTP Server		Content Update data
	CNCS HDD Storage		Collects MDS failure data form LIC. Record of automatic Update Logger.
Target	Main	Data	FTPData Process status information
	Shared Memory		FTPData Processwork state information
	FTP Server		Collects MDS failure data from LIC. Record of automatic Update Logger.
	CNCS HDD Storage		Content Update data

4.3.1.4.4. PP.F

Identifier	PP.F
Purpose	Manages LCD main display.

4.3.1.4.5. SP.F

Identifier	SP.F
Purpose	Manages LCD's sub-display.

4.3.1.5. Launcher

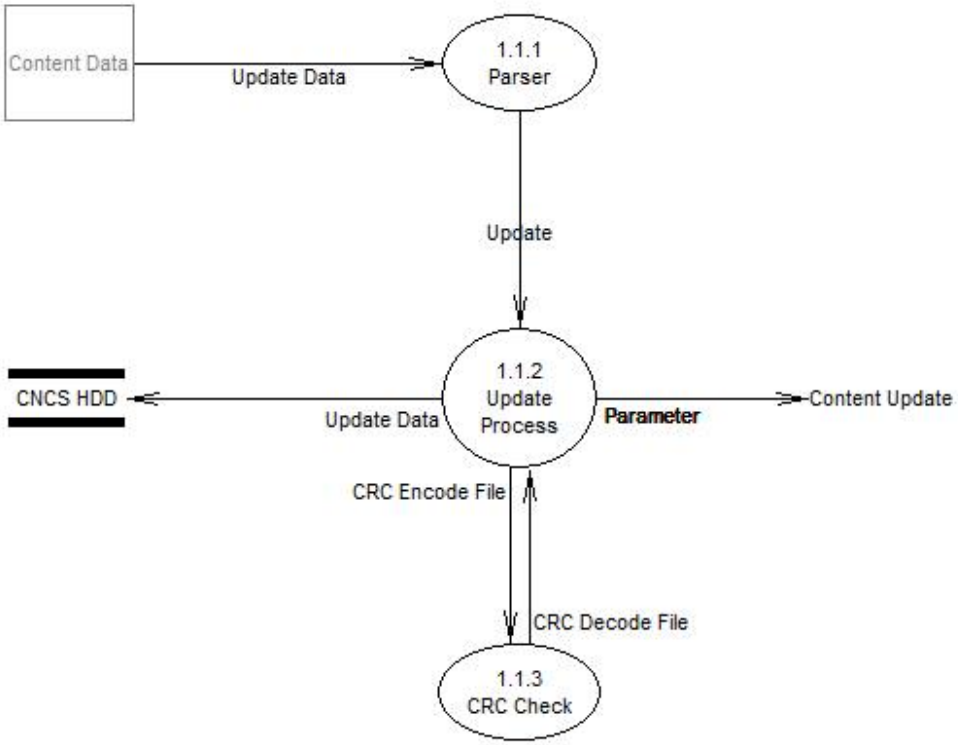


Figure 74. DFD level-2

4.3.1.5.1. Parser.F

Identifier	Parser.F		
Purpose	It analysis condition that could be Launcher program's update.		
Source	Content Data	Data	Update data file
Target	Update Process	Data	Launcher Update information

4.3.1.5.2. Update Process.F

Identifier	Update Process.F		
Purpose	Updates CNCS operation program (ContentUpdate, Main, MDSData, FTPData, PP, and SP). Executes Content Update after completed update process.		
Source	Parser	Data	Launcher update information

	CRC Check		CRC result and CRC Decoder File.
Target	CRC Check	Data	CRC Encoder File
	CNCS HDD Storage		Decodered Update Data
	Content Update		Execute ContentUpdate.exe and delivers parameter.

4.3.1.5.3. CRC Check.F

Identifier	CRC Check.F		
Purpose	Check update target file's CRC.		
Source	Update Process	Data	CRC Encoder File
Target	Update Process	Data	CRC result and CRC Decoder File.

4.3.1.6. Content Update

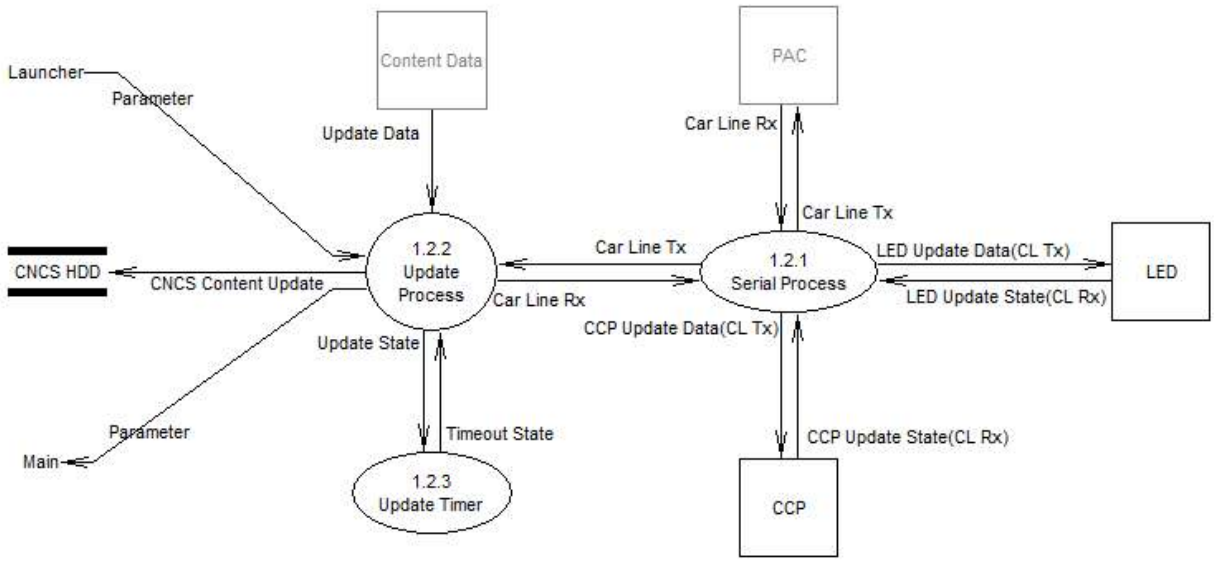


Figure 75. DFD level-3

4.3.1.6.1. Serial Process.F

Identifier	Serial Process.F
Purpose	It both receives and transmits the data necesarry for the content to

	maintain the communication authority and to update sub-system by using Car Line.		
Source	PAC	Data	Car Line Tx Data
	LED		LED content update status information in Car Line
	CCP		CCP content update status information in Car Line
	Update Process		Car Line Tx Data
Target	PAC	Data	CNCS of Car Line Rx Data
	LED		LED content update data in Car Line
	CCP		CCP content update data in Car Line
	Update Process		Car Line Rx Data.

4.3.1.6.2. Update Process.F

Identifier	Update Process.F		
Purpose	Management process for each sub-device update using Content Data.		
Source	Launcher Process	Data	Automated update execution Parameter
	Content Data		Contents data to update sub-system.
	Serial Process		Car Line Rx Data.
	Update Timer		Update status information
Target	Serial Process	Data	Car Line Tx Data.
	Update Timer		Update Timeout State.
	CNCS HDD Storage		CNCS content update data.
	Main Process		Main Process execution Parameter.

4.3.1.6.3. Update Timer.F

Identifier	Update Timer.F		
Purpose	The section that controls Time Out when Content Update.		
Source	Update Process	Data	Status information of Content update
Target	Update Process	Data	Status of Time Out Update

4.3.1.7. Main

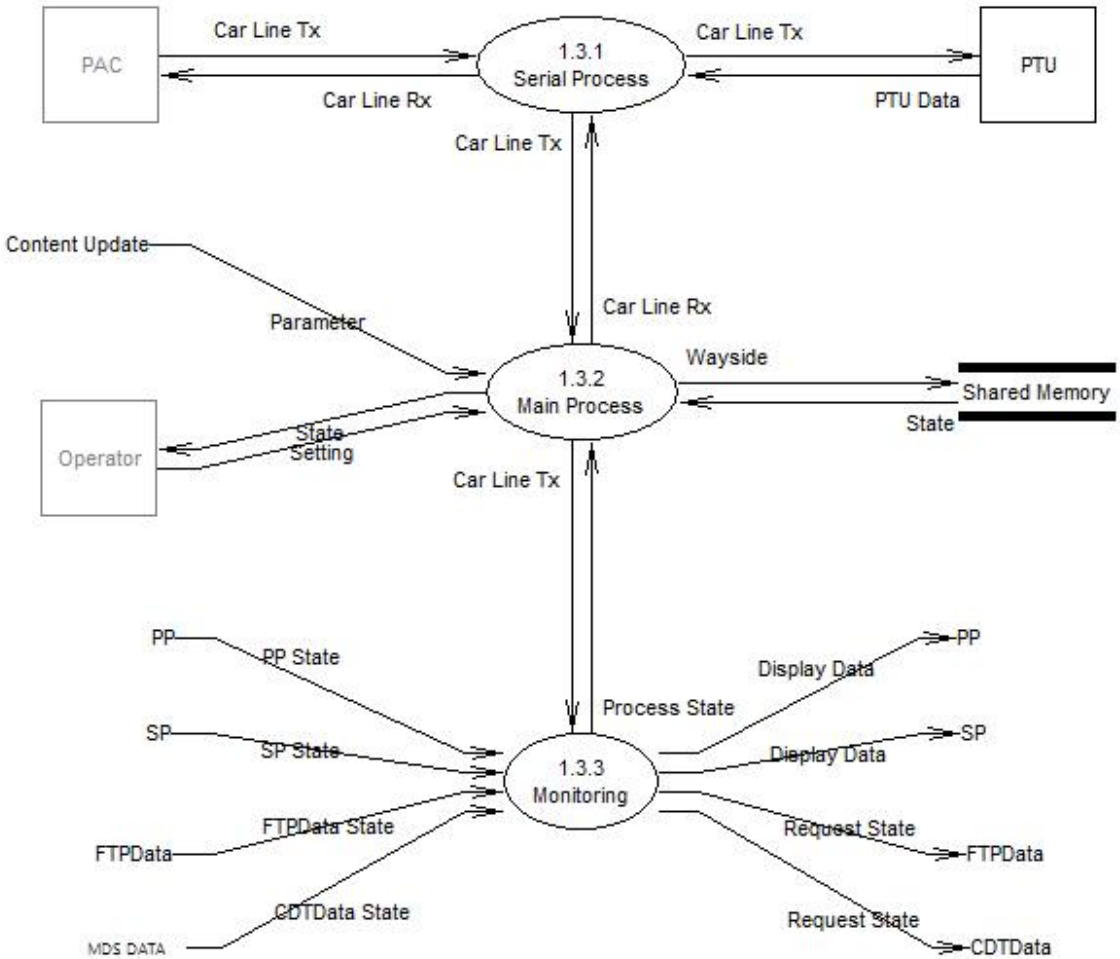


Figure 76. DFD level-4.

4.3.1.7.1. Serial Process.F

Identifier	Serial Process.F		
Purpose	Manages Car Line communication.		
Source	PAC	Data	Car Line Tx Data.
	PTU		PTU Data
	Main Process		Car Line Tx Data.
Target	PAC	Data	Car Line Rx Data
	PTU		Car Line Tx Data.
	Main Process		Car Line Rx Data.

4.3.1.7.2. Main Process.F

Identifier	Main Process.F		
Purpose	It displays the status information of PA system and processes the input from users.		
Source	Content Update	Data	Executive Parameter of Main process.
	Serial Process		Car Line Tx Data
	Shared Memory		Wayside status information.
	Monitoring		Car Line Tx Data
	Operator		PA system status information.
Target	Serial Process	Data	Car Line Rx Data
	Shared Memory		Work status information of the process of MDSData and FTP Data.
	Monitoring		It observes the action status of each Sub Process.
	Operator		It receives the volume set value of PA system.

4.3.1.7.3. Monitering.F

Identifier	Monitering.F		
Purpose	It manages the status of Sub Process.		
Source	MDSData	Data	MDSData Process status informaton.
	FTPData		FTPData Process status information
	PP		PP Process status information
	SP		SP Process status information
Target	MDSData	Data	Request State Data
	FTPData		Request State Data
	PP		Display Data
	SP		Display Data

4.3.1.8. MDSData

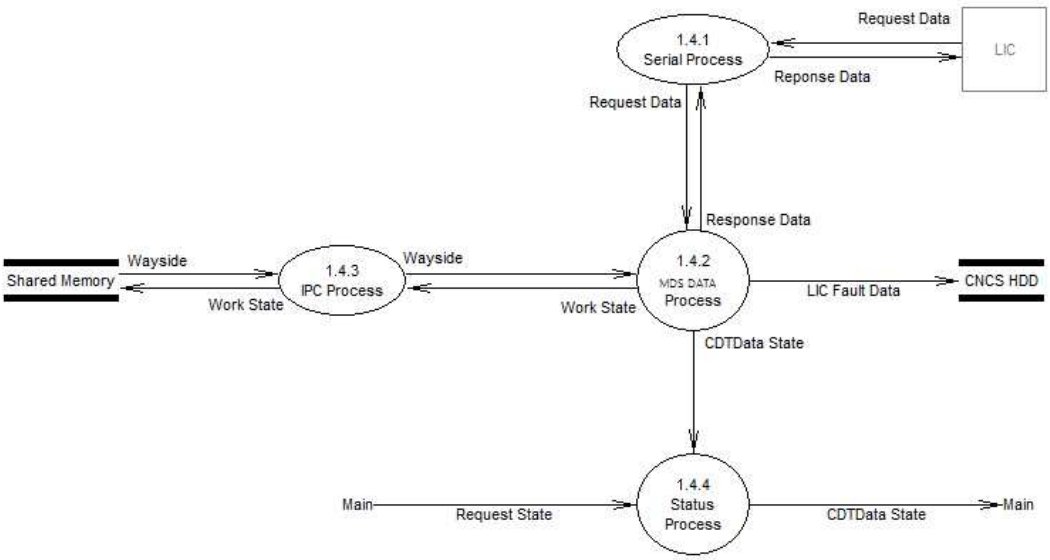


Figure 77. DFD level-5

4.3.1.8.1. Serial Process.F

Identifier	Serial Process.F		
Purpose	Process responsible for LIC and Serial communication.		
Source	LIC	Data	LIC's request data.
	MDSData Process		Response data requested by LIC
Target	LIC	Data	Response data that LIC requests.
	MDSData Process		LIC's request data.

4.3.1.8.2. MDSData Process.F

Identifier	MDSData Process.F		
Purpose	The section that controls LIC's action by Wayside status of vehicles.		
Source	Serial Process	Data	Data that LIC requests CNCS.
	IPC Process		Wayside status information.
Target	Serial Process	Data	CNCS response data that LIC requests.
	IPC Process		The work status of MDSData Process.
	Status Process		The work status information of MDSData Process.
	CNCS HDD Storage		Failure data information requested by LIC.

4.3.1.8.3. IPC Process.F

Identifier	IPC Process.F		
Purpose	The section that either receives Wayside status information from Main Process or transferes work status of existing MDSData.		
Source	Shared Memory	Data	Wayside status information
	MDSData Process		MDSData Process work state information
Target	Shared Memory	Data	MDSData Process work state information
	MDSData Process		Wayside status information

4.3.1.8.4. Status Process.F

Identifier	Status Process.F		
Purpose	It displays work status of LIC and CNCS on the screen.		
Source	MDSData Process	Data	MDSData Process status information
	Main Process		Status request information
Target	Main Process	Data	MDSData Process status information

4.3.1.9. FTPData

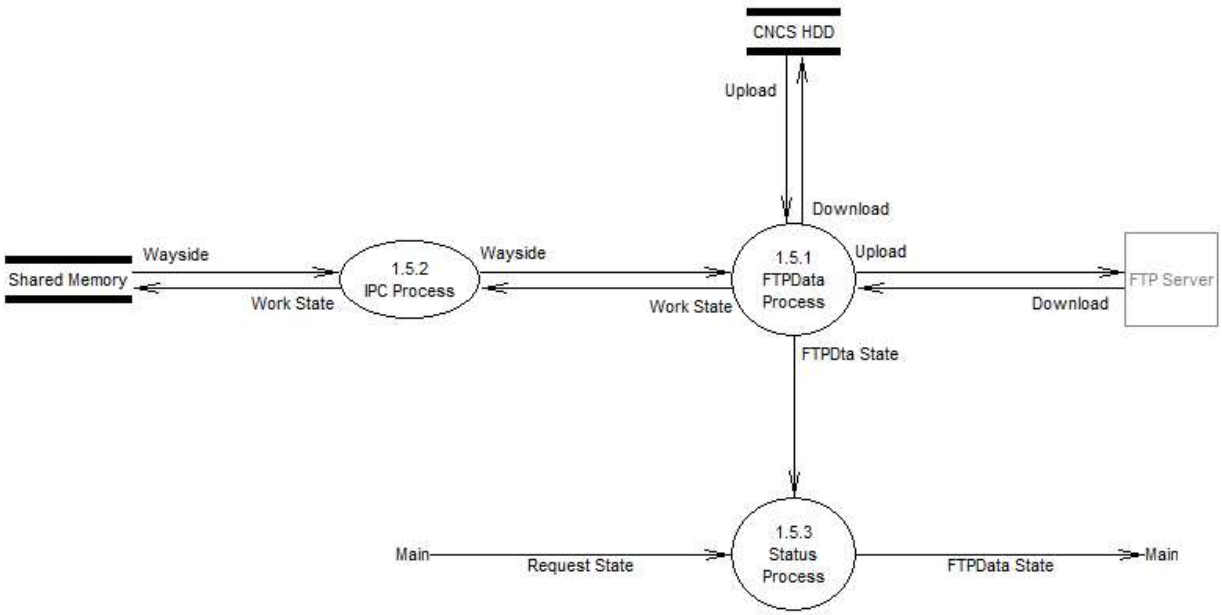


Figure 78. DFD level-6

4.3.1.9.1. FTPData Process.F

Identifier	FTPData Process.F		
Purpose	It manages the work with FTP server on the ground.		
Source	CNCS HDD Storage	Data	Data to be transfered to FTP Server.
	FTP Server		Contents data registered on FTP Server.
	IPC Process		Wayside status information.

Target	CNCS HDD Storage	Data	Contents data registered on FTP Server.
	FTP Server		Data to be transfered to FTP Server.
	IPC Process		Work status of FTPData Process.
	Status Process		Work status information of FTPData Process

4.3.1.9.2. IPC Process.F

Identifier	IPC Process.F		
Purpose	The section that either receives Wayside status information from Main Process or transferes work status of existing MDSData.		
Source	Shared Memory	Data	Wayside status information
	FTPData Process		FTPData Process work information
Target	Shared Memory	Data	FTPData Process work information
	FTPData Process		Wayside status information

4.3.1.9.3. Status Process.F

Identifier	Status Process.F		
Purpose	It displays work status of LIC and CNCS on the screen.		
Source	FTPData Process	Data	FTPData Process status infomation
	Main Process		Status request information
Target	Main Process	Data	FTPData Process status information

4.3.1.10. FTP Process.F

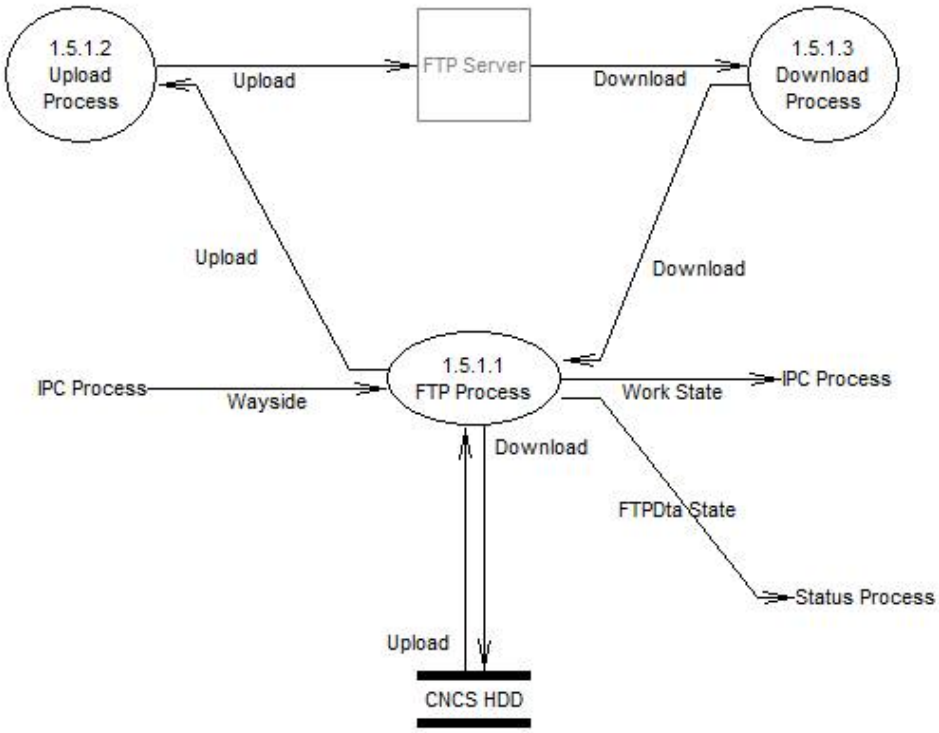


Figure 79. DFD level-7

4.3.1.10.1. FTP Process.F

Identifier	FTP Process.F		
Purpose	It manages the work between Wayside and FTP Server.		
Source	IPC Process	Data	Wayside status information.
	Download Process		Contents data received from FTP Server.
	CNCS HDD Storage		Data to be transfered to FTP Server appointed in CNCS HDD.
Target	Upload Process	Data	Data to be transfered to FTP Server appointed in CNCS HDD(MDS record and automatic updates record.)
	IPC Process		FTP Process work state information
	Status Process		FTP Process status information

	CNCS HDD Storage		Contents data received from FTP Server.

4.3.1.10.2. Upload Process.F

Identifier	Upload Process.F		
Purpose	It uploads data to FTP Server.		
Source	FTP Process	Data	Data to be transfered to FTP Server.
Target	FTP Server	Data	Data to be transfered to FTP Server.

4.3.1.10.3. Download Process.F

Identifier	Download Process.F		
Purpose	It downloads the contents data from FTP Server.		
Source	FTP Server	Data	Contents data received from FTP Server.
Target	FTP Process	Data	Contents data received from FTP Server.

4.3.1.11. PP

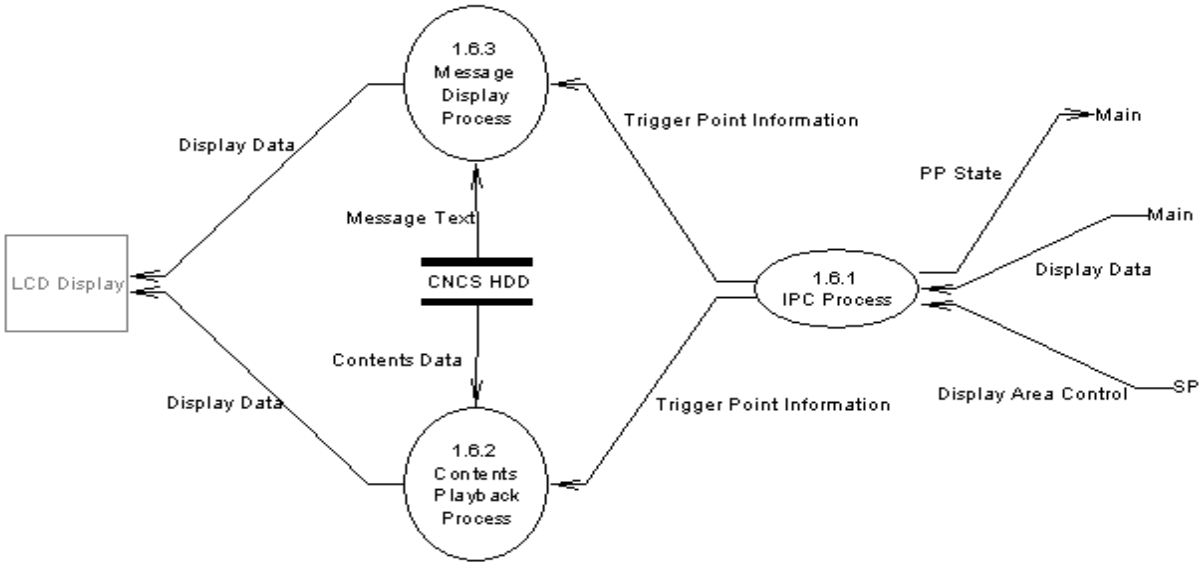


Figure 80. DFD level-8

4.3.1.11.1. IPC Process.F

Identifier	IPC Process.F		
Purpose	It both receives and transmits driving information with Main Process and it receives windows control messages from SP.		
Source	Main	Data	Trigger points reception
	SP		Display area control messages reception.
Target	Main	Data	Status data transmission.
	Message Display Process		Display passengers guide message data
	Contents Playback Process		Display Video and still image display data

4.3.1.11.2. Contents Playback Process.F

Identifier	Contents Playback Process.F

Purpose	It plays videos as listed and stops when it receives trigger points.		
Source	IPC Process	Data	Trigger points reception.
	CNCS HDD		Video, still image play data and schedules.
Target	LCD Display	Data	Video and still image data.

4.3.1.11.3. Message Display Process.F

Identifier	Message Display Process.F		
Purpose	It displays passengers guide messages by receiving trigger points.		
Source	IPC Process	Data	Trigger points reception.
	CNCS HDD		Guide messages, time display data and schedules.
Target	LCD Display	Data	Passengers guide image data.

4.3.1.12. SP

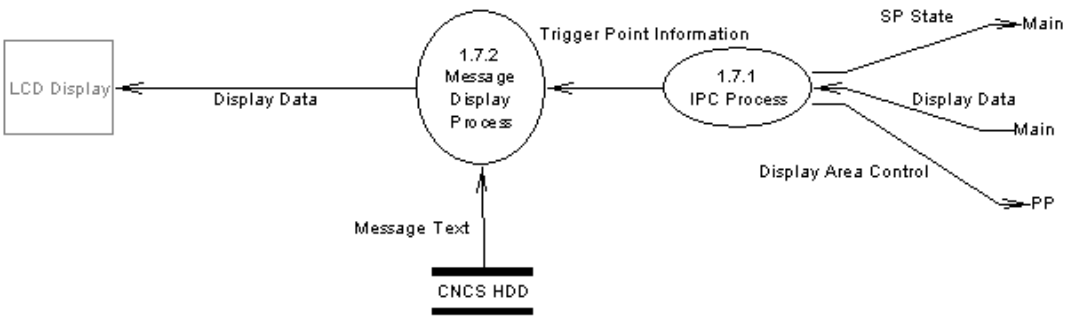


Figure 81. DFD level-9

4.3.1.12.1. IPC Process.F

Identifier	IPC Process.F
Purpose	It both receives and transmits driving information with Main Process and it receives windows control messages from SP.

Source	Main	Data	Trigger points reception.
Target	Main	Data	Status data transmission.
	Message Display Process		Trigger points transmission.
	PP		Display area change messages transmission.

4.3.1.12.2. Message Display Process.F

Identifier	Message Display Process.F		
Purpose	It displays passengers guide messages by receiving trigger points.		
Source	IPC Process	Data	Trigger points reception.
	CNCS HDD		Guide messages, time display data and schedules.
Target	LCD Display	Data	Passengers guide image data.

CPlayerTestDlg
+m_TrigCode
+m_prevTrigCode
+m_ReceiveData
+m_ScreenX
+m_ScreenY
+m_vList
+m_vTrigList
+m_hBitmapUpText
+m_hBitmapMidText
+m_hBitmapScreen
+m_hBitmapUpTextTime
+m_XMLBuffer
+nRect
+nTextCnt
+OnMediaNotify(WPARAM, LPARAN)
+LoadPlayList(LPCTSTR)
+LoadSetupFile(LPCTSTR)
+OnSocket(WPARAM, LPARAM)
+OnOneshotEvent(WPARAM, LPARAM)
+OnUpdateText(WPARAM, LPARAM)
+LoadTrigPointFile(int)
+DisplayInfoScreen()
+OnResizeScr(WPARAM, LPARAM)
+ParseStringAndGenerateBitmap()
+InitBitmap()
+GetTagType(LPCWSTR)
+GetAttrType(TagsType, LPCWSTR)
+TextDataFileLoad(LPCTSTR)
+UpdateTimeBitmap(HWND hWnd)
+TimeValidateCheck(tGIA_DATE_TIME*)

4.3.2. Class Daigram

In this paragraph, the class diagram, the class details and the sequence diagram for CCP realization function written in SRS are described.

When describing the overlap of the class diagram, text color to gray and the class details should be omitted.

4.3.2.1. Launcher

4.3.2.2. Validate & Update Software Components

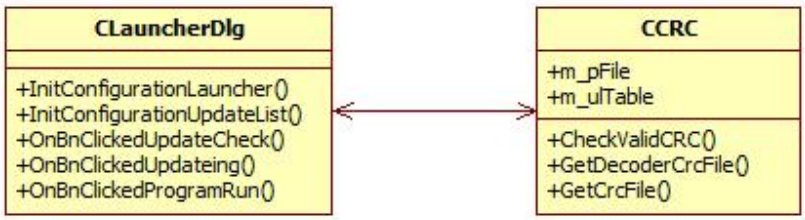


Figure 82. Class diagram about SRS 3.1.3.2-1

Class Name	CLauncherDlg		
Function	Return	Name	Desc
	void	InitConfigurationLauncher	Initialization of Launcher program.
	Void	InitConfigurationUpdateList	Searching the update program targeted and testing CRC.
	Void	OnBnClickedUpdateCheck	Searching the program updated.
	Void	OnBnClickedUpdating	Applying the program updated.
	Void	OnBnClickedProgramRun	Running ContentUpdate program.

Class Name	CCRC		
Attribute	Data type	Name	Desc
	Class	m_pFile	Managing the updates data files.
	ULONG	m_ulTable	Managing hash code for CRC Check.

Function	Return	Name	Desc
	BOOL	CheckValidCRC	Returning the CRC comparisons results of the updates files targeted.
	BOOL	GetDecoderCrcFile	The section that generates the updates files targeted in the memory.
	CFile	GetCrcFile	The section that returns the updates files targeted.

4.3.2.3. Content Update

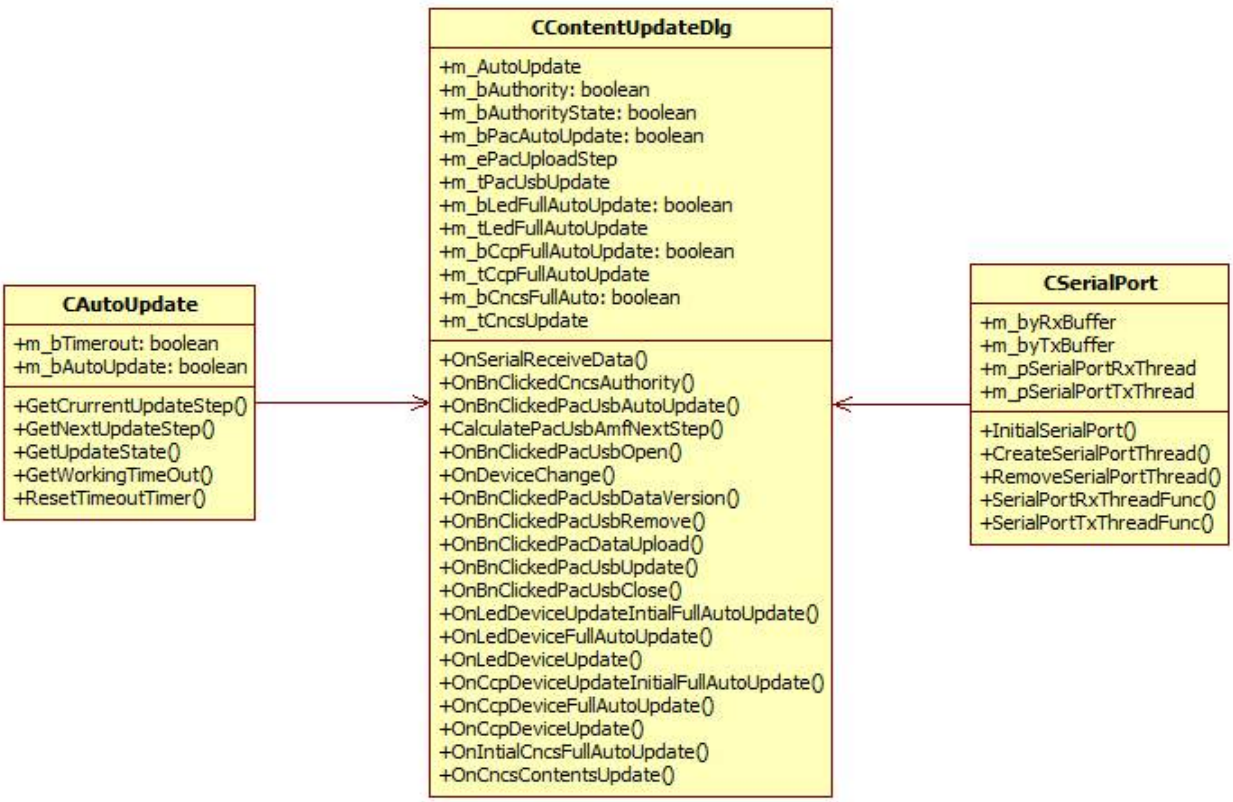


Figure 83. Content Update

Class Name	CAutoUpdate		
Attribute	Data Type	Name	Desc
	boolean	m_bTimerout	Managing automatic updates Time Out.
	boolean	m_bAutoUpdate	Managing whether automatic updates.
Function	Return	Name	Desc
	eUPDATE_STEP	GetCurrentUpdateStep	Managing existing updates Step.
	eUPDATE_STEP	GetNextUpdateStep	Managing each sub-system updates Step.
	eUPDATE_STATE	GetUpdateState	Returning existing updates status information.
	boolean	GetWorkingTimeOut	.Managing whether automatic updates' time is exceeded.
	void	ResetTimeoutTimer	initialization of Timeout Timer.

Class Name	CContentUpdateDlg		
Attribute	Data Type	Name	Desc
	class	m_AutoUpdate	The section that manages automatic updates Step.
	boolean	m_bAuthority	Request staus of authority demand.
	boolean	m_bAuthorityState	CNCS authority acquisition status information of Car Line.
	boolean	m_bPacAutoUpdate	PAC content automatic updates status
	struct	m_ePacUploadStep	Managing automatic updates Step of PAC content Data
	boolean	m_bLedFullAutoUpdate	Managing whether automatic updates of LED PID.
	struct	m_tLedFullAutoUpdate	Managing automatic updates data and status of LED PID.
	boolean	m_bCcpFullAutoUpdate	Managing whether automatic updates of CCP

	struct	m_tCcpFullAutoUpdate	Managing automatic updates data and Step of CCP.
	boolean	m_bCncsFullAuto	Managing whether automatic updates of CNCS
	struct	m_tCncsUpdate	Managing automatic updates and Step of CNCS.
Function	Return	Name	Desc
	void	OnSerialReceiveData	The section that processes the data received from Car Line.
	void	OnBnClickedCncsAuthority	The section that acquires the communication authority of Car Line from PAC.
	void	OnBnClickedPacUsbAutoUpdate	The section that automatically updates Content Update of PAC.
	void	CalculatePacUsbAmfNextStep	The section that searches the next updates work when updating Content Data of PAC.
	void	OnBnClickedPacUsbOpen	The section that links PAC's USB to CNCS.
	void	OnDeviceChange	Event when PAC's USB is connected to CNCS.
	void	OnBnClickedPacUsbDataVersion	The section that compares the version of USB Content Data of PAC.
	void	OnBnClickedPacUsbRemove	The section that removes the USB data of PAC.
	void	OnBnClickedPacDataUpload	The section that copies Content Data renewed in PAC's USB to the updates folder.
	void	OnBnClickedPacUsbUpdate	The section that copies Content Data service folder copied to the updates folder of PAC USB.
	void	OnBnClickedPacUsbClose	The section that lifts the USB connection of PAC from CNCS.
	void	OnLedDeviceUpdateIntialFullAutoUpdate	The section that prepares Content Data updates of LED.
	void	OnLedDeviceFullAutoUpdate	Setting Content data automatic updates of

			LED.
	void	OnCcpDeviceUpdateInitialFullAutoUpdate	Preparing Content Data updates of CCP.
	void	OnCcpDeviceFullAutoUpdate	Setting Content Data automatic updates of CCP.
	void	OnCcpDeviceUpdate	Running the automatic updates of CCP
	void	OnIntialCncsFullAutoUpdate	preparing the automatic updates of CNCS.
	void	OnCncsContentsUpdate	Running the automatic updates of CNCS.

Class Name	CSerialPort		
Attribute	Data Type	Name	Desc
	byte	m_byRxBuffer	Data reception buffer of Serial Port.
	byte	m_byTxBuffer	Data transmission buffer of Serial Port
	class	m_pSerialPortRxThread	Serial Port data reception Thread
	class	m_pSerialPortTxThread	Serial Port data transmission Thread
Function	Return	Name	Desc
	boolean	InitialSerialPort	The section that initializes Serial Port.
	void	CreateSerialPortThread	The section that generates Serial Port Thread.
	void	RemoveSerialPortThread	The section that removes Serial Port Thread.
	UINT	SerialPortRxThreadFunc	Process that receives Serial data.
	UINT	SerialPortTxThreadFunc	Process that transmits Serial data.

4.3.2.4. Main

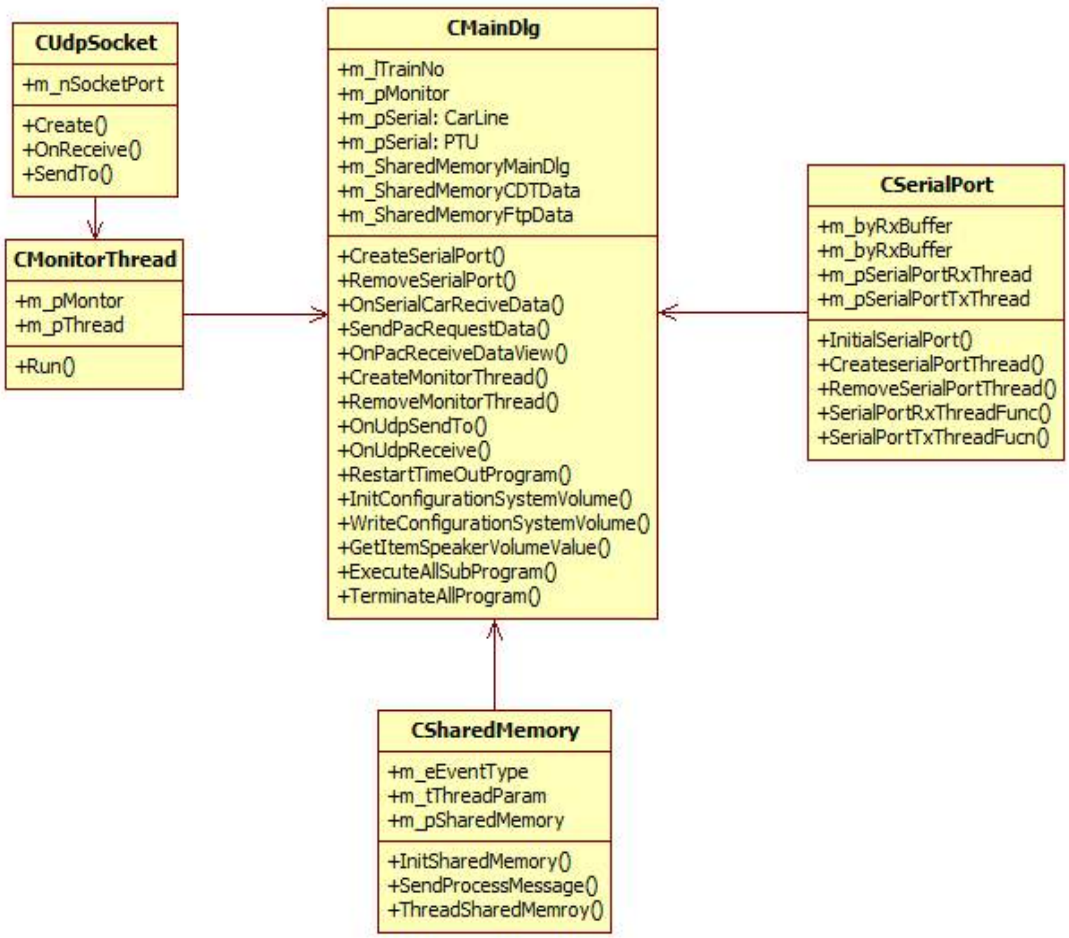


Figure 84. Main.

Class Name	CUdpSocket		
Attribute	Data Type	Name	Desc
	int	m_nSocketPort	It stores the UDP communication Socket Port number.
Function	Return	Name	Desc
	boolean	Create	The section that generates the IDP communication socket.
	void	OnReceive	The section that receives data from the UDP

			socket.
	int	SendTo	The section that transferes data to the UDP socket.

Class Name	CMonitorThread		
Attribute	Data Type	Name	Desc
	struct	m_pMonitor	It stores the program information to be monitoring.
	class	m_pThread	Monitoring Thread Process.
Function	Return	Name	Desc
	UINT	Run	Monitoring Sub Process by using UDP.

Class Name	CMainDlg		
Attribute	Data Type	Name	Desc
	long	m_lTrainNo	The section that manages vehicle numbers.
	class	m_pMonitor	The section that manages UDP communication.
	class	m_pSerial[CarLine]	The section that manages Car Line communication.
	class	m_pSerial[PTU]	The section that manages PTU communication.
	class	m_SharedmemoryMainDlg	The section that manages Main process work of Wayside.
	class	m_SharedmemoryMDSData	The section that manages MDSData process work of Wayside.
	class	m_SharedmemoryFtpData	The section that manages FTPData process work of Wayside.
Function	Return	Name	Desc
	void	CreateSerialPort	The section that generates Serial Port.
	void	RemoveSerialPort	The section that removes Serial Port.

	LRESULT	OnSerialCarReciveData	The section that receives communication data from Car Line.
	void	SendPacRequestData	The section that transmits CNCS's set-up information from Car Line.
	void	OnPacReceiveDataView	The section that displays the data information received from Car Line.
	void	CreateMonitorThread	The section that generates Monitoring Thread of Sub Process.
	void	RemoveMonitorThread	The section that removes Monitoring Thread of Sub Process.
	LRESULT	OnUdpSendTo	The section that requests the status information of Sub Process.
	LRESULT	OnUdpReceive	The section that manages the responses of Sub Process.
	void	RestartTimeOutProgram	The section that restarts the excess process
	void	InitConfigurationSystemVolume	The section that brings speaker volume information.
	void	WriteConfigurationSystemVolume	The section that stores speaker volume information.
	byte	GetItemSpeakerVolumeValue	The section that searches speaker volume information.
	void	ExecuteAllSubProgram	The section that runs Sub Process.
	void	TerminateAllProgram	The section that ends Sub Process.

Class Name	CSharedMemory		
Attribute	Data Type	Name	Desc
	enum	m_eEventType	It stores Event occurence subject of shared memory
	struct	m_tThreadParam	It stores Thread information to share data with Sub Process.
	struct	m_pSharedMemroy	Wayside status information to share with Sub Process.

Function	Return	Name	Desc
	boolean	InitSharedMemory	The section that generates shared memory
	boolean	SendProcessMemssage	The section that transferes Message to Sub Process.
	uint	ThreadSharedMemroy	Thread that processes the event of shared memory.

Class Name	CSerialPort		
Attribute	Data Type	Name	Desc
	byte	m_byRxBuffer	Serial Port data reception buffer.
	byte	m_byTxBuffer	Serial Port data transmission buffer.
	class	m_pSerialPortRxThread	Serial Port data reception Thread.
	class	m_pSerialPortTxThread	Serial Port data transmission Thread.
Function	Return	Name	Desc
	boolean	InitialSerialPort	The section that initializes Serial Port.
	void	CreateSerialPortThread	The section that generates Serial Port Thread.
	void	RemoveSerialPortThread	The section that removes Serial Port Thread.
	UINT	SerialPortRxThreadFunc	Process that receives Serial data.
	UINT	SerialPortTxThreadFunc	Process that transmits Serial data.

4.3.2.5. MDSData

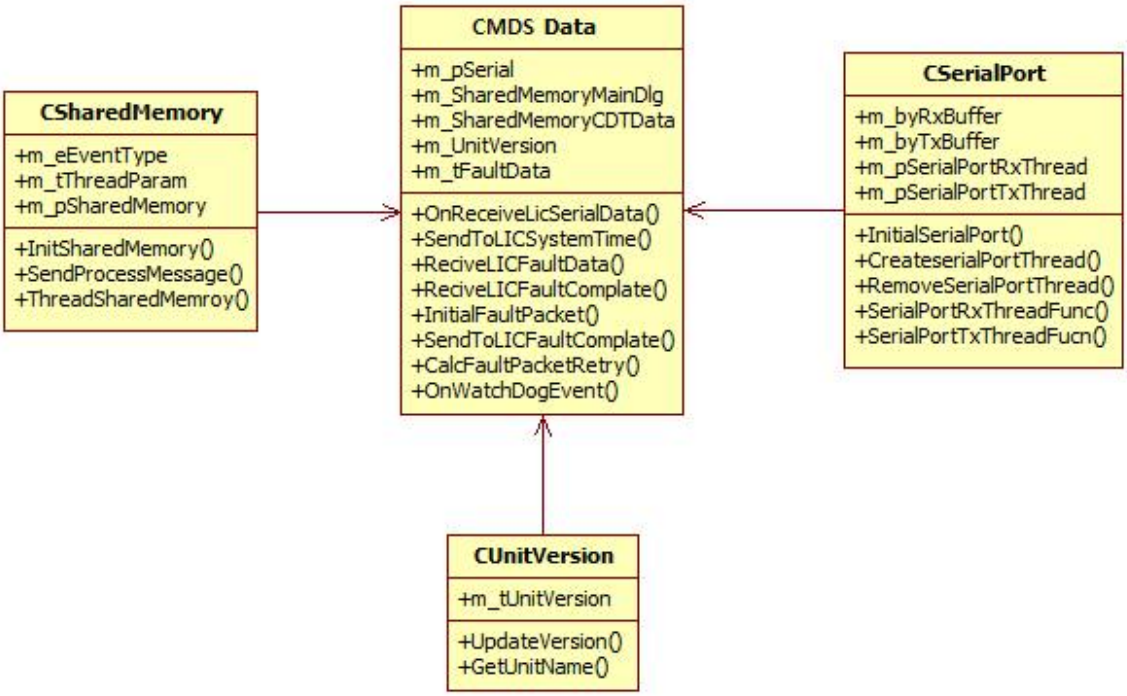


Figure 85. MDSData

Class Name	CSharedMemory		
Attribute	Data Type	Name	Desc
	enum	m_eEventType	It stores Event occurence subject of shared memory
	struct	m_tThreadParam	It stores Thread information to share data with Sub Process.
	struct	m_pSharedMemroy	Wayside status information to share with Sub Process.
Function	Return	Name	Desc
	boolean	InitSharedMemory	The section that generates shared memory.
	boolean	SendProcessMemssage	The section that transmits Message to Sub Process.
	uint	ThreadSharedMemroy	Thread that processes event of shared

			memory

Class Name	CMDSData		
Attribute	Data Type	Name	Desc
	class	m_pSerial	Class that manages LIC communication.
	class	m_SharedmemoryMainDlg	The section that transferes the work information of MDSData process to Main Process.
	class	m_SharedmemoryMDSData	The section that shares Wayside status in Main Process.
	class	m_UnitVersion	Class that manages the version information of PA system.
	struct	m_tFaultData	Clat that manages failure information received from LIC.
Function	Return	Name	Desc
	void	OnReceiveLicSerialData	In case of receiving data from LIC.
	void	SendToLICSystemTime	It transmits time and version information of CNCS to LIC.
	void	ReciveLICFaultData	In case of receiving failure information from LIC.
	void	InitialFaultPacket	It initializes the failure information reception buffer.
	void	SendToLICFaultComplate	It transmits the failure information reception result to LIC.
	boolean	CalcFaultPacketRetry	It manages re-transmitting requests of the data which ends in failure.
	void	OnWatchDogEvent	It manages WatchDog of Wayside work.

Class Name	CSerialPort		
Attribute	Data Type	Name	Desc
	byte	m_byRxBuffer	Serial Port data reception buffer.
	byte	m_byTxBuffer	Serial Port data transmission buffer.
	class	m_pSerialPortRxThread	Serial Port data reception Thread.
	class	m_pSerialPortTxThread	Serial Port data transmission Thread.
Function	Return	Name	Desc
	boolean	InitialSerialPort	The section that initializes Serial Port.
	void	CreateSerialPortThread	The section that generates Serial Port Thread.
	void	RemoveSerialPortThread	The section that removes Serial Port Thread.
	uint	SerialPortRxThreadFunc	Process that receives Serial data
	uint	SerialPortTxThreadFunc	Process that transmits Serial data.

Class Name	CUnitVersion		
Attribute	Data Type	Name	Desc
	struct	m_tUnitVersion	It stores sub-system version information.
Function	Return	Name	Desc
	void	UpdateVersion	The section that renews version information.
	String	GetUnitName	The section that manages sub-system name.

4.3.2.6. FTPData

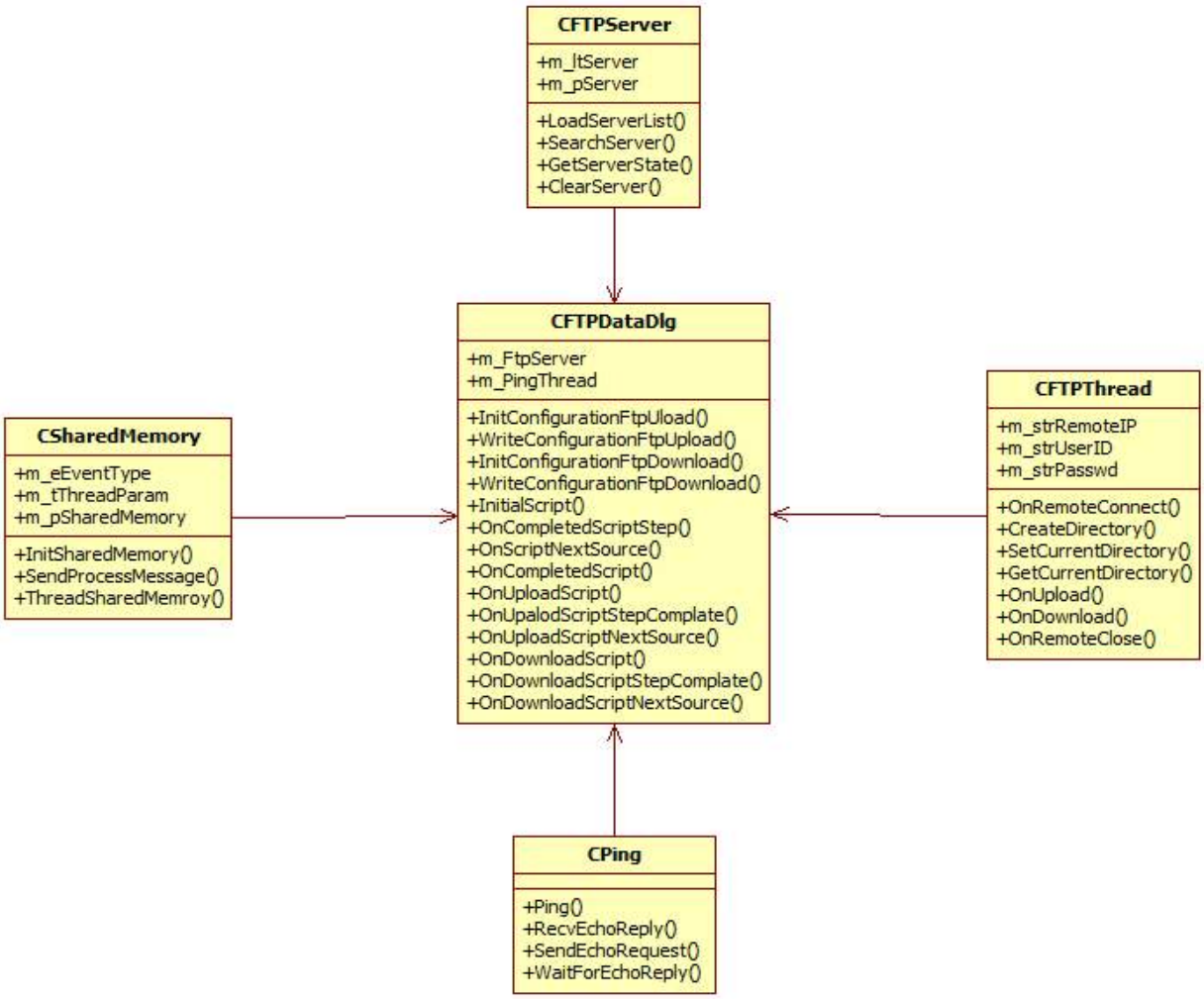


Figure 86. FTPData

Class Name	CSharedMemory		
Attribute	Data Type	Name	Desc
	enum	m_eEventType	It stores event occurence subject of shared memory.
	struct	m_tThreadParam	It stores Thread information to share information with Sub Process.
	struct	m_pSharedMemroy	Wayside status information to share with Sub Process.

Function	Return	Name	Desc
	boolean	InitSharedMemory	The section that generates shared memory.
	boolean	SendProcessMemssage	The seciton that tranferes message to Sub Process.
	uint	ThreadSharedMemroy	Thread that processes event of shared memory.

Class Name	CFTPDataDlg		
Attribute	Data Type	Name	Desc
	class	m_FtpServer	It manages FTP Server lists.
	class	m_PingThread	It manages the connection response test between FTP Server and Ping.
Function	Return	Name	Desc
	void	InitConfigurationFtpUload	It initializes information to be uploaded on FTP Server.
	void	WriteConfigurationFtpUpload	It stores Upload set-up information to FTP Server in Register.
	void	InitConfigurationFtpDownload	It initializes information to be downloaded on FTP Server.
	void	WriteConfigurationFtpDownload	It stores Download set-up information to FTP Server in Register.
	void	InitialScript	. It initializes UP/Download work information of FTP Server.
	void	OnCompletedScriptStep	The process section, if Up/download work Step of FTP Server is completed.
	void	OnCompletedScript	If the entire work of FTP Server and Up/Download is completed.
	void	OnUploadScript	The section that uploads the data appointed on FTP Server.
	void	OnUpalodScriptStepComplate	If uploading to FTP Server is completed.
	void	OnScriptNextSource	The section that searches Up/Download work target information.
	void	OnDownloadScript	The section that downloads the data

			appointed on FTP Server.
	void	OnDownloadScriptStepComplate	If downloading the data from FTP Server is completed.
	void	OnDownloadScriptNextSource	It searches data information to download from FTP Server.

Class Name	CFTPServer		
Attribute	Data Type	Name	Desc
	struct	m_ltServer	It manages FTP Server information registered.
	struct	m_pServer	FTP Server information retrieved.
Function	Return	Name	Desc
	void	LoadServerList	The section that reads FTP Server information.
	struct	SearchServer	It searches FTP Server.
	boolean	GetServerState	It returns the search results of FTP Server.
	void	ClearServer	It removes the server information retrieved.

Class Name	CFTPThread		
Attribute	Data Type	Name	Desc
	string	m_strRemoteIP	Save FTP server address.
	string	m_strUserID	Save FTP server User ID.
	string	m_strPasswd	Save FTP server Password.
Function	Return	Name	Desc
	void	OnRemoteConnect	Manage connection with FTP Server.
	void	CreateDirectory	Create directory in FTP Server.
	void	SetCurrentDirectory	Move to Directory from FTP server.
	void	GetCurrentDirectory	It brings the existing route of FTP Server.
	void	OnUpload	It transmits files to FTP Server.

	void	OnDownload	It brings files from FTP Server.
	void	OnRemoteClose	It ends the connnection with FTP Server.

Class Name	CPing		
Function	Return	Name	Desc
	void	Ping	It executes Ping test on FTP Server.
	dword	RecvEchoReply	It waits for the response of Ping Packet on the IP appointed.
	int	SendEchoRequest	It transferes Ping Packet to the IP appointed.
	int	WaitForEchoReply	It waits til events of communication Socket occurs.

4.3.2.7. PP

Figure 87 PP

	CPlayerTestDlg		
Attribute	Data Type	Name	Desc
	int	m_TrigCode	Trigger code.
	int	m_prevTrigCode	Previous trigger code.
	tPAC_ALL_RECEIVE_FORMAT	m_ReceiveData	Reception structures.
	int	m_ScreenX	Screen height.
	int	m_ScreenY	Screen width.

	vector<PlayList>	m_vList	Video still image play lists.
	vector<PlayListT>	m_vTrigList	Trigger points display contents lists.
	HBITMAP	m_hBitmapUpText	Upper destination bitmap.
	HBITMAP	m_hBitmapMidText	Middle bitmap.
	HBITMAP	m_hBitmapScreen	Screen bitmap.
	HBITMAP	m_hBitmapUpTextTime	Upper destination bit map when time is indicated.
	WCHAR	m_XMLBuffer	Text files' buffer.
	RECT	nRect	Guide messages rectangle.
	int	nTextCnt	Entire text counts.
Function	Return	Name	Desc
	long	OnMediaNotify	It processes events when playing a video is completed.
	BOOL	LoadPlayList	it reads play lists.
	BOOL	LoadSetupFile	it reads setting files.
	LRESULT	OnSocket	it processes when receiving socket data.
	long	OnOneshotEvent	It processes the event reception when the message display is completed.
	long	OnUpdateText	In case of displaying messages, It processes events whenever outputting each text.
	BOOL	LoadTrigPointFile	Opens the trigger points file and checks whether it is valid or not then starts outputting if it is vaild.
	HRESULT	DisplayInfoScreen	It outputs messages on the screen or processes them to fit the screen.
	long	OnResizeScr	It processes events when receiving the screen size changes message.
	void	ParseStringAndGenerate Bitmap	Parsing computing and generating bitmap.

	void	InitBitmap	Initialzing bitmap
	TagsType	GetTagType	Getting tag type.
	AttrType	GetAttrType	Getting attributes by reading the tag.
	void	TextDataFileLoad	Loading textdata files.
	void	UpdateTimeBitmap	Updating time butmap.
	BOOL	TimeValidityCheck	Checking if time is vaild.

4.3.2.8. SP

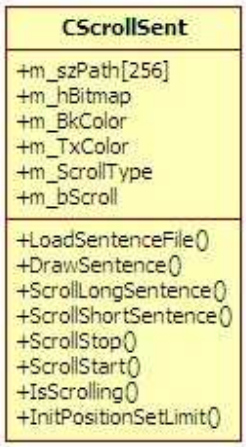


Figure 88. SP

Class Name	CScrollSent		
Attribute	Data Type	Name	Desc
	TCHAR	m_szPath[256];	The route that text files are located.
	HBITMAP	m_hBitmap	Grill bitmap.
	Color	m_BkColor	Background color.
	Color	m_TxColor	Foreground color.
	ScrollType	m_ScrollType	Text color.
	BOOL	m_bScroll	Whether scroll or not.

Function	Return	Name	Desc
	BOOL	LoadSentenceFile	Loading sectence files.
	HBITMAP	DrawSentence	Drawing sentences on the bitmap.
	BOOL	ScrollLongSentence	Scrolling a long sentence.
	BOOL	ScrollShortSentence	Upscrolling a short sentence.
	BOOL	ScrollStop	Stop scrolling.
	BOOL	ScrollStart	Start scrolling.
	BOOL	IsScrolling	Returns whether it is scrolling or not
	BOOL	InitPositionSetLimit	Decides whether It is upscroll or leftscroll.

4.4. LEDC Description

This section describes dependency of LEDC software design entities using structure chart and data flow diagram.

4.4.1. Interprocess Dependency-Data Flow Diagram

In this subsection to describe relationship among entities DFD is used.

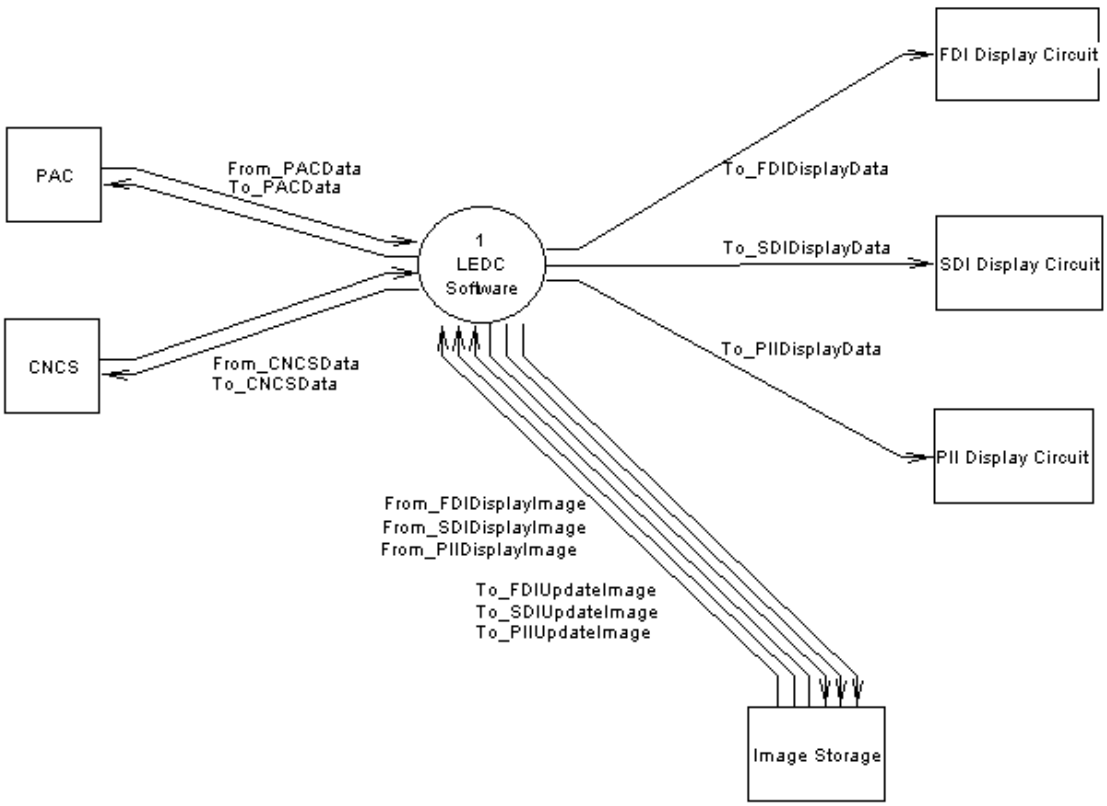


Figure 89. DFD level-0 (Context Diagram)

4.4.1.1. LEDC.Software.F

Identifier	LEDC.Software.F		
Purpose	LEDC unit reads device IDs when initializing and runs as FDI, SDI, and PII unit. It saves image data received from CNCS unit in the image storage, and displays the data retrieved from the image storage on FDI, SDI, and PII display circuit when LED display command is received.		
Source	PAC	Data	From_PACData
	CNCS		From_CNCSData
	Image Storage		From_FDIDisplayImage From_SDIDisplayImage From_PIIDisplayImage
Target	PAC	Data	To_PACData
	CNCS		To_CNCSData
	Image Storage		To_FDIDisplayImage To_SDIDisplayImage To_PIIDisplayImage
	FDI Display Circuit		To_FDIDisplayData
	SDI Display Circuit		To_SDIDisplayData
	PII Display Circuit		To_PIIDisplayData

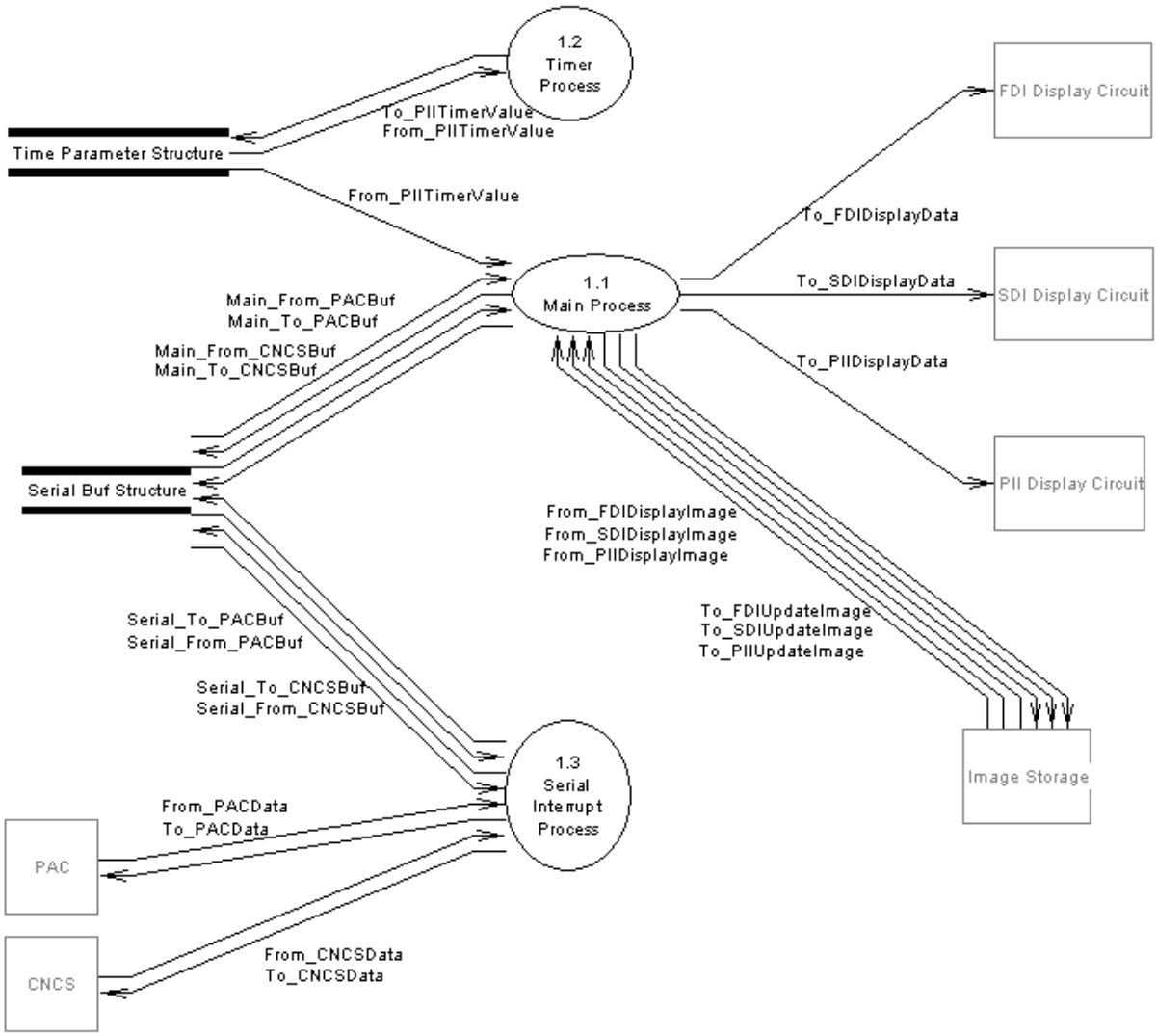


Figure 90. DFD Level-1

4.4.1.2. LEDC.MainProcess.F

Identifier	LEDC.MainProcess.F		
Purpose	It runs as FDI, SDI,and PII unit according to ID set. If PAC data of Serial buf structure is FDI or SDI unit, it displays the image retrieved from image storage on FDI, SDI display circuit. For PII, it scrolls with respect to PII Timer value. CNCS data of serial buf structure is saved in image storage.		
Source	Time Parameter Structure	Data	From_PIITimerValue

	Serial Buf Structure		Main_From_PACBuf Main_From_CNCSBuf
	Image Storage		From_FDIDisplayImage From_SDIDisplayImage From_PIIDisplayImage
Target	Serial Buf Structure	Data	Main_To_PACBuf Main_To_CNCSBuf
	Image Storage		To_FDIDisplayImage To_SDIDisplayImage To_PIIDisplayImage
	FDI Display Circuit		To_FDIDisplayData
	SDI Display Circuit		To_SDIDisplayData
	PII Display Circuit		To_PIIDisplayData

4.4.1.2.1. LEDC.TimerProcess.F

Identifier	LEDC.TimerProcess.F		
Purpose	When timer interrupt event occurs, it displays From_PIITImverValue retrieved from From_PIITimerValue in Time Parameter Structure.		
Source	Time Parameter Structure	Data	From_PIITimerValue
Target	Time Parameter Structure	Data	To_PIITimerValue

4.4.1.2.2. LEDC.SerialInterruptProcess.F

Identifier	LEDC.SerialInterruptProcess.F		
Purpose	With connection to external device thru RS485, the data from PAC is saved on PAC receved buffer on serial buf structure while the data from CNCS is saved on CNCS received buffer on serial buf structure. PAC transmitting data of serial buf structure is sent to PAC while CNCS transmitting data is sent to CNCS.		
Source	PAC	Data	From_PACData
	CNCS		From_CNCSData
	Serial Buf Structure		Serial_From_PACBuf Serial_From_CNCSBuf
Target	PAC	Data	To_PACData

	CNCS		To_CNCSData
	Serial Buf Structure		Serial_To_PACBuf Serial_To_CNCSBuf

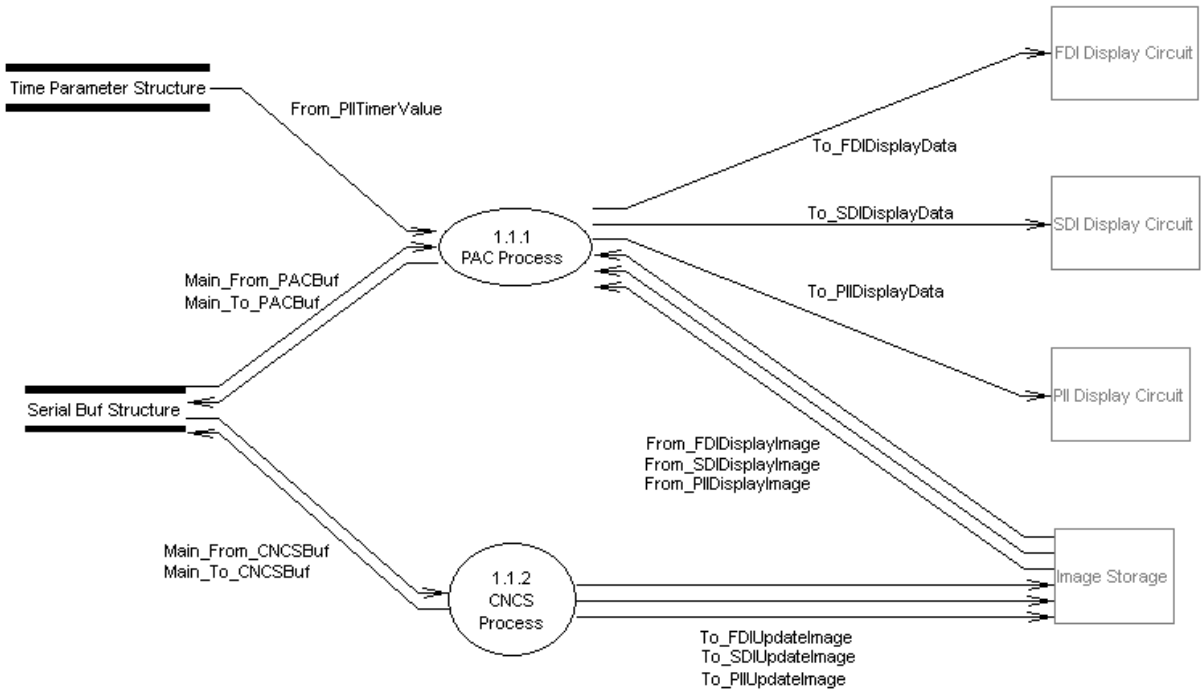


Figure 91. DFD Level-2 Process 1.1

4.4.1.3. LEDC.PACProcess.F

Identifier	LEDC.PACProcess.F		
Purpose	In case that the PAC command data of serial buf structure is display command, the image data retrieved from image storage is sent to FDI, SDI, PII display circuit. The data is saved in PAC transmitter buffer on serial buf structure for a response command.		
Source	Time Parameter Structure	Data	From_PIITimerValue
	Serial Buf Structure		Main_From_PACBuf
	Image Storage		From_FDIDisplayImage From_SDIDisplayImage

			From_PIIDisplayImage
Target	Serial Buf Structure	Data	Main_To_PACBuf
	FDI Display Circuit		To_FDIDisplayData
	SDI Display Circuit		To_SDIDisplayData
	PII Display Circuit		To_PIIDisplayData

4.4.1.3.1. LEDC.CNCSProcess.F

Identifier	LEDC.CNCSProcess.F		
Purpose	CNCS updated image from Serial Buf Structure is saved in image storage and update confirmation is sent back to Serial Buf Structure.		
Source	Serial Buf Structure	Data	Main_From_CNCSBuf
Target	Serial Buf Structure	Data	Main_To_CNCSBuf
	Image Storage		To_FDIDisplayImage To_SDIDisplayImage To_PIIDisplayImage

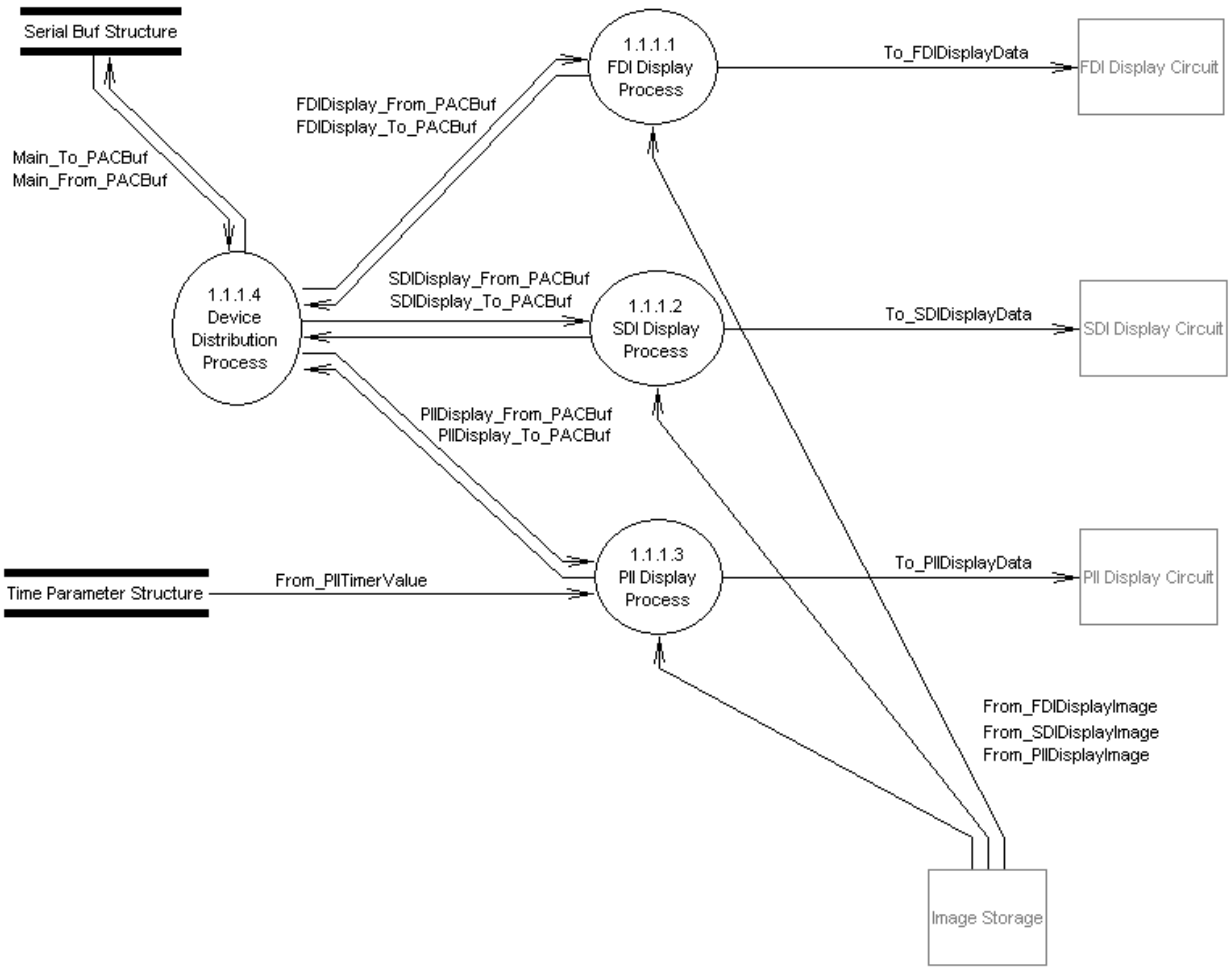


Figure 92. DFD Level-3 Process 1.1.1

4.4.1.4. LEDC.FDIDisplayProcess.F

Identifier	LEDC.FDIDisplayProcess.F		
Purpose	For feedback requested when interpreting the FDI data from Serial Buf Structure, the feedback signal is sent back to Seral Buf Structure. When receiving FDI display command, it displays the data retrieved from image storage on FDI display circuit.		
Source	Device Distribution Process	Data	FDIDisplay_From_PACBuf
	Image Storage		From_FDIDisplayImage
Target	Device Distribution Process	Data	FDIDisplay_To_PACBuf
	FDI Display Circuit		To_FDIDisplayData

4.4.1.4.1. LEDC.SDIDisplayProcess.F

Identifier	LEDC.SDIDisplayProcess.F		
Purpose	For feedback requested when interpreting SDI data from Serial Buf Structure, the feedback signal is sent back to Serial Buf Structure. When receiving SDI Display command, it displays the data retrieved from Image Storage on SDI Display Circuit.		
Source	Device Distribution Process	Data	SDIDisplay_From_PACBuf
	Image Storage		From_SDIDisplayImage
Target	Device Distribution Process	Data	SDIDisplay_To_PACBuf
	FDI Display Circuit		To_SDIDisplayData

4.4.1.4.2. LEDC.PIIDisplayProcess.F

Identifier	LEDC.PIIDisplayProcess.F		
Purpose	For feedback requested when interpreting PII data from Serial Buf Structure, the feedback signal is sent back to Serial Buf Structure. When receiving PII Display command, it displays the data retrieved from Image Storage on PII Display Circuit. After, it works according to scroll time received from Time Parameter Structure.		
Source	Device Distribution Process	Data	PIIDisplay_From_PACBuf
	Image Storage		From_PIIDisplayImage
	Time Parameter Structure		From_PIITimerValue
Target	Device Distribution Process	Data	PIIDisplay_To_PACBuf
	FDI Display Circuit		To_PIIDisplayData

4.4.1.4.3. LEDC. DeviceDistributionProcess.F

Identifier	LEDC. DeviceDistributionProcess.F		
Purpose	Virtual dummy process to separate the serial data buffer into FDI Display Process, SDI Display Process, and PII Display Process.		
Source	FDI Display Process	Data	FDIDisplay_To_PACBuf
	SDI Display Process		SDIDisplay_To_PACBuf
	PII Display Process		PIIDisplay_To_PACBuf
	Serial Buf Structure		Main_From_PACBuf

Target	FDI Display Process	Data	FDIDisplay_From_PACBuf
	SDI Display Process		SDIDisplay_From_PACBuf
	PII Display Process		PIIDisplay_From_PACBuf
	Serial Buf Structure		Main_To_PACBuf

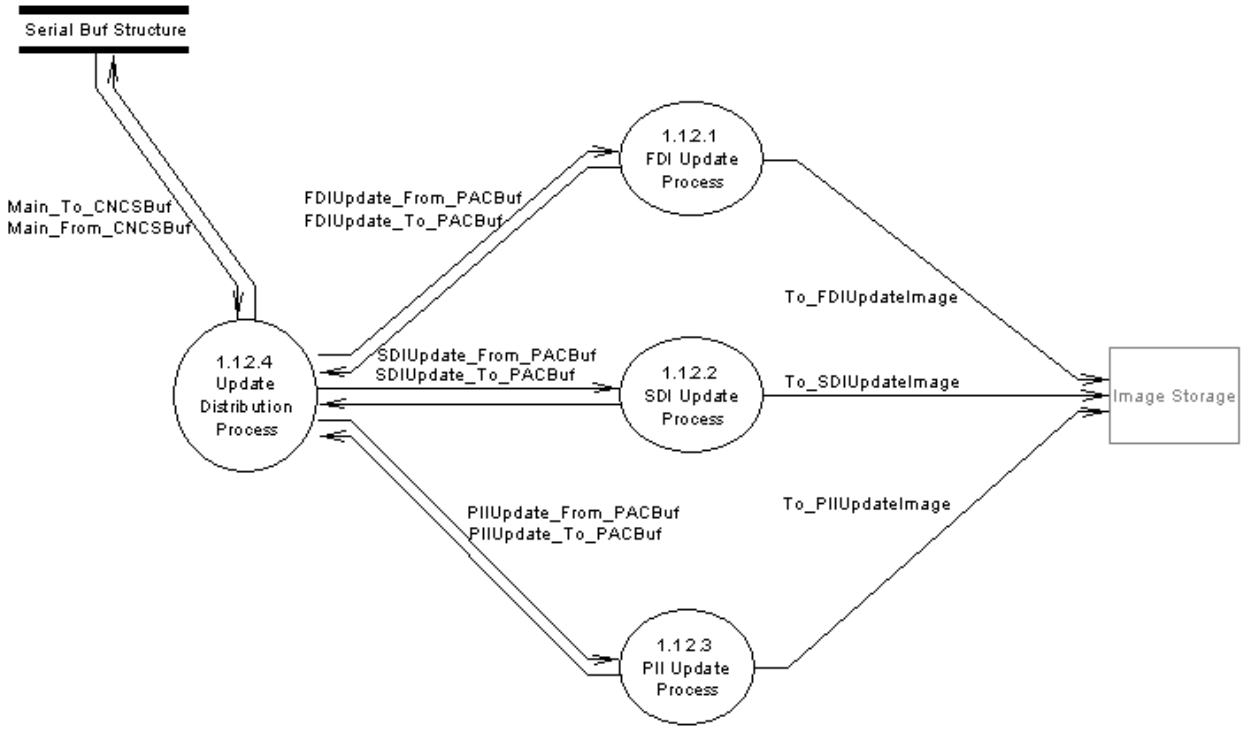


Figure 93. DFD Level-3 Process 1.1.2

4.4.1.5. LEDC.FDIUpdateProcess.F

Identifier	LEDC.FDIUpdateProcess.F		
Purpose	FDI updated image from Serial Buf Structure is saved in image storage and update confirmation is sent to back to Serial Buf Structure.		
Source	Update Distribution Process	Data	FDIUpdate_From_PACBuf
Target	Update Distribution Process	Data	FDIUpdate_To_PACBuf
	Image Storage		To_FDIUpdateImage

4.4.1.5.1. LEDC.SDIUpdateProcess.F

Identifier	LEDC.SDIUpdateProcess.F		
Purpose	SDI updated image from Serial Buf Structure is saved in image storage and update confirmation is sent to back to Serial Buf Structure.		
Source	Update Distribution Process	Data	SDIUpdate_From_PACBuf
Target	Update Distribution Process	Data	SDIUpdate_To_PACBuf
	Image Storage		To_SDIUpdateImage

4.4.1.5.2. LEDC.PIIUpdateProcess.F

Identifier	LEDC.PIIUpdateProcess.F		
Purpose	PII updated image from Serial Buf Structure is saved in image storage and update confirmation is sent to back to Serial Buf Structure.		
Source	Update Distribution Process	Data	PIIUpdate_From_PACBuf
Target	Update Distribution Process	Data	PIIUpdate_To_PACBuf
	Image Storage		To_PIIUpdateImage

4.4.1.5.3. LEDC. UpdateDistributionProcess.F

Identifier	LEDC. UpdateDistributionProcess.F		
Purpose	Virtual dummy process to separate the serial data buffer into FDI Update Process, SDI Update Process and PII Update Process.		
Source	FDI Update Process	Data	FDIUpdate_To_PACBuf
	SDI Update Process		SDIUpdate_To_PACBuf
	PII Update Process		PIIUpdate_To_PACBuf
	Serial Buf Structure		Main_From_CNCSBuf
Target	FDI Update Process	Data	FDIUpdate_From_PACBuf
	SDI Update Process		SDIUpdate_From_PACBuf
	PII Update Process		PIIUpdate_From_PACBuf
	Serial Buf Structure		Main_To_CNCSBuf

4.4.2. Interprocess Dependency-Structure Chart

4.4.2.1. In this subsection to describe relationship among entities structure chart is used.

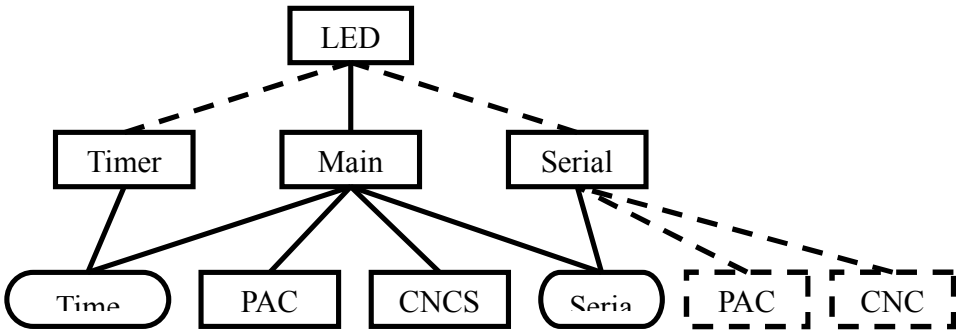


Figure 94. Structure Chart

4.4.2.1.1. LEDC Software

ID	Process Name	Identifier
	LEDC Software	LEDC.Software.F
Relation	Call	Called
	LEDC.TimerProcess.F LEDC.MainProcess.F LEDC.SerialInterruptProcess.F	-
Parameter/Return	Parameter	Return
	Void	Void

4.4.2.1.2. Timer Process

ID	Process Name	Identifier
	Timer Process	LEDC.TimerProcess.F
Relation	Call	Called
	-	LEDC.Software.F
Parameter/Return	Parameter	Return
	Void	Void

4.4.2.1.3. Main Process

ID	Process Name	Identifier
	Main Process	LEDC.MainProcess.F
Relation	Call	Called
	LEDC.PACProcess.F LEDC.CNCSProcess.F	LEDC.Software.F

Parameter/Return	Parameter	Return
	Void	Void

4.4.2.1.4. Serial Interrupt Process

ID	Process Name	Identifier
	Serial Interrupt Process	LEDC.SerialInterruptProcess.F
Relation	Call	Called
	-	LEDC.Software.F
Parameter/Return	Parameter	Return
	Void	Void

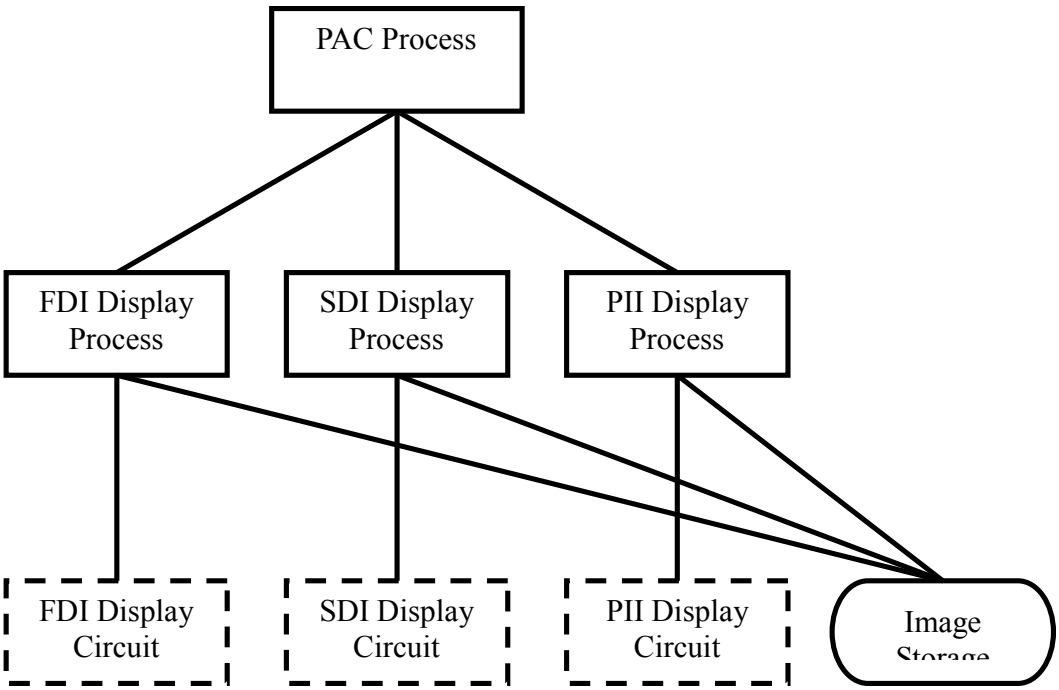


Figure 95. Structure Chart

4.4.2.2. PAC Process

ID	Process Name	Identifier
	PAC Process	LEDC.PACProcess.F
Relation	Call	Called
	LEDC.FDIDisplayProcess.F LEDC.SDIDisplayProcess.F LEDC.PIIDisplayProcess.F	LEDC.MainProcess.F
Parameter/Return	Parameter	Return
	Void	Void

4.4.2.2.1. FDI Display Process

ID	Process Name	Identifier
	FDI Display Process	LEDC.FDIDisplayProcess.F
Relation	Call	Called
	-	LEDC.PACProcess.F
Parameter/Return	Parameter	Return
	Void	Void

4.4.2.2.2. SDI Display Process

ID	Process Name	Identifier
	SDI Display Process	LEDC.SDIDisplayProcess.F
Relation	Call	Called
	-	LEDC.PACProcess.F
Parameter/Return	Parameter	Return
	Void	Void

4.4.2.2.3. PII Display Process

ID	Process Name	Identifier
	PII Display Process	LEDC.PIIDisplayProcess.F
Relation	Call	Called
	-	LEDC.PACProcess.F
Parameter/Return	Parameter	Return
	Void	Void

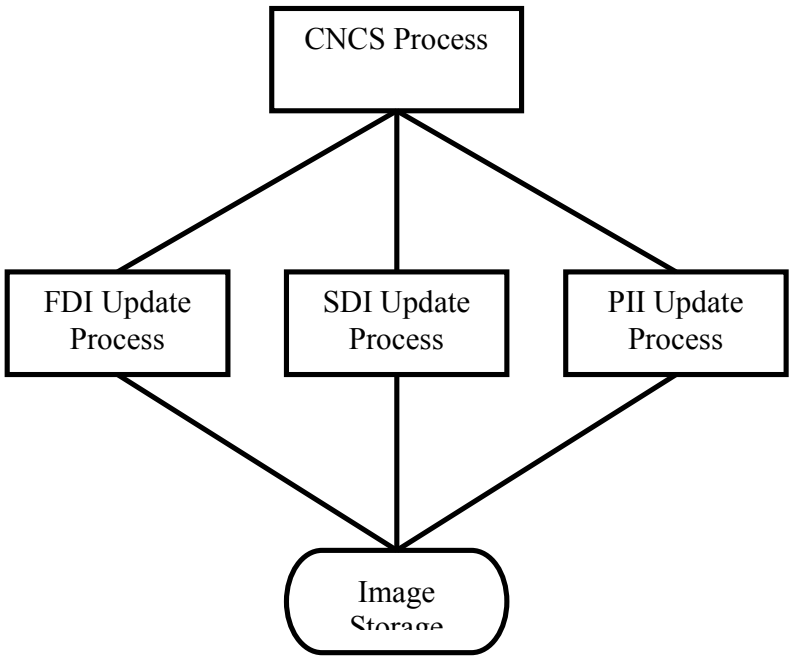


Figure 96. Structure Chart

4.4.2.3. CNCS Process

ID	Process Name	Identifier
	CNCS Process	LEDC.CNCSProcess.F
Relation	Call	Called
	LEDC.FDIUpdateProcess.F LEDC.SDIUpdateProcess.F LEDC.PIIUpdateProcess.F	LEDC.MainProcess.F
Parameter/Return	Parameter	Return
	Void	Void

4.4.2.3.1. FDI Update Process

ID	Process Name	Identifier
	FDI Update Process	LEDC.FDIUpdateProcess.F
Relation	Call	Called
	-	LEDC.CNCSProcess.F
Parameter/Return	Parameter	Return
	Void	Void

4.4.2.3.2. SDI Update Process

ID	Process Name	Identifier
	SDI Update Process	LEDC.SDIUpdateProcess.F
Relation	Call	Called
	-	LEDC.CNCSProcess.F
Parameter/Return	Parameter	Return
	Void	Void

4.4.2.3.3. PII Update Process

ID	Process Name	Identifier
	PII Update Process	LEDC.PIIUpdateProcess.F
Relation	Call	Called
	-	LEDC.CNCSProcess.F
Parameter/Return	Parameter	Return
	Void	Void

4.5. LIC Description

4.5.1. Interprocess Dependency-Data Flow Diagram

4.5.1.1. LIC (Context Diagram)

In this subsection to describe relationship among entities DFD is used.

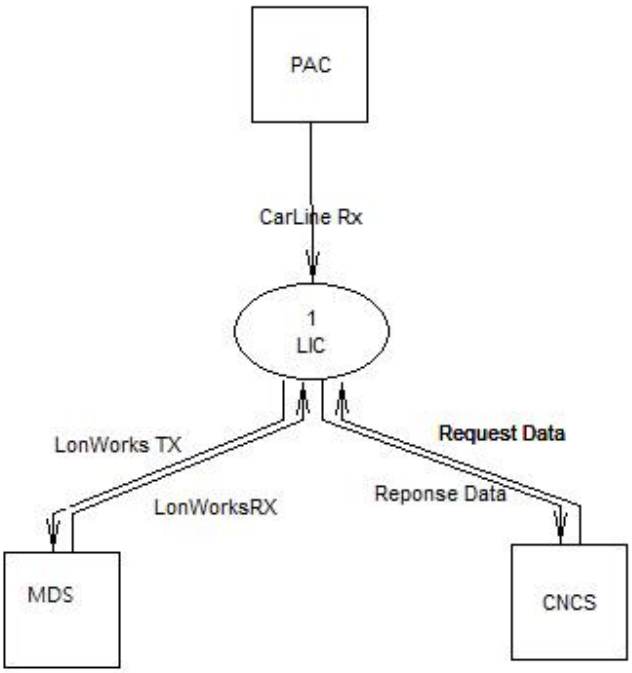


Figure 97. DFD level-0 (Context Diagram)

4.5.1.1.1. LIC.F

Identifier	LIC.F		
Purpose	It monitors each devce's status and manages MDS failure records via Car Line.		
Source	PAC	Data	Car Line Rx
	CNCS.		GPS time information reception.
	MDS		Failure record device of vehicles.
Target	CNCS	Data	Failure record data.
	MDS		Vehicles status information.

4.5.1.2. DFD level-1

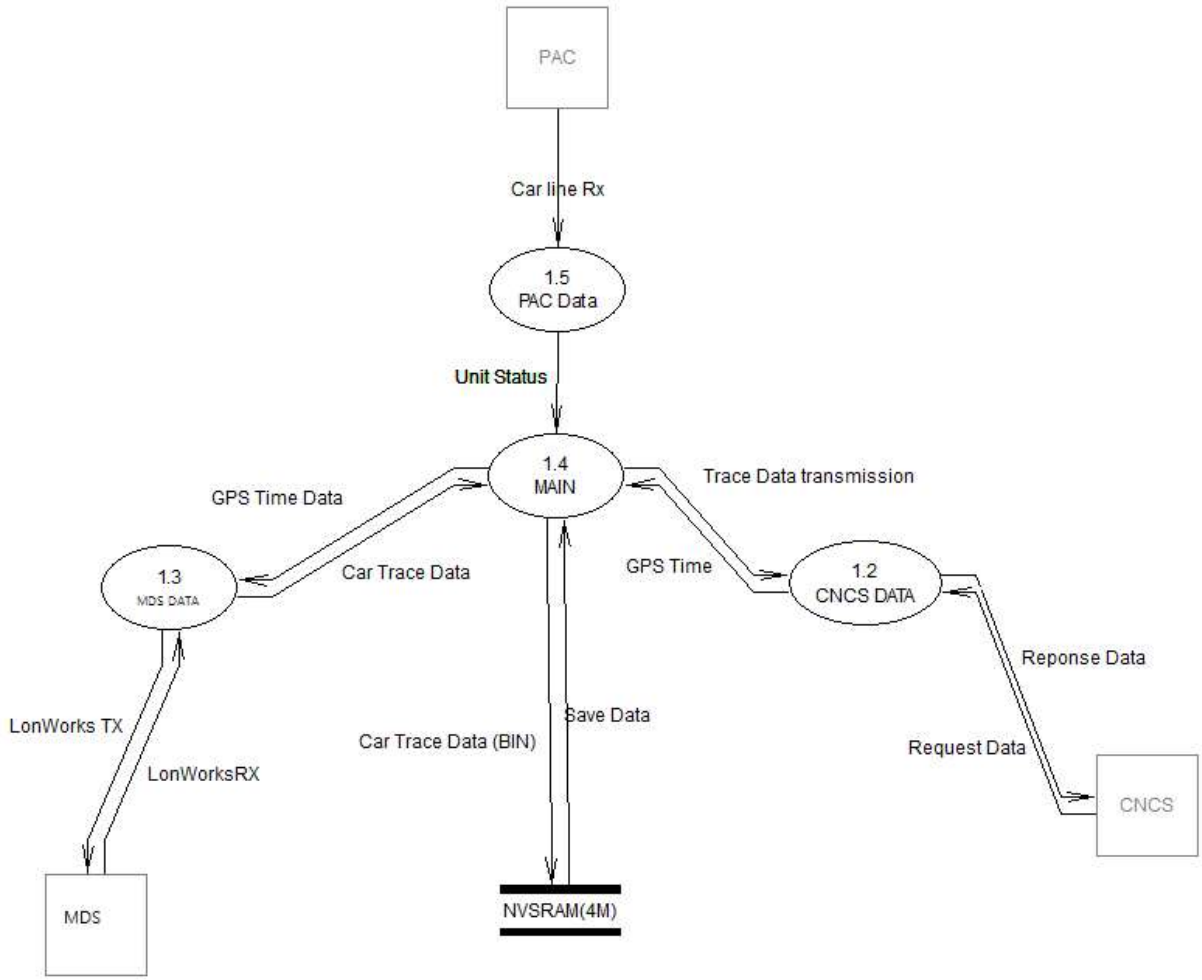


Figure 98. DFD level-1

4.5.1.2.1. PACDATA.F

Identifier	PACDATA.F		
Purpose	It manages each device's status from PAC by using Car Line.		
Source	Car Line Rx Data	Data	It checks the device status by monitoring Car Line Rx Data.
Target			

4.5.1.2.2. MDSDATA.F

Identifier	MDSDATA.F		
Purpose	It interfaces MDS LonWorks, transferes time information and transmits vehicles failure DATA to CNCS.		
Source	MDS	Data	It receives vehicles Trace Data.
	MAIN		MDS time setting Data.
Target	MDS	Data	Unit status Data.
	MAIN		Vehicles Trace Data.

4.5.1.2.3. CNCSDATA.F

Identifier	CNCSDATA.F		
Purpose	It transferes GPS Time Data given from CNCS to MDS and receives MDS failure data.		
Source	MAIN	Data	Trace Data.
	CNCS		GPS Time Data.
Target	MAIN	Data	GPS Time Data.
	CNCS		Trace Data.

4.5.1.2.4. MAIN.F

Identifier	MAIN.F		
Purpose	LIC SYSTEM OPERATION.		
Source	PACDATA	Data	Vehicles status Data.
	MDSDATA		Trace Data.
	CNCSDATA		GPS Time Data.
Target	MDSDATA	Data	GPS Time Data.
	CNCSDATA		Trace Data.

4.5.1.3. PACDATA.

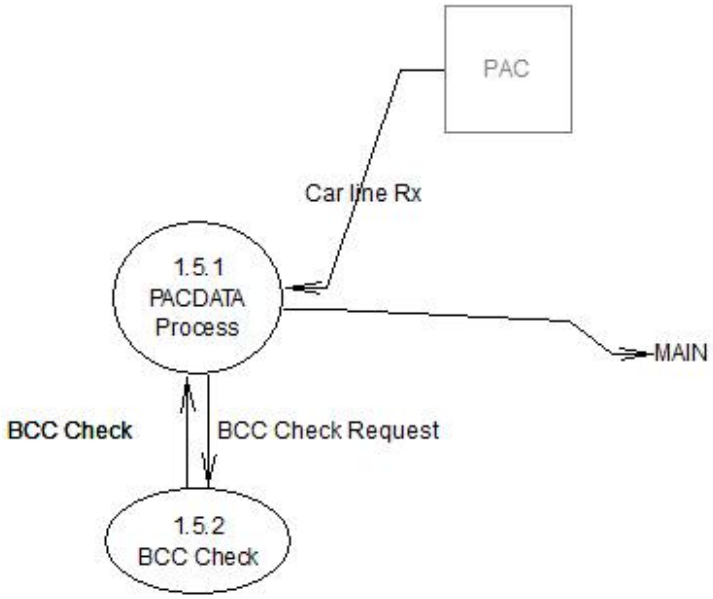


Figure 99. DFD level-2

4.5.1.3.1. PACDATA.F

Identifier	PACDATA.F		
Purpose	It transmits PAC communication protocol to device status Main after BBC Check in CAR Line.		
Source	PAC	Data	It only receives PAC Data from Car Line.
	BCCCHECK		It receives the Data completed Data validity test.
Target	BCCCHECK	Data	Requests PAC Data BCC.
	MAIN		Transmits the Data which is finally confirmed.

4.5.1.3.2. BCCCHECK.F

Identifier	BCCCHECK.F		
Purpose	It executes Car Line Data BCC test.		
Source	PACDATA	Data	BCC Encoder File.

Target	PACDATA	Data	BCC results.

4.5.1.4. MDSDATA.

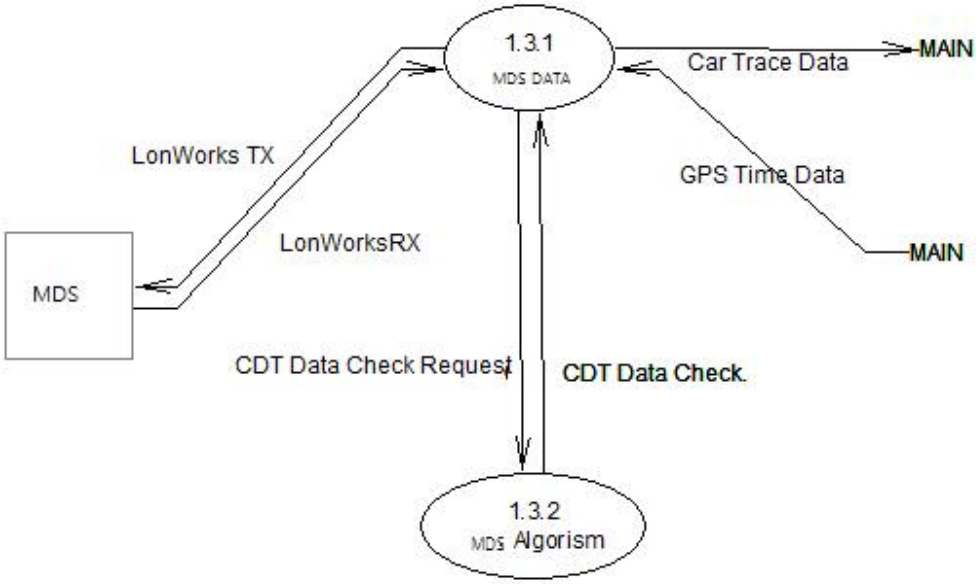


Figure 100. DFD level-3

4.5.1.4.1. MDSDATA.F

Identifier	MDSDATA.F		
Purpose	It transmits device status information to MDS and receives failure Data.		
Source	MDS	Data	Vehicles failure Data.
	MDS Algorism		It checks the validity of the failure data received.
	MAIN		GPS Time Data to be tranmitted to MDS.
Target	MDS	Data	GPS Time Data.
	MDS Algorism		It decides the suitability of the failure data received.
	MAIN		It transmits the valid failure data.

4.5.1.4.2. MDS Algorism.F

Identifier	MDS Algorism.F		
Purpose	It checks the validity of the data received from MDS.		
Source	MDSDATA	Data	MDS reception Data.
Target	MDSDATA	Data	MDS data completed confirming.

4.5.1.5. CNCSDATA.

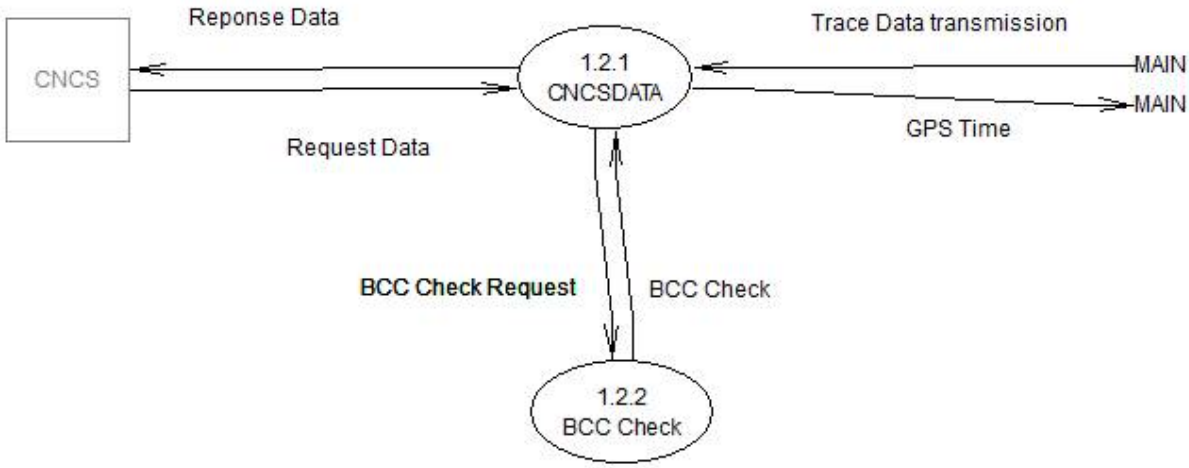


Figure 101. DFD level-4

4.5.1.5.1. CNCSDATA.F

Identifier	CNCSDATA.F		
Purpose	It receives time data from CNCS and transmits the failure data received from MDS.		
Source	BCCCHECK	Data	BCC indentification completed DATA.
	CNCS		GPS Time Data.
	MAIN		Trace Data.
Target	BCCCHECK	Data	CNCS Data.
	CNCS		Trace Data.

	MAIN		GPS Time Data.

4.5.1.5.2. BCCCHECK.F

Identifier	BCCCHECK.F		
Purpose	It executes Car Line Data BCC test.		
Source	CNCSDATA	Data	BCC Encoder File.
Target	CNCSATA	Data	**BCC 결과. BCC results.**

4.5.1.6. MAIN

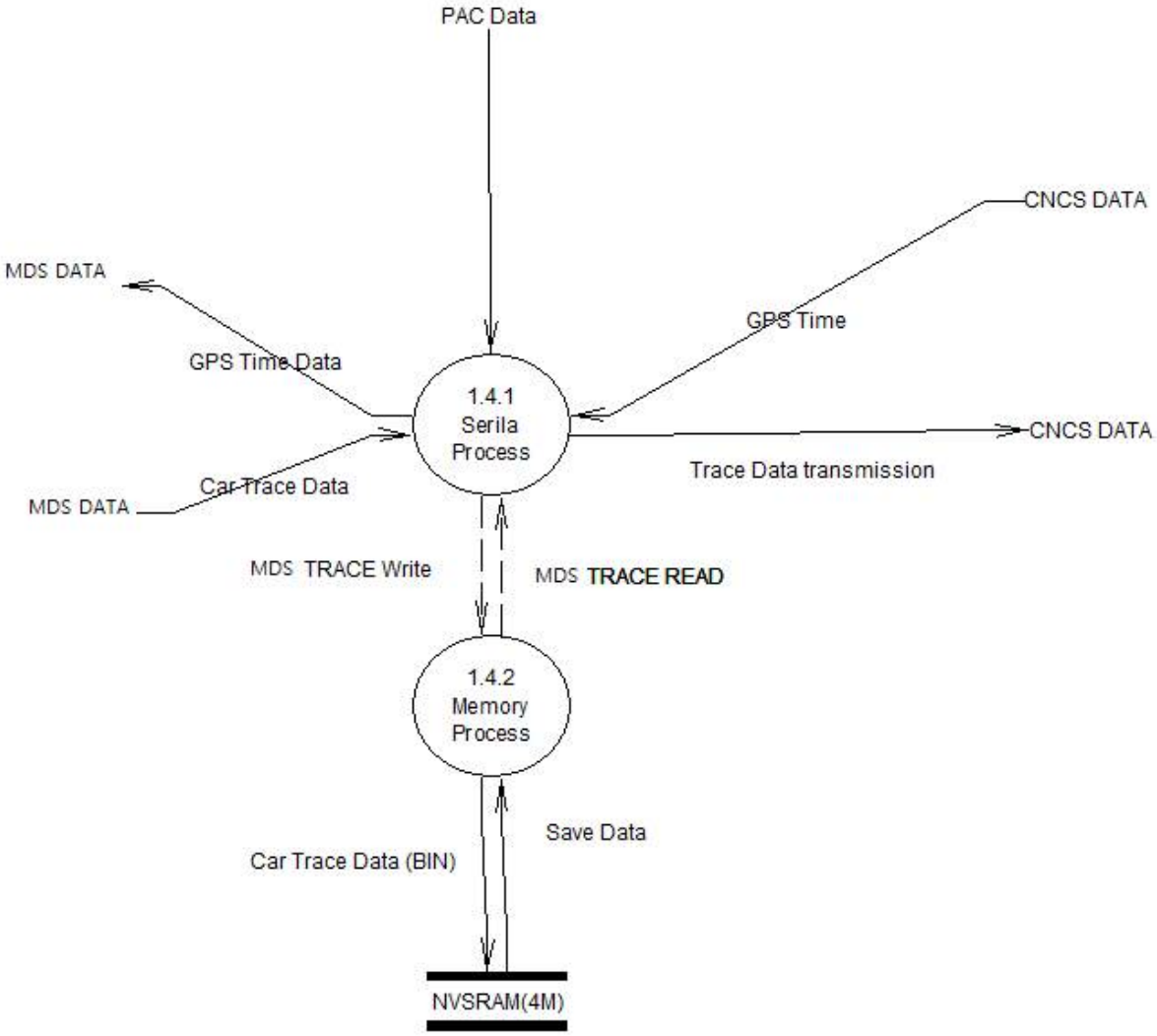


Figure 102. DFD level-5.

4.5.1.6.1. Serial Process.F

Identifier	Serial Process.F		
Purpose	CNCS, MDS and PAC Serial communication process.		
Source	MDSDATA	Data	It processes the failure data as LonWorks.
	PACDATA		It monitors the device status by using RS-485 communication.

	CNCSDATA		It receives GPS data by using RS-232 communication.
	Memory Process.		Memory Read Data.
Target	MDSDATA	Data	It transmits GPS time and the decive status to LonWorks communication.
	CNCSDATA		It transmits the failure data to CNCS by using RS-232 communication.
	Memory Process.		It encourages MDS failure data to the memory.

4.5.1.6.2. Memory Process.F

Identifier	Memory Process.F		
Purpose	It both reads and writes MDS reception data to the memory.		
Source	Serial Process	Data	It reads the data stored.
Target	Serial Process	Data	It writes MDS reception data.

5. Interface Description

5.1. PAC Interface

5.1.1. External Interface

5.1.1.1. Relative Input Interface

Signals are connected to AAB or Back Plane via hard-wired connection and can be read as Boolean data.

PAC_ID signal	
PAC type is CRB	0
PAC type is CRA	1

CAB_A signal	
CAB of CRA is activated	0
CAB of CRA is not activated	1

CAB_B signal	
CAB of CRB is activated	0
CAB of CRB is not activated	1

CAB signal	
CAB is activated	0
CRA is not activated	1

Uncouple_A signal	
A direction of Train is connected	0
A direction of Train is not connected	1

Uncouple_B signal	
B direction of Train is connected	0
B direction of Train is not connected	1

Reverse signal	
Reverse is activated	0
Reverse is not activated	1

Forward signal	
Forward is activated	0
Forward is not activated	1

5.1.1.2. Door Input Interface

Door LL signal	
Door Left Low is opened	0
Door Left Low is closed	1

Door LH signal	
Door Left High is opened	0
Door Left High is closed	1

Door RL signal	
Door Right Low is opened	0
Door Right Low is closed	1

Door RH signal	
Door Right High is opened	0
Door Right High is closed	1

5.1.1.3. Intercom Address Input Interface

Signals are connected to AAB or Back Plane via hard-wired connection and can be read as Boolean data.

DPO1 Address signal	
DPO1 is activated	0
DPO1 is not activated	1

DPO2 Address signal	
DPO2 is activated	0
DPO2 is not activated	1

PEI1 Address signal	
PEI1 is activated	0
PEI1 is not activated	1

PEI2 Address signal	
PEI2 is activated	0
PEI2 is not activated	1

PEI3 Address signal	
PEI3 is activated	0
PEI3 is not activated	1

PEI4 Address signal	
PEI4 is activated	0

PEI4 is not activated	1

PEI5 Address signal	
PEI5 is activated	0
PEI5 is not activated	1

PEI6 Address signal	
PEI6 is activated	0
PEI6 is not activated	1

PEI7 Address signal	
PEI7 is activated	0
PEI7 is not activated	1

PEI8 Address signal	
PEI8 is activated	0
PEI8 is not activated	1

5.1.1.4. Intercom Live Input interface.

Signals are connected to AAB or Back Plane via hard-wired connection and can be read as Boolean data.

DPO1 Live signal	
DPO1 is alive	0
DPO1 is dead	1

DPO2 Live signal	
DPO2 is alive	0
DPO2 is dead	1

PEI1 Live signal	
PEI1 is alive	0
PEI1 is dead	1

PEI2 Live signal	
PEI2 is alive	0
PEI2 is dead	1

PEI3 Live signal	
PEI3 is alive	0
PEI3 is dead	1

PEI4 Live signal	
PEI4 is alive	0
PEI4 is dead	1

PEI5 Live signal	
PEI5 is alive	0
PEI5 is dead	1

PEI6 Live signal	
PEI6 is alive	0
PEI6 is dead	1

PEI7 Live signal	
PEI7 is alive	0
PEI7 is dead	1

PEI8 Live signal	
PEI8 is alive	0
PEI8 is dead	1

5.1.1.5. Broadcast Input Interface

Signals are connected to AAB or Back Plane via hard-wired connection and can be read as Boolean data.

Old PA1 signal	
Any train is not in broadcast	0
Some train is in broadcast	1

Old PA2 signal	
Any train is not in broadcast	0
Some train is in broadcast	1

5.1.1.6. Broadcast Input Interface

5.1.1.7. Signals are connected to AAB or PAC Volume Card via hard-wired connection and can be read as Boolean data.

Select Intercom signal	
Intercom audio source is selected	0
Intercom audio source is not selected	1

Select MP3 signal	
MP3 or PEI audio source are selected	0
MP3 or PEI audio source are not selected	1

Select Exterior Right Speaker signal	
ESP Right speaker is muted	0
ESP Right speaker is activated	1

Select Exterior Left Speaker signal	
ESP Left speaker is muted	0
ESP Left speaker is activated	1

Select Interior Speaker signal	
ISP speaker is muted	0
ISP speaker is activated	1

5.2. CCP Interface

5.2.1. External Interface

5.2.1.1. Cab Signal Interface

Signal for determining status of plugged in Cab-Key on onboard CCP on the vehicle.

Status	Input Boolean data
Cab-Key plugged.	1
No Cab-Key plugged.	0

5.2.1.2. Conductor signal interface

Signal for determining status of plugged in Conductor- Key on onboard CCP on the vehicle.

Status	Input Boolean data
Conductor –Key plugged.	1
No Conductor -Key plugged.	0

5.2.1.3. Address signal interface

Signal determining train location.

Status	Input Boolean data
Address A Input On and B Input On	1
Address A Input On and B Input Off	1
Address A Input Off and B Input On	0

5.3. CNCS Interface

5.3.1. External Interface

5.3.1.1. Launcher

None

5.3.1.2. ContentUpdate

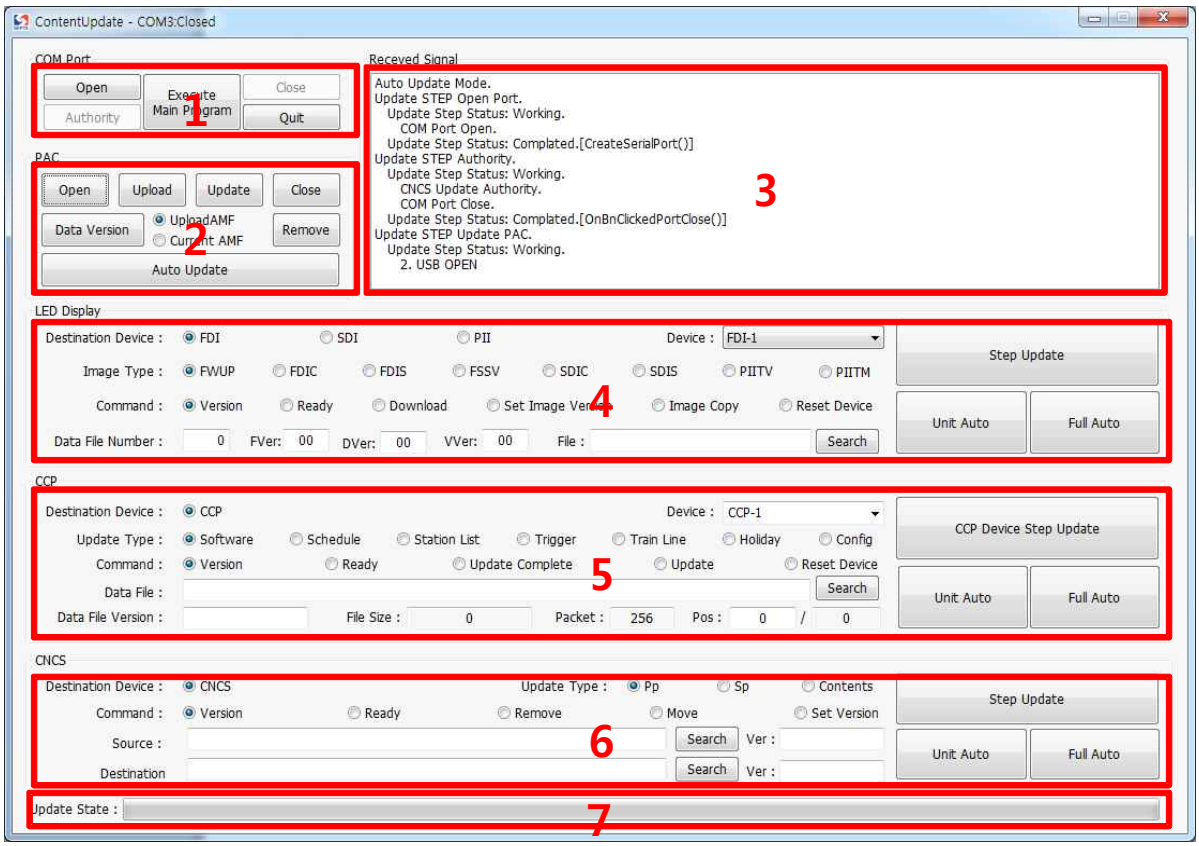


Figure 145 Content Update Process Screen

1. Bringsor lift the communication authority from PAC
2. Control PAC's updates.
3. Display the communication status of Car Line.
4. Update contents data of LED PID system.
5. Update CCP contents data.
6. Update CNCS contents data
7. Show the progress of the status of each update.

5.3.1.3. Main Process

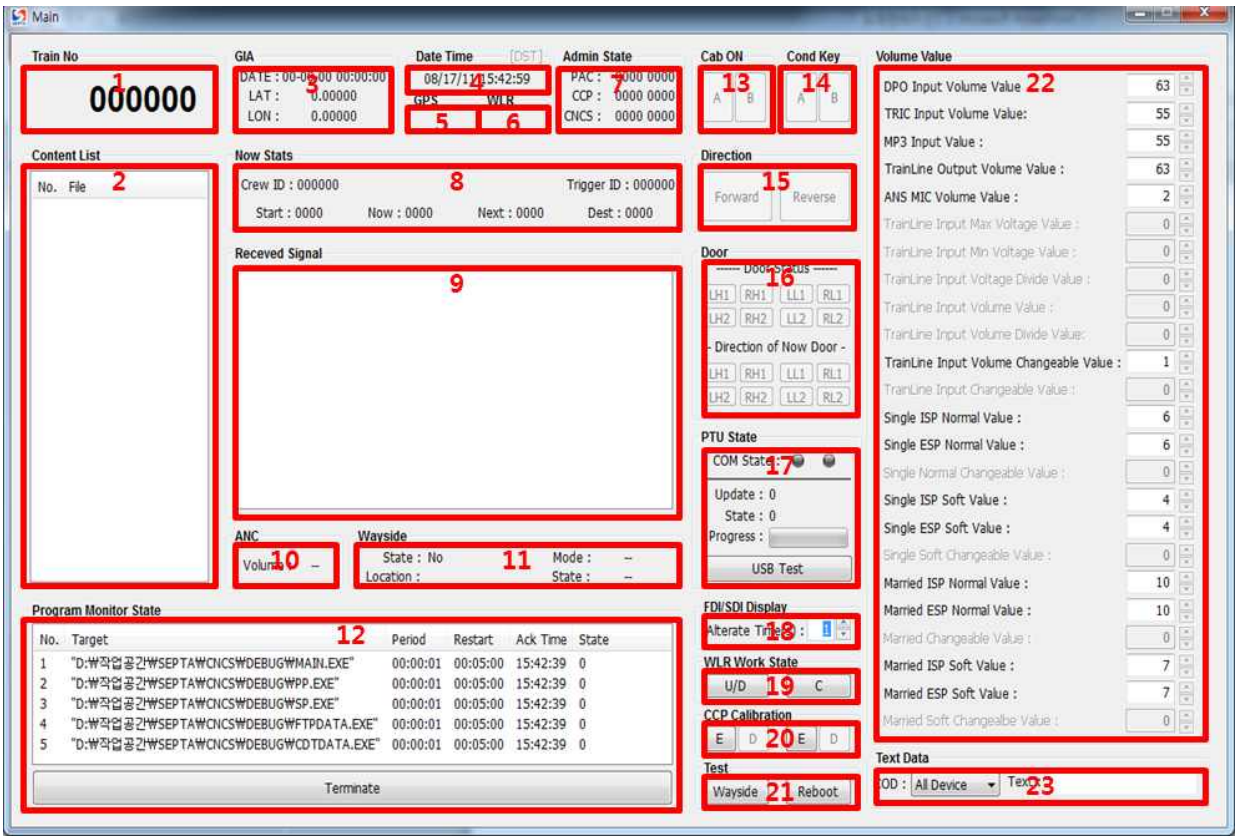


Figure 146 Main Process Screen

1. Train Number input in CCP.
2. Video lists registered in PP schedule.
3. Time and coordinate information received by GPS.
4. CNCS time information.
5. Status that displays whether the GPS receives the data.
6. Status display of WLR.
7. Display the communication leadership of Car Line.
8. Crew ID and Trigger of CCP, and station information display.
9. Car Line communication status display.
10. PAC ANC Mic. Level display.
11. Wayside work status display.
12. Sub Process Monitoring status display.
13. Broadcasting equipment Cab Key status display.
14. Broadcasting crew Key status display.
15. Direction of Train display
16. Door status display.

17. PTU communication status display.
18. Cotrol the Alternate conversion time of FDI/SDI.
19. Display the work status of Wayside.
20. Control CCP Calibration.
21. Section that tests Wayside work and CNCS Reboot function.
22. The section that controls the volume of PA system.

5.3.1.4. MDSData

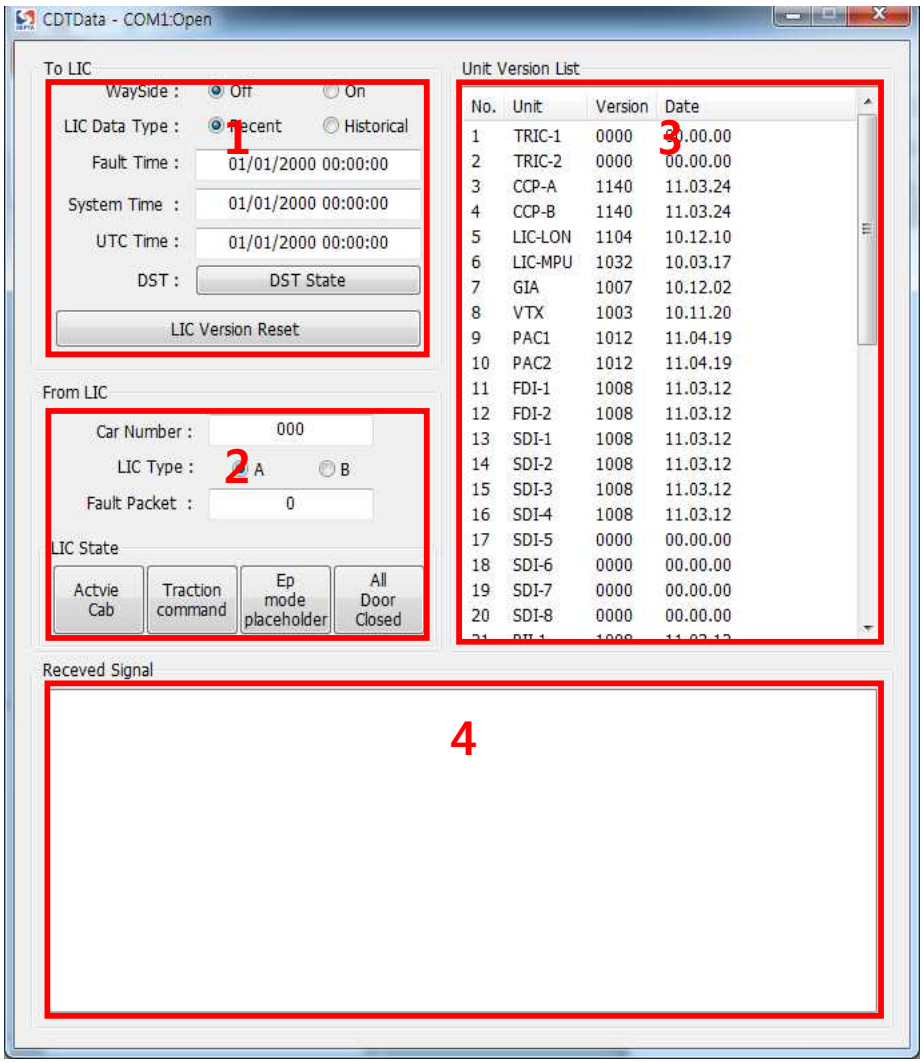


Figure 147 MDS Data Process Screen

1. Data status to be transmitted to LIC.
2. Data status received from LIC.
3. Display the version information of PA system.
4. Display 232 communication status of LIC and CNCS.

5.3.1.5. FTPData

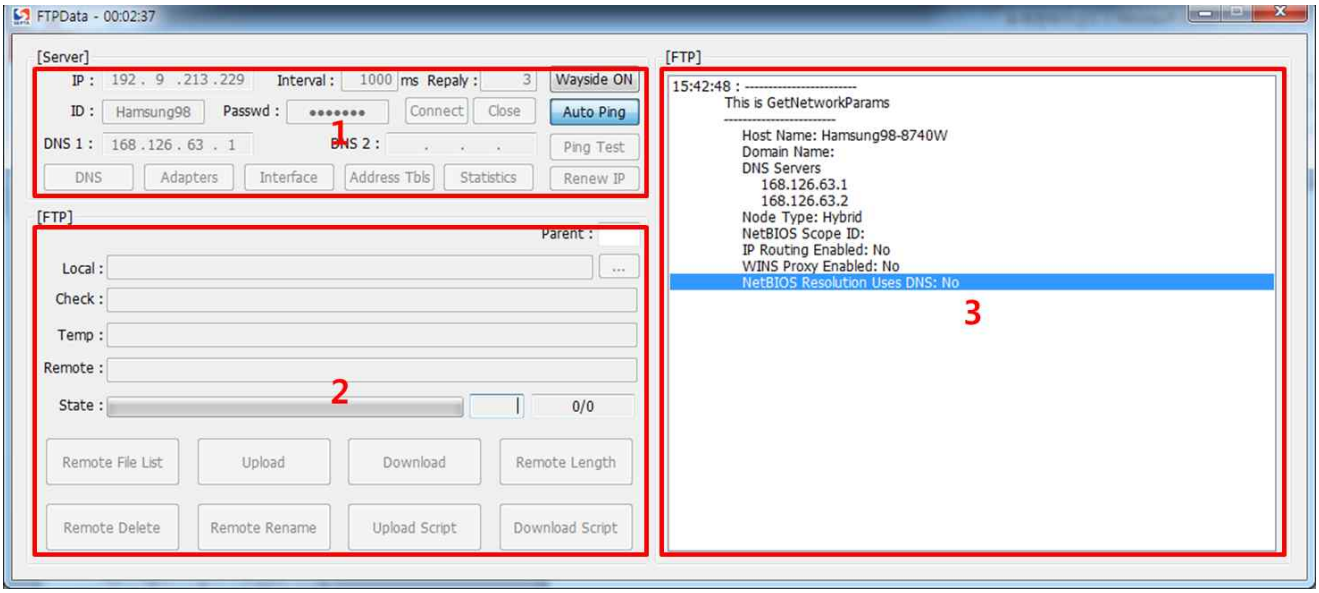


Figure 147 FTP Data Process Screen

1. Set up the access status with Wayside Server.
2. Control FTP work when connecting Wayside.
3. Display the work status with Wayside.

5.3.1.6. PP

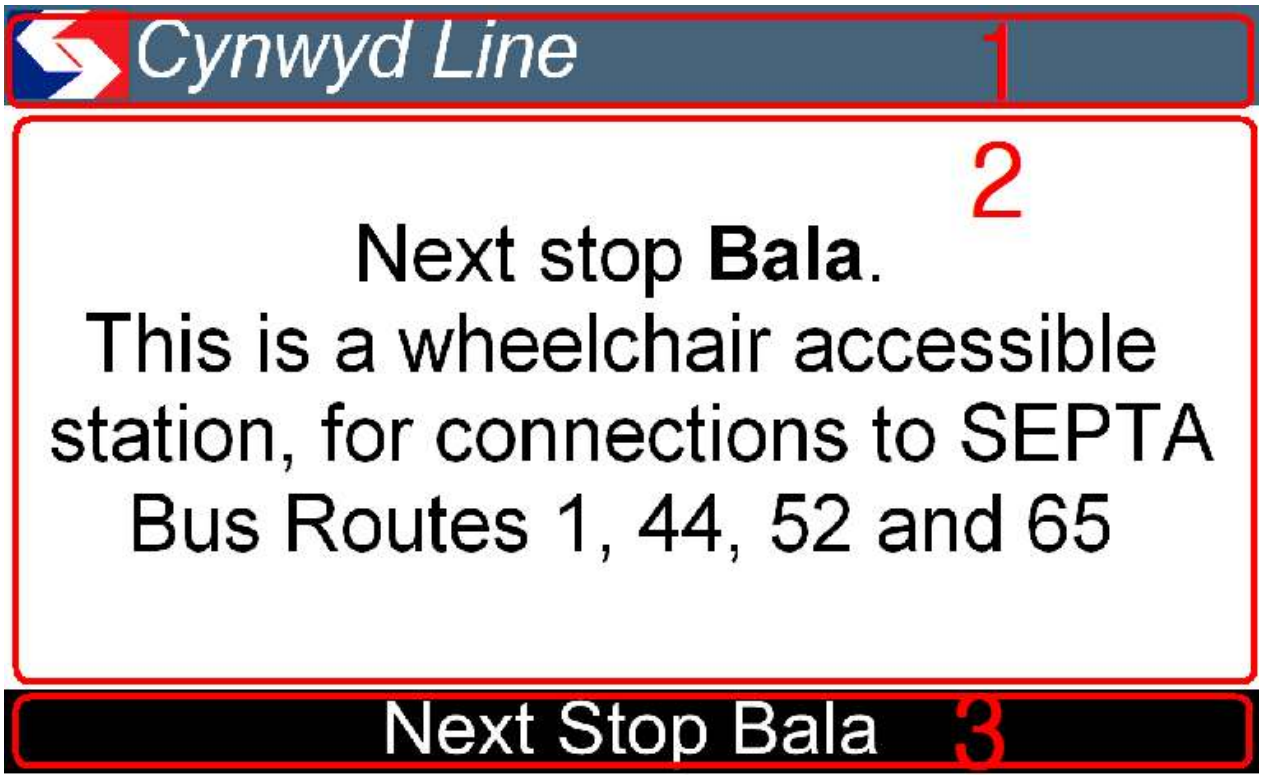


Figure 148 PP Process Screen

1. Display the exisiting routes.
2. Display the guide information for each TP.

5.3.1.7. SP

Display the guide information for each TP(Upscroll or leftscroll on the lower part of the screen)

5.4. LEDC Interface

5.4.1. External Interface

External interface on LEDC unit is divided into PAC communication interface between PAC unit and RS485, CNCS communication interface between CNCS unit and RS485, and LDM interface to send out the data to LDM unit.

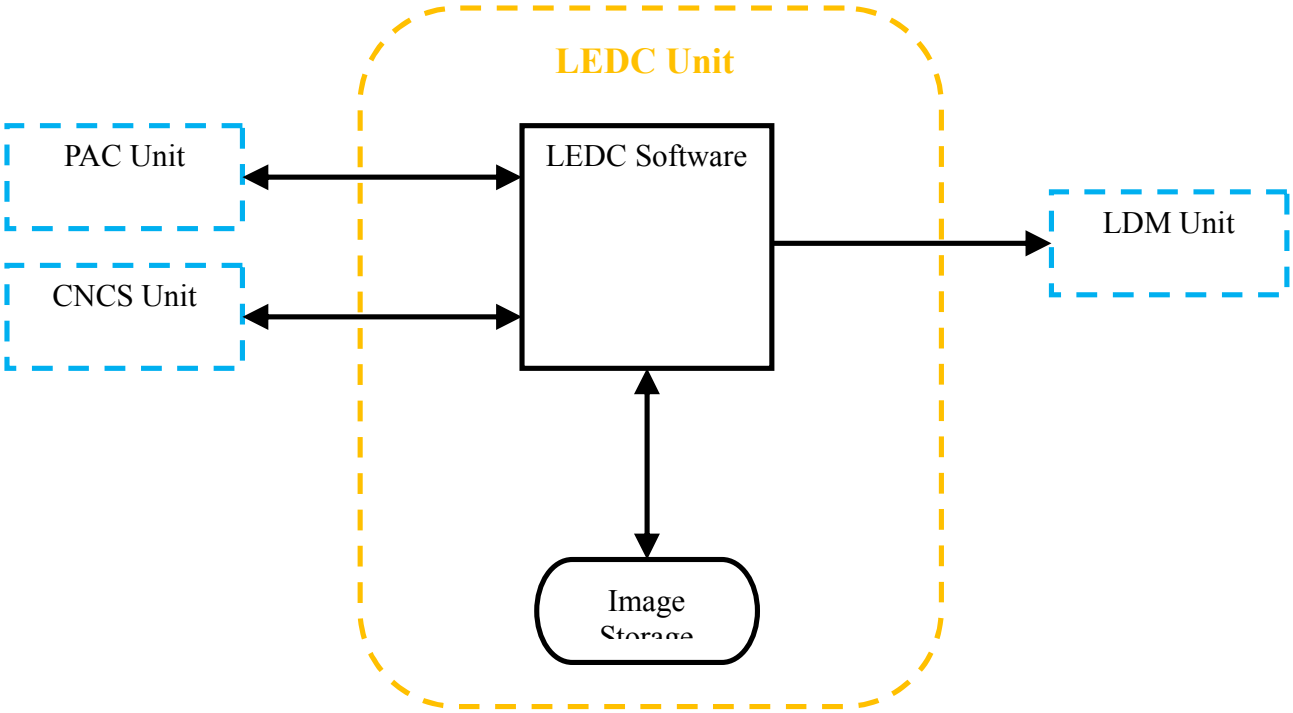


Figure 103. LEDC Interface

5.4.1.1. External PAC Communication Interface

PAC communication interface follows the specification of RS485 communication protocol.

5.4.1.2. External CNCS Communication Interface

CNCS communication interface follows the specification of RS485 communication protocol.

5.4.1.3. External LDM Interface

External LDM interface follows the specification of SRAM Memory BUS interface.

Data	Address
Source Data start address [fADD]	0xf302
Source Data Range [fRANGE]	0xf314
Source Yellow Data [fDATA_WY]	0xf304
Source Red Data [fDATA_WR]	0xf505
Source Green Data [fDATA_WG]	0xf507
Source Blue Data [fDATA_WB]	0xf509
Bright Control Value [fBRIGHT]	0xf31c

5.4.2. Internal Interface

5.4.2.1. Internal Image Storage Interface

Internal Image Storage interface is connected to SPI Memory Interface and saves FAI Image Data, SDI Image Data and PII Image Data.

Data	Address
FDI Station Image [Application]	0x00000000
FDI Station Image [Update]	0x00080000
FDI Color Image [Application]	0x00220000
FDI Color Image [Update]	0x00228000
SDI Station Image [Application]	0x00100000
SDI Station Image [Update]	0x00140000
SDI Color Image [Application]	0x00230000
SDI Color Image [Update]	0x00238000
PII Message [Application]	0x00190000
PII Message [Update]	0x001d0000

5.5. LIC Interface

5.5.1. External Interface

5.5.1.1. PACDATA

None

5.5.1.2. MDSDATA

None

5.5.1.3. CNCSDATA

None

5.5.1.4. MAIN

None

6. Detailed Description

6.1. PAC Description

6.1.1. Module Detailed Design

6.1.1.1. Main Process

ID	PAC.Main.F

Processing Logic	SET default value to Global variables
	CHECK live all devices
	DO WHILE Infinitely
	  PERFORM InOut Process
	  PERFORM CarLine Polling Process
	  PERFORM TrainLine Polling Process
	END DO

6.1.1.2. InOut Process

ID	PAC.InOut.F
Processing Logic	PERFORM PAC Relative Input PERFORM Door Input PERFORM Married Intercom Input PERFORM Broadcast Process

6.1.1.3. PAC Relative Input

ID	PAC.RelativeInput.F

Processing Logic	RESET PAC_ID
	RESET CAB
	IF CAB_On = true and Coupling_ID = 0 THEN
	  PERFORM Inauguration
	ENDIF
	RESET Conductor
	RESET Forward
	RESET Reverse
	RESET Uncouple

	IF Uncouple change = true THEN
	  PERFORM Inauguration
	ENDIF

6.1.1.4. Door Input

ID	PAC.DoorInput.F
Processing Logic	RESET Door_RH (right high) RESET Door_RL (right low) RESET Door_LH (left high) RESET Door_LL (left low)

6.1.1.5. Single Intercom Input

ID	PAC.SingleIntercomInput.F
Processing Logic	RESET live and address DPO1, DPO2 RESET live and address PEI1, PEI2, PEI3, PEI4, PEI5, PEI6, PEI7, PEI8

6.1.1.6. Broadcast Process

ID	PAC.Broadcast.F

Processing Logic	RESET Intercom Tone Signal
	READ Ans Mic Value
	
	IF Broad_State = Intercom THEN
	  DO Broadcast Intercom to Internal or External Speaker
	ELSE IF Broad_State = PEI THEN
	  DO PAC Broadcast MP3 of PEI to Internal Speaker
	ELSE IF Broad_State = Auto MP3 THEN
	  DO PAC Broadcast MP3 to Internal or External Speaker
	ELSE IF Broad_State = Old_Car_PA1 THEN
	  DO Broadcast source change and connect Internal Speaker
	ELSE IF Broad_State = Old_Car_PA2 THEN
	  DO Broadcast source change and connect Internal Speaker
	END IF

6.1.1.7. CarLine Polling Process

ID	PAC.CarLinePolling.F

Processing Logic	IF PAC_Main = true THEN
	  IF CarLine_Polling_Timing = 0 THEN
	  RESET CarLine_Polling_Timing
	PERFORM CarLine Transmit
	  END IF
	END IF
	
	IF CarLine_Packet_Receive_OK = true THEN
	  PERFORM CarLine Receive
	END IF

6.1.1.8. CarLine Receive

ID	PAC.CarLineReceive.F

Processing Logic	SELECT CASE
	  CASE LIC
	  DO save LIC variables
	  CASE PAC
	  DO save PAC variables
	  CASE CCP
	  DO save CCP variables
	  CASE CNCS
	  DO save CNCS variables
	  CASE, PII, FDI, SDI
	  DO save PIS variables
	END CASE

6.1.1.9. CarLine Transmit

ID	PAC.CarLineTransmit.F
Processing Logic	READ Target Device ID PERFORM Make CarLine Packet

6.1.1.10. Make CarLine Packet

ID	PAC.MakeCarLinePacket.F

Processing Logic	READ Target Device ID
	
	SELECT CASE
	  CASE LIC
	  DO make LIC Packet
	  CASE PAC
	  DO make PAC Packet
	  CASE CCP
	  DO make CCP Packet
	  CASE CNCS
	  DO make CNCS Packet
	  CASE, PII, FDI, SDI
	  DO make PIS Packet
	END CASE

6.1.1.11. TrainLine Polling Process

ID	PAC.TrainLinePolling.F

Processing Logic	IF Coupling_ID <> 0 THEN
	IF PAC_Master = true THEN
	IF TrainLine_Polling_Timing = 0 THEN
	  RESET TrainLine_Polling_Timing
	PERFORM TrainLine Transmit
	  END IF
	END IF
	END IF

	IF TrainLine_Packet_Receive_OK = true THEN
	  PERFORM TrainLine Receive
	END IF

6.1.1.12. Inauguration

ID	PAC.Inauguration.F
Processing Logic	RESET Coupling variables RESET Coupling Car Numbers DO READ Train Car Number READ Train Devices Error Status UNTIL GET Coupling_ID GET Train Audio Valance Value APPLY Train Audio Valance Value to all trains

6.1.1.13. TrainLine Recevie

ID	PAC.TrainLineReceive.F
Processing Logic	READ Train_ID SAVE Train Error Status

6.1.1.14. TrainLine Transmit

ID	PAC.TrainLineTransmit.F
Processing Logic	READ Train_ID PERFORM Make TrainLine Packet

6.1.1.15. Make TrainLine Packet

ID	PAC.MakeTrainLinePacket.F
Processing Logic	READ Train_ID READ Train Coupling_ID DO make TrainLine Packet

6.1.1.16. Interrupt Handler

ID	PAC.InterruptHandler.F

Processing Logic	GET Interrupt Event
	
	SELECT CASE
	  CASE get CarLine 485 Rx Event
	  PERFORM CarLine Rx Packet
	  CASE get TrainLine 485 Rx Event
	  PERFORM TrainLine Rx Packet
	  CASE exist value at CarLine 485 Tx buffer
	  PERFORM CarLine Tx Packet
	  CASE exist value at TrainLine 485 Tx buffer

	PERFORM TrainLine Tx Packet CASE get Timer Event PERFORM Timer Process END CASE

6.1.1.17. CarLine Rx Packet

ID	PAC.CarLineRxPacket.F
Processing Logic	READ CarLine Rx Buffer Index SAVE Recevie Data to CarLine Rx Storage

6.1.1.18. TrainLine Rx Packet

ID	PAC.TrainLineRxPacket.F
Processing Logic	READ TrainLine Rx Buffer Index SAVE Recevie Data to TrainLine Rx Storage

6.1.1.19. CarLine Tx Packet

ID	PAC.CarLineTxPacket.F
Processing Logic	READ CarLine Packet Index SAVE Read Data to CarLine 485 Tx Buffer

6.1.1.20. TrainLine Tx Packet

ID	PAC.TrainLineTxPacket.F
Processing Logic	READ TrainLine Packet Index SAVE Read Data to TrainLine 485 Tx Buffer

6.1.1.21. Timer Process

ID	PAC.Timer.F
Processing Logic	GET Timer Interrupt Event SELECT CASE CASE get CarLine Timer Event PERFORM CarLine Timer CASE get TrainLine Timer Event PERFORM TrainLine Timer END CASE

6.1.1.22. CarLine Timer

ID	PAC.CarLineTimer.F
Processing Logic	DECREASE CarLine_Polling_Timing IF not respond Devices THEN SELECT CASE CASE LIC DO save LIC error

	CASE PAC DO save PAC error CASE CCP DO save CCP error CASE CNCS DO save CNCS error CASE PII DO save PII error CASE FDI DO save FDI error CASE SDI DO save SDI error END CASE END IF

6.1.1.23. TrainLine Timer

ID	PAC.TrainLineTimer.F

Processing Logic	DECREASE CarLine_Polling_Timing
	
	IF not respond TrainLine Coupling Car THEN
	DO SAVE Train Error Status
	END IF

6.1.2. Data Detailed Design

6.1.2.1. Port

Struct/File ID	PAC.PortInOut.D	
process	write	InOut Port Process
	read	InOut Port Process
Filed name	data type	description
PORT_State	Byte	-Read signal of CAB, Conductor, Forward, Reverse, Uncouple. - Read Door signal. - Read Car DPO1, DPO2, PEI1, PEI2, PEI3, PEI4, PEI5, PEI6, PEI7, PEI8 Live signal . Address signal.
Chattering	Byte	Port signal Chattering process variable.

6.1.2.2. Device

Struct/File ID	PAC.DeviceData.D	
process	write	InOut Port Process
		CarLine Polling Process
		TrainLine Polling Process
		Timer Process
	read	InOut Port Process
		CarLine Polling Process
		TrainLine Polling Process
		Timer Process
Filed name	data type	description

PAC_Token	Byte	PAC Master Sub Token
CCP_Token	Byte	CCP Master Sub Token
CNCS Token	Byte	CNCS Master Sub Token
Cont_PAC_CCP	Byte	Reverse, Forward, Uncouple, Simulation, CAB Control Signal Variable.
Cont_PAC_Route	Byte	Conductor, Router Number
T_Train_Num	Byte	Train Number
T_Crue_ID	Byte	Crew ID
T_Door_State	Byte	LH, LL, RH, RL signal
T_Trig_ID	Byte	Trigger Point Number
T_Car_Num	Byte	Car Number
Spk_State	Byte	Status of Speaker
T_Status_Comm	Byte	Multi-coupled all train device Failure information.
Start_Stt	Byte	Train number origin station
Now_Stt	Byte	Train number current station
Next_Stt	Byte	Train number next station
Dest_Stt	Byte	Train number last station
Broad_State	Byte	Announcement Source Status
T_Vol_Val	Byte	Announcement Volume Table
TL_Car_Num	Byte	Multi-coupled train number sequence
Err_Device	Byte	Device Error Counter
Err_Cars	Byte	Multi-coupled train error counter
T_Firm_Ver	Byte	Firmware Version Information.
T_Date	Byte	Firmware Build date information.

6.1.2.3. CarLine Rx

Struct/File ID	PAC.CarLineRx.D	
process	write	CarLine Receive Transmit Process
	read	CarLine Polling Process
Filed name	data type	description
CL_Rx_SrcID	Byte	CarLine receiving packet source device ID
CL_Rx_DstID	Byte	CarLine receiving packet destination ID
CL_Rx_MCnt	Byte	CarLine receiving packet message counter
CL_Rx_Cmd	Byte	CarLine receiving packet Command
CL_Rx_Len	Byte	CarLine receiving packet Data Length
CL_Rx_Data	Byte	CarLine receiving packet data
CL_Rx_ChkSum	Byte	CarLine receiving packet checksum
CL_Rx_Index	Byte	CarLine receiving packet location index
CL_OK_Packet	Bool	Confirm CarLine completed packet reception.

6.1.2.4. CarLine Tx

Struct/File ID	PAC.CarLineTx.D	
process	write	CarLine Polling Process
	read	CarLine Receive Transmit Process
Filed name	data type	description
CL_Tx_SrcID	Byte	CarLine transmit packet source device ID
CL_Tx_DstID	Byte	CarLine transmit packet destination ID

CL_Tx_MCnt	Byte	CarLine transmit packet message counter
CL_Tx_Cmd	Byte	CarLine transmit packet Command
CL_Tx_Len	Byte	CarLine transmit packet Data Length
CL_Tx_Data	Byte	CarLine transmit packet data
CL_Tx_ChkSum	Byte	CarLine transmit packet checksum
CL_Tx_Index	Byte	CarLine transmit packet location index

6.1.2.5. TrainLine Rx

Struct/File ID	PAC.TrainLineRx.D	
process	write	TrainLine Receive Transmit Process
	read	TrainLine Polling Process
Filed name	data type	description
TL_Rx_SrcID	Byte	TrainLine receiving packet source device ID
TL_Rx_DstID	Byte	TrainLine receiving packet destination ID
TL_Rx_MCnt	Byte	TrainLine receiving packet message counter
TL_Rx_Cmd	Byte	TrainLine receiving packet Command
TL_Rx_Len	Byte	TrainLine receiving packet Data Length
TL_Rx_Data	Byte	TrainLine receiving packet data
TL_Rx_ChkSum	Byte	TrainLine receiving packet checksum
TL_Rx_Index	Byte	TrainLine receiving packet location index
TL_OK_Packet	bool	Confirm TrainLine completed packet reception.

6.1.2.6. TrainLine Tx

Struct/File ID	TrainLineTx.D	
process	write	TrainLine Polling Process
	read	TrainLine Receive Transmit Process
Filed name	data type	description
TL_Tx_SrcID	Byte	TrainLine transmit packet source device ID
TL_Tx_DstID	Byte	TrainLine transmit packet destination ID
TL_Tx_MCnt	Byte	TrainLine transmit packet message counter
TL_Tx_Cmd	Byte	TrainLine transmit packet Command
TL_Tx_Len	Byte	TrainLine transmit packet Data Length
TL_Tx_Data	Byte	TrainLine transmit packet data
TL_Tx_ChkSum	Byte	TrainLine transmit packet checksum
TL_Tx_Index	Byte	TrainLine transmit packet location index

6.2. CCP Description

6.2.1. Module Detailed Design

6.2.1.1. Main Process

Identifier	CCP.MainProcess.F
Processing Logic	

```
READ Resource Data from SD Storage
PERFORM Sub process
DO WHILE Infinitely
    PERFORM Main process
ENDO
```

6.2.1.2. LoadUI

Identifier	CCP.LoadUI.F
Processing Logic	
LOAD Image resource GENERATE UI Control GENERATE Graphics Buffer	

6.2.1.3. Initail Sub Process

Identifier	CCP.InitailSubProcess.F
Processing Logic	
PERFORM Timer Event Handler PERFORM Serial Process	

6.2.1.4. Event Handler

Identifier	CCP.EventHandler.F
Processing Logic	
GET event PROESS event	

6.2.1.5. Input Read

Identifier	CCP.InputRead.F
Processing Logic	
GET Screen touch Event UPDATE UI Control	

6.2.1.6. Timer Event

Identifier	CCP.TimerEvent.F
Processing Logic	
CATCH Timer Event PERFORM UpdateSchedule	

6.2.1.7. Update Schedule

Identifier	CCP.UpdateSchedule.F
Processing Logic	
UPDATE Time Clock UPDATE Departure Staiton UPDATE Actual Station UPDATE Next Station UPDATE Delay time UPDATE Current Staiton UPDATE Train Line UPDATE Comment	

6.2.1.8. Route Process

Identifier	CCP.Route.Process.F

Processing Logic

READ Train Number
IF Train Number is Valid THEN
  GENERATE Route Data for Train Number
END IF
READ GIA_DATA_DATA Parameter
IF Route Data has GIA_DATA_DATA Parameter THEN
  GENERATE Data for PA
END IF

6.2.1.9. Route Generate

Identifier	CCP.RouteGenerate.F

Processing Logic

READ Train Number Parameter
SEARCH Train Number from Schedule DB
IF Schedule DB has Train Number THEN
  GENERATE Route data
  UPDATE Schedule Information
  READY to Receive GIA_DATA_DATA Parameter
END IF

6.2.1.10. Route Operate

Identifier	CCP.RouteOperate.F

Processing Logic

READ GIA_DATA_DATA Parameter
SEARCH Station near by GIA_DATA_DATA Parameter
IF Detect Station THEN
  GENERATE PA Data
END IF

6.2.1.11. Determine Current Station

Identifier	CCP.DetermineCurrentStation.F

Processing Logic
READ GIA_DATA_DATA Parameter CALCULATE LOT & LAT FROM GIA_DATA_DATA Parameter SEARCH Station by LOT & LAT form Route DB

6.2.1.12. Generate Data for PA

Identifier	CCP.GenerateDataForPA.F

Processing Logic
SET Actual Station Number Parameter SET Departure Station Number Parameter

```
SET Next Station Number Parameter
SET Train Line Number Parameter
SET Train Number Parameter
```

6.2.1.13. Serial Process

Identifier	CCP.SerialProcess.F

Processing Logic

```
READ Serial Packet
CALCULATE Check Sum
IF Valid Packet THEN
    PERFORM ParsetPacket
    IF Source ID == PAC THEN
        PERFORM Process PAC Data
    END IF
    IF Source ID == GIA_DATA_DATA THEN
        PERFORM Process GIA_DATA_DATA Data
    END IF
    IF Source ID == CNCS THEN
        PERFORM Process CNCS Data
    END IF
ELSE
    FLUSH Buffer
    READY TO Receive Packet
END IF
```

6.2.1.14. Parse Packet

Identifier	CCP.ParsePacket.F

Processing Logic

```
READ Serial Data
SET Source ID Parameter
SET Destination ID Parameter
IF Source ID == PAC THEN
    SET PAC Structure
END IF
IF Source ID == GIA_DATA_DATA THEN
    SET GIA_DATA_DATA Structure
END IF
IF Source ID == CNCS THEN
    SET CNCS Structure
END IF
```

6.2.1.15. Process PAC Data

Identifier	CCP.ProcessPACData.F

Processing Logic

```
IF Calibration Parameter is Set THEN
    PERFORM Touch Calibration
END IF
CALCULATE Car Count FORM CCI Parameter
```

```
UPDATE Car Parameter
IF Route is Set and Cab Signal Is Not Set THEN
    READ Simulation Parameter
    IF Simulation Parameter Is Set THEN
        PERFORM Simulation
    ELSE
        STOP Simulation
    END IF
END IF
SEND Ack Packet TO PAC

IF Train Number is Not Set THEN
    READ Train Number Parameter form PAC Data
    GENERATE Route DB
END IF
```

6.2.1.16. Process GIA_DATA_DATA Data

Identifier	CCP.ProcessGIA_DATA_DATAData.F
Processing Logic	

```
IF Route DB Is Set THEN
    PERFORM Route Operate Function
END IF
```

6.2.1.17. Process CNCS Data

Identifier	CCP.ProcessCNCSData.F
Processing Logic	

```
READ Ready Parameter
IF Ready Parameter Is Set THEN
    READY Update
END IF
READ Update Packet
WRITE Packet to Temp Storage
IF Complete to Update THEN
    REPLACE Temp File
    SYSTEM RESET
END IF
```

6.2.1.18. Transfer Packet

Identifier	CCP.TransferPacket.F
Processing Logic	

```
GENERATE Transfer Packet Data
SEND Data Through RS-485 Serial Line
```

6.2.1.19. Data Detailed Design

6.2.1.20. Route Data

Route Data has route information for automatic announcement. Route information consists combine information of schedule, station, trigger, train line, holiday and config informations.

Data field name	Field size	Data field description
CDBSchedule	-	Schedule Information DB
CDBStation	-	Station Information DB
CDBTrigger	-	Station coordinate DB
CDBTrainLine	-	Train Line DB
CDBHoliday	-	Holiday Information DB
CDBConfig	-	Configuration Information DB
ActualStationIdx	4	Current Station Index
ActualTriggerIdx	4	Current Station TP Index
TPNumber	4	TP Number
CrewID	4	Crew ID
TrainNumber	4	Operation Train Number

6.2.1.21. Schedule

Data field name	Field size	Data field description
RouteNo	4	Route Number
TrainNo	4	Train Number
StartStationNo	4	Origin Number
LastStationNo	4	Terminus Number
ArrivalStationNo	4	Arrival Station Number
strDepartureTime	-	Departure Time String
strArriveTime	-	Arrival Time String
UpDown	1	Up and Down Devision
Week	1	Week / Weekend
TrafficType	1	SKIP or STOP
TrafficSetType	1	TP announcement sequence
ArriveStationName	-	Arrival Station Name
SetArrive	1	Arrival Status
TimeArrive	-	Actual arrival time
DestinationNo	4	Destination Code Number
DestinationCode	-	Destination Code Name
LED	4	LED Generating information
TP	36	TP Generating information
SPK	36	SPK Volume information

6.2.1.22. Station

Data field name	Field size	Data field description
StationNo	4	Station Number
IsViaCenterCity	1	VCC
HaveDualName	1	Multi Station Name
Ascent	1	TP Increment / Decrement
StationName	-	Station Name

6.2.1.23. Trigger

Data field name	Field size	Data field description
TP_INDEX	4	TP Index
TP_LAT	8	LAT Coordinates Information
TP_LOT	8	LOT Coordinates Information
TP_ANNOUNCED	1	Whether it makes announcement or not.

6.2.1.24. TrainLine

Data field name	Field size	Data field description
TrainLineNo	4	Train Line code
UpDown	1	Up/ Down
TrainLineCode	-	Train Line Name

6.2.1.25. Holiday

Data field name	Field size	Data field description
YEAR	4	Year
MONTH	4	Month
DAY	4	Day
DAYOFWEEK	4	Day of the Week

WEEKOFMONTH	4	Week of the Month
WEEK	1	Applied Week
HolidayName	-	Holiday Name

6.2.1.26. Config

Data field name	Field size	Data field description
TrainNo	4	Train Number
strDisplay[0]	-	Station Name Display
strDisplay[1]	-	Time Display
strDisplay[2]	-	Lag Time Display

6.2.1.27. Serial Data

Data field name	Field size	Data field description
SourceID	4	Source ID
DestinationID	4	Destination ID
CommandCode	4	Command Code
DataLength	4	Data Length
PAC_DATA	-	PAC DATA Structure
CNCS_DATA	-	CNCS DATA Structure
GIA_DATA_DATA_DATA	-	GIA_DATA DATA Structure

6.2.1.28. PAC Data

Data field name	Field size	Data field description
PAC_T	1	PAC Status Flag
CCP_M	1	CCP Status Flag
CNCS_T	1	CNCS Status Flag
D0	1	CAB, Simulation, Couple, Forward, Reverse Status Flag

D1	1	CON, Route Number Information
TRAN	4	Operation Train Number
CID	4	Crew ID
DS	1	Door State
NOWD	1	Current Station Door State
NEXD	1	Next stop Station Door State
TRIP	6	TRIP Information
CARN	4	Train Number
D2	1	Indicator Display Flag
DTI	1	Delay Time
SPK	1	Speaker Volume Information
CCI	1	VCC Display Information
CROSS_TIME	1	Cross Display Information
SKIP	20	SKIP Station Information

6.2.1.29. CNCS Data

Data field name	Field size	Data field description
Target	1	Update Target
FIileSize	4	File Size
FrameCount	4	Frame Count

6.2.1.30. GIA_DATA Data

Data field name	Field size	Data field description
Date	6	Data time
LAT	8	LAT Coordinates
LOT	8	LOT Coordinates

6.2.1.31. System Status Data

Data field name	Field size	Description
IntSpeaker	1	Internal Speaker Volume
ExtSpeaker	1	External Speaker Volume
Brightness	4	Screen Brightness
CCPID	1	CCP ID
IsUpdating	1	Update Process
SetCabonKey	1	Cab Key Setting
SetConductorKey	1	Con Key Setting
Inuguration	1	Multi-coupled trainset process
SetPEI	1	PEI setting
Version	8	Version String
BuildDate	8	Build Date String
CAR_DATA	-	Train Information

6.3. CNCS Description

6.3.1. Module Detailed Design

6.3.1.1. Launcher

6.3.1.1.1. InitConfigurationLauncger Interface

Identifier	CLauncherDlg.InitConfigurationLauncher
Processing Logic	
LOAD Version FROM Register LOAD ExecuteProgram From Register	

6.3.1.1.2. InitConfigurationUpdateList Interface

Identifier	CLauncherDlg.InitConfigurationUpdateList
Processing Logic	

LOAD number of update FROM Register
FOR count FROM 0 TO number of update - 1
LOAD update target FROM Register
BEGIN IF
IF there is update target.
THEN analysis update target of CRC.
END IF
END FOR	

6.3.1.1.3. OnBnClickedUpdateCheck Interface

Identifier	CLauncherDlg.OnBnClickedUpdateCheck
Processing Logic	

FOR count FROM 0 TO number of update - 1
BEGIN IF
IF CRC is normal.
THEN bring update target of version information.
END IF
END FOR	

6.3.1.1.4. OnBnClickedUpdating Interface

Identifier	CLauncherDlg.OnBnClickedUpdating
Processing Logic	

FOR count FROM 0 TO number of update - 1
BEGIN IF
IF CRC is normal and version is changed.
THEN proceeds target program update.
END IF
END FOR	

6.3.1.1.5. OnBnClickedProgramRun Interface

Identifier	CLauncherDlg.
Processing Logic	
Execute ContentUpdate program.	

6.3.1.1.6. CheckValidCRC Interface

Identifier	CCRC.CheckValidCRC
Processing Logic	
Reads target file. Calculates CRC code and source. Compare to CRC and calculated CRC.	

6.3.1.1.7. GetDecoderCrcFile Interface

Identifier	CCRC.GetDecoderCrcFile
Processing Logic	
IF there is pre-created file Remove pre-created file Remove CRC and create update target file.	

6.3.1.1.8. GetCrcFile Interface

Identifier	CCRC.GetCrcFile
Processing Logic	
CRC returns removed file.	

6.3.1.2. Content Update

6.3.1.2.1. GetCurrentUpdateStep

Identifier	CAutoupdate.GetCurrentUpdateStep
Processing Logic	
Returns current update Step information.	

6.3.1.2.2. GetNextUpdateStep

Identifier	CAutoupdate.GetNextUpdateStep
Processing Logic	
Stores current UpdateStep on CurrnetStep. if Auto update is TRUE Set UpdateStep to Port Close. if update status is FAILED Set CurrentStep as CNCS_UPDATE. END if if CurrentStep is STEP_NONE, Set CurrentStep as Port Open.	

```
        else if CurrentStep is Port Open,
                Set CurrentStep as MDS Wait Timer.
        else if CurrentStep is MDS Wait Timer
                Set CurrentStep as Inauguration check.
        else if CurrentStep is Inauguration check,
                Set CurrentStep as Wayside Check.
        else if CurrentStep is Wayside Check,
                Set CurrentStep as Authority.

        else if CurrentStep is Authority,
                Set CurrentStep as PAC Update.
        else if CurrentStep is PAC Update,
                Set CurrentStep as LED Update.
        else if CurrentStep is LED Update,
                Set CurrentStep as CCP Update.
        else if CurrentStep is CCP Update,
                Set CurrentStep as CNCS Update.
        else if CurrentStep is CCP Update,
                Set CurrentStep as CNCS Update.
        else if CurrentStep is CNCS Update,
                Set CurrentStep as Port Close.
        else if CurrentStep is Port Close,
                Set CurrentStep as Update Complate.
        else if CurrentStep is Update Compleat,
                Set CurrentStep as Content Update Quit.
        END if
END if
```

6.3.1.2.3. GetUpdateState

Identifier	CAutoupdate.GetUpdateState
Processing Logic	
return UpdateState	

6.3.1.2.4. GetWorkingTimeOut

Identifier	CAutoupdate.GetWorkingTimeOut
Processing Logic	
If TimeOut timer is 0, return Ture.	

6.3.1.2.5. ResetTimeoutTimer

Identifier	CAutoupdate.ResetTimeoutTimer
Processing Logic	
Set current time in Timeout timer.	

6.3.1.2.6. OnSerialReceiveData

Identifier	CContentUpdateDlg.OnSerialReceiveData
Processing Logic	

```
IF DID is CNCS,
        IF SID is  PAC,
                IF authority request is underway,
                        Transmit CNCS authority request data.
                ELSE IF authority acquisition is in progress,
                        Transmit authority maintain data.
                ELSE
                        Transmit CNCS response data.
                END IF
        ELSE IF SID is LED,
                IF Led automatic update,
                        CALL OnLedFullAuto
                        CALL ResetTimeoutTimer
                END IF
        ELSE IF SID is CCP,
                IF CCP automatic update,
                        CALL OnCcpFullAuto
                        CALL ResetTimeoutTimer
                END IF
        END IF
        CALL CalcualteUpdatenextStep
ELSE IF DID is CP, .
        IF train is multi unit,
                        CALL ResetTimeoutTimer                    // Wait until train multi unit is
completed.
        END IF
ELSE IF SID is ACK of GIA_DATA
        IF Wayside is inspection,
                IF GIA_DATA time information is normal,
                        IF has data been receiving and completed from FTP server, normally?
                                IF Did the update target date has passed?
                                        Perform next update.
                                ELSE
                                        Close update task.
                                END IF
                        END IF
                END IF
        END IF
ELSE IF it's between PAC communication,
        Store train information.
END IF
```

6.3.1.2.7. nBnClickedCncsAuthority

Identifier	CContentUpdateDlg.OnBnClickedCncsAuthority
Processing Logic	

```
IF it's request to CNCS,
        Initialization automatic update information.
END IF
```

6.3.1.2.8. OnBnClickedPacUsbAutoUpdate

Identifier	CContentUpdateDlg.OnBnClickedPacUsbAutoUpdate
Processing Logic	
Set PAC automatic update. Initialization PAC automatic update. IF automatic update, CALL CalculatePacUsbAmfNextStep END IF	

6.3.1.2.9. CalculatePacUsbAmfNextStep

Identifier	CContentUpdateDlg.CalculatePacUsbAmfNextStep
Processing Logic	
In case of automatic updates. Count PAC updates with the step below. SELECT CASE If next step is USB OPEN. CALL OnBnClickedPacUsbOpen If next step is WAIT CONNET CALL OnDeviceChange If next step is DATA VERSION CHECK CALL OnBnClickedPacUsbDataVersion If next step is REMOVE UPDATE DATA CALL OnBnClickedPacUsbRemove . If next step is UPLOADDING CALL OnBnClickedPacDataUpload If next step is REMOVE CURRENT DATA CALL OnBnClickedPacUsbRemove If next step is DATA UPDATE CALL OnBnClickedPacUsbUpdate If next step is USB CLOSE CALL OnBnClickedPacUsbClose If next step is COMPLEATED Shut down PAC automatic updates. END CASE END IF	

6.3.1.2.10. OnBnClickedPacUsbOpen

Identifier	CContentUpdateDlg.OnBnClickedPacUsbOpen
Processing Logic	

6.3.1.2.11. OnDeviceChange

Identifier	CContentUpdateDlg.OnDeviceChange
Processing Logic	
CALL **GetDriveList**	

6.3.1.2.12. OnBnClickedPacUsbDataVersion

Identifier	CContentUpdateDlg.OnBnClickedPacUsbDataVersion
Processing Logic	

Initialize the updates content data information.
 	
If Content Data is existent.
Store the version information of the object information and the new data.
If the versions between the updates folder and the new data are the same.
Set up the next work as PAC's update data.
ELSE
Delete folder's content of PAC for the next work.
END IF
END IF	

6.3.1.2.13. OnBnClickedPacUsbRemove

Identifier	CContentUpdateDlg.OnBnClickedPacUsbRemove
Processing Logic	
Remove PAC update folder content.	

6.3.1.2.14. OnBnClickedPacDataUpload

Identifier	CContentUpdateDlg.OnBnClickedPacDataUpload
Processing Logic	
Copy CNCS Update folder content to PAC Update folder.	

6.3.1.2.15. OnBnClickedPacUsbUpdate

Identifier	CContentUpdateDlg.OnBnClickedPacUsbUpdate
Processing Logic	
Copy Pac Update folder content to PAC operational data.	

6.3.1.2.16. OnBnClickedPacUsbClose

Identifier	CContentUpdateDlg.OnBnClickedPacUsbClose
Processing Logic	
Disconnect USB connection between CNCs and PAC.	

6.3.1.2.17. OnLedDeviceUpdateIntialFullAutoUpdate

Identifier	CContentUpdateDlg.OnLedDeviceUpdateIntialFullAutoUpdate
Processing Logic	
Initialization LED updates information. WHILE search whole LED files. IF match with file name type, Set update target device. Set update target data. Register to pre-set data on update list.	

```
        END IF
        Sorted the list searched by the type of data.                    // Sorted based on downloads.
        Download data and Mark the data to reset.
        Set LED update Timer.
END WHILE
```

6.3.1.2.18. OnLedDeviceFullAutoUpdate

Identifier	CContentUpdateDlg.OnLedDeviceFullAutoUpdate

Processing Logic

```
IFIt is LED automatic updates,
        If the LED updates status is NG
                Shut down LED updates.
        ELSE
                SELECT UPDATE_STEP
                        If it is INTIAL
                                Prepare for LED updates.
                        If  it is VERSION
                                Request the version information for the LED updates object.
                If the versions between CNCS data and the updates memory are not the same.
                                        Progress to download.
                If the versions between the update memory and the running memory are not the same.
                                        Progress to update.
                                ELSE
                                        Shut down the updates work.
                                END IF
                If it is READY.
                                Prepare to download LED updates data.
                If it is DOWNLOAD.
                                Download LED updates data.
                        If it is DOWNLOAD COMPLETE.
                        Request again the version information If down is completed.
                If it is UPDATE
                         Transfer from the download memory to the running memory.
                If it is RESET.
                                Reset the device.
                If it is COMPLETE.
                                Shut down LED updates if all LED is done updating.

                                ELSE
                                        Progress to update the next LED device.
                                END IF
                        END CASE
        END IF
END IF
```

6.3.1.2.19. OnCcpDeviceUpdateInitialFullAutoUpdate

Identifier	CContentUpdateDlg.OnCcpDeviceUpdateInitialFullAutoUpdate

Processing Logic

Initialization CCP update information

```
WHILE Search all CCP file
        IF match file name type
                Set update target device.
                Set update target data.
                Register to pre-set data on update list.
        END IF
        Set CCP update Timer.
END WHILE
```

6.3.1.2.20. OnCcpDeviceFullAutoUpdate

Identifier	CContentUpdateDlg.OnCcpDeviceFullAutoUpdate
Processing Logic	

6.3.1.2.21. OnCcpDeviceUpdate

Identifier	CContentUpdateDlg.OnCcpDeviceUpdate
Processing Logic	

```
IF CCP is automatic updates.
        SELECT CASE
                If it is INITIAL.
                Initialize CCP Content Update.
                If it is VERSION.
                Request the data version to upload on CCP.
                        If the verison to upload is not the same.
                        Prepare to upload.
                        IF the version uploaded and the version in operation are not the same.
                        Prepare to update.
                        ELSE
                                Shut down CCP update work.
                        END IF
                If it is READY.
                Prepare for CCP Upload.
                If it is UPLOAD COMPLETED.
                        Request the packet information not received of the CCP upload data.
                        If the packet not received is existent.
                                Move to the UPLOAD work.
                        ELSE
                        Move to the RESET work.
                        END IF
                IF it is UPLOAD.
                        CCP transferes the packet not received.
                If it is RESET.
                        If all Update packets are transmitted
                        Update CCP.
                        ELSE
                                Update the next content data.
                        END IF
        END CASE
END IF
```

6.3.1.2.22. OnIntialCncsFullAutoUpdate

Identifier	CContentUpdateDlg.OnIntialCncsFullAutoUpdate

Processing Logic

```
Prepare CNCS automatic update

IF CNCS all data is updated
        Terminate CNCS automatic update
        Clear communication authority of CAR LINE.
ELSE
        SELECT CASE
                CASE PP
                        Prepare PP Content update.
                CASE SP
                        Prepare PP Content update
                CASE CONTENT
                        Prepare content update
        END CASE

        Search relevant update information.

        CALL OnCncsConetnsUpdate
END IF
```

6.3.1.2.23. OnCncsContentsUpdate

Identifier	CContentUpdateDlg.OnCncsContentsUpdate

Processing Logic

```
Move CNCS update STEP to the next work.

SELECT CASE
        If it is CASE VERSION.
                Search the version information of Update object.
                IF the version information is not the same.
                Start to update.
                ELSE
                        Shut down the update work.
                END IF
        If it is READY.
                Delete the existing data version information of the folder in operation.
        If it is REMOVE.
                Delete the data in operation.
        If it is MOVE.
                Copy the update data to the data in operation.
        If it is SET_VERSION.
                Mark the data version information when the copy is completed.
END CASE
```

6.3.1.2.24. InitialSerialPort

Identifier	CSerialPort.InitialSerialPort
Processing Logic	
Initialization communication port information. Create usable communication port list.	

6.3.1.2.25. CreateSerialPortThread

Identifier	CSerialPort.CreateSerialPortThread
Processing Logic	
Initialization transmission buffer. Initialization receiving buffer. Create Car Line Rx Thread. Create Car Line Tx Thread.	

6.3.1.2.26. RemoveSerialPortThread

Identifier	CSerialPort. RemoveSerialPortThread
Processing Logic	
Delete Car Line Rx Thread. Delete Car Line Tx Thread.	

6.3.1.2.27. SerialPortThreadFunc

Identifier	CSerialPort.SerialPortRxThreadFunc
Processing Logic	

```
Initialize CAR Line reception buffer.

Until WHILE program is finished.
        If STX and ETX codes are existent.
                If the date received is ASCII.
                        BCC is normal from STX to ETX.
                                CALL OnSerialReceiveData
                        END IF
                END IF
        END IF
END WHILE
```

6.3.1.2.28. SerialPortTxThreadFunc

Identifier	CSerialPort.SerialPortTxThreadFunc
Processing Logic	

```
Initialize CAR Line transmission buffer.

Until WHILE program is finished.
        If the data is existent in the buffer to transfer.
                Create CHECK SUM of the data to tranfer.
                Change CHAR data to BYTE data.
                Transmit the data to CAR LINE.
```

```
        END IF
END WHILE
```

6.3.1.3. Main

6.3.1.3.1. Create

Identifier	CUdpSocket.Create
Processing Logic	
Create UDP communication socket.	

6.3.1.3.2. OnReceive

Identifier	CUdpSocket.OnReceive
Processing Logic	
If it receives the data. Transmit the data received to Main Process. END IF	

6.3.1.3.3. SendTo

Identifier	CUdpSocket.SendTo
Processing Logic	
If Main Process is in operation. Transmit the reception datat to Main Process. END IF Transmit UDP data to Sub Process.	

6.3.1.3.4. Run

Identifier	CMonitorThread.Run
Processing Logic	
If WHIEL Sub Process is in monitoring. IF Monitoring ITEM is existent. The SELECT PORT number is. If it is CONTENT_UPDATE. Transmit CNCS status requests data of PAC. If it is PP or SP. Transmit the entire communication data of PAC. . If it is Main, FTPData or MDSData. Transmit CNCS status requests data of PAC. END CASE END IF END WHEIL	

6.3.1.3.5. CreateSerialPort

Identifier	CMainDlg.CreateSerialPort
Processing Logic	
IF it hasn't yet been created by using PTU port.	

```
        Create PTU Serial Port.
END IF

IF it hasnt yet been created by using CarLine port.
        Create Car Line port.
END IF
```

6.3.1.3.6. RemoveSerialPort

Identifier	CMainDlg.RemoveSerialPort
Processing Logic	

6.3.1.3.7. OnSerialCarReciveData

Identifier	CMainDlg.OnSerialCarReciveData
Processing Logic	

```
IF PTU port create
        Close PTU port.
END IF

IF CAR Line port create
        Close CAR Line port.
END IF
```

6.3.1.3.8. SendPacRequestData

Identifier	CMainDlg.SendPacRequestData
Processing Logic	

```
Create transmissible buffer.

IF set point is normal.
        Create CAR LINE data using set point.
        Transfer created data to Car Line.
END IF
```

6.3.1.3.9. OnPacReceiveDataView

Identifier	CMainDlg.OnPacReceiveDataView
Processing Logic	

Analysis received data from CAR LINE.
Indicate analysis data to screen.

6.3.1.3.10. CreateMonitorThread

Identifier	CMainDlg.CreateMonitorThread
Processing Logic	

```
IF Monitoring processor doesn't create
        Create UDP socket
```

```
        IF UDP socket created fail
                Delete failed socket.
        END IF.
END IF
```

6.3.1.3.11. RemoveMonitorThread

Identifier	CMainDlg.RemoveMonitorThread
Processing Logic	
IF Monitoring socket create Delete created socket. END IF	

6.3.1.3.12. OnUdpSendTo

Identifier	CMainDlg.OnUdpSendTo
Processing Logic	
Renew the status requests information to SUB Process.	

6.3.1.3.13. OnUdpReceive

Identifier	CMainDlg.OnUdpReceive
Processing Logic	
Renew the response information of SUB Process.	

6.3.1.3.14. RestartTimeOutProgram

Identifier	CMainDlg.RestartTimeOutProgram
Processing Logic	
If the response time of Sub Process is exceeded. Shut down the process in operation. Rerun the Sub process with the response time excess END IF	

6.3.1.3.15. InitConfigurationSystemVolume

Identifier	CMainDlg.InitConfigurationSystemVolume
Processing Logic	
Read the Speaker information set last.	

6.3.1.3.16. WriteConfigurationSystemVolume

Identifier	CMainDlg.WriteConfigurationSystemVolume
Processing Logic	
Store the Speaker information chagened by the user	

6.3.1.3.17. GetItemSpeakerVolumeValue

Identifier	CMainDlg.GetItemSpeakerVolumeValue

Processing Logic
If ITEM SPEAKER volume that the user requests is existent. Bring the relevant ITEM information that the user requests. END IF

6.3.1.3.18. ExecuteAllSubProgram

Identifier	CMainDlg.ExecuteAllSubProgram
Processing Logic	
If Sub Process is not in operation. Run Sub Process. END IF	

6.3.1.3.19. TerminateAllProgram

Identifier	CMainDlg.TerminateAllProgram
Processing Logic	
IF Sub process execute. Sub process terminates. END IF	

6.3.1.3.20. InitSharedMemory

Identifier	CSharedMemory InitSharedMemory.
Processing Logic	
Create the shared memory. If the shared memory is existent. Open the existing memory. END IF If shared memory creation and Open work are successful. Store the memory information created. END IF	

6.3.1.3.21. SendProcessMemssage

Identifier	CSharedMemory. SendProcessMemssage
Processing Logic	
If the shared memory is created. . Record the date to transmit in the shared memory. Sub Process event occured. END IF	

6.3.1.3.22. ThreadSharedMemroy

Identifier	CSharedMemory. ThreadSharedMemroy
Processing Logic	
Repeat til WHILE program is completed.	

```
        If Sub Process event  occured.
                Transmit the data recorded in the shared memory to Main Process.
        END IF
END WHILE
```

6.3.1.3.23. InitialSerialPort

Identifier	CSerialPort.InitialSerialPort
Processing Logic	
Communication port information Initialization. Create useable communication port list.	

6.3.1.3.24. CreateSerialPortThread

Identifier	CSerialPort.CreateSerialPortThread
Processing Logic	
Transmission buffer Initialization. Receiving buffer initialization. Creates Car Line Rx Thread. Creates Car Line Tx Thread.	

6.3.1.3.25. RemoveSerialPortThread

Identifier	CSerialPort. RemoveSerialPortThread
Processing Logic	
Car Line Rx Thread deletes. Car Line Tx Thread deletes.	

6.3.1.3.26. SerialPortRxThreadFunc

Identifier	CSerialPort.SerialPortRxThreadFunc
Processing Logic	

```
Initialize Car Line reception buffer.

Until WHILE program is completed.
        If STX and ETX codes are existent
                If the date received is ASCII
                If BCC is normal from STX to ETX.
                                CALL OnSerialReceiveData
                        END IF
                END IF
        END IF
END WHILE
```

6.3.1.3.27. SerialPortTxThreadFunc

Identifier	CSerialPort.SerialPortTxThreadFunc
Processing Logic	
Initialize Car Line transmisstion buffer.	

```
Until WHILE program is completed.
        If the buffer to transimt is existent
                Create CHECK SUM of the date to transmit
                Change CHAR data to BYTE data.
                Transmit the data to CAR LINE.
        END IF
END WHILE
```

6.3.1.4. MDSData

6.3.1.4.1. IniSharedMemory

Identifier	CSharedMemory InitSharedMemory.
Processing Logic	

```
Create the shared memory.

If the shared memory is existent.
        Open the existing memory.
END IF

If shared memory creation and Open work are successful.
        Store the memory information created.
END IF
```

6.3.1.4.2. SendProcessMemssage

Identifier	CSharedMemory. SendProcessMemssage
Processing Logic	

```
If the shared memory is created.
        Record the data to transmit in the shared memory.
        Create MDS Process events.
END IF
```

6.3.1.4.3. ThreadSharedMemroy

Identifier	CSharedMemory. ThreadSharedMemroy
Processing Logic	

```
Repeat til WHILE program is completed.
        If Sub Process occurs.
                Transmit the date recorded in the shared memory to MDS Process.
        END IF
END WHILE
```

6.3.1.4.4. OnReceiveLicSerialData

Identifier	CMDSData. OnReceiveLicSerialData
Processing Logic	

```
SELECT LIC requests data.
        If time information is requested.
                CALL SendToLICSystemTime
        If it is MDS failure data.
```

```
                CALL ReciveLICFaultData
        If MDS failure date is completed.
                CALL ReciveLICFaultComplate
END CASE
Reset WatchDog Timer.
```

6.3.1.4.5. SendToLicSystemTime

Identifier	CMDSData. SendToLICSystemTime
Processing Logic	
Renew the PA system version information. If the time received from GIA_DATA is greater than 2010. Search the existing CNCS time information. Search the version information of PA system. . Transmit the data to LIC. END IF	

6.3.1.4.6. ReceiveLicFaultData

Identifier	CMDSData. ReciveLICFaultData
Processing Logic	
If either Wayside is On or the Packet number is 1. CALL InitialFaultPacket() Store the new files packet information. END IF If the Packet is normally received. Record the packet information received on the files. Store the packet information normally received to the lists. END IF	

6.3.1.4.7. ReceiveLicFaultComplete

Identifier	CMDSData. ReciveLICFaultComplate
Processing Logic	
Store the last packet information received from LIC.	

6.3.1.4.8. InitialFaultPacket

Identifier	CMDSData. InitialFaultPacket
Processing Logic	
Initialize the packet information normally received.	

6.3.1.4.9. SendToLicFaultComplete

Identifier	CMDSData. SendToLICFaultComplate
Processing Logic	
WHILE reception packet lists.	

```
        Search the packet lists not received.
END WHILE

CALL SendToLICFaultComplate
```

6.3.1.4.10. SendToLicFaultComplete

Identifier	CMDSData. SendToLICFaultComplate

Processing Logic

```
If the packet lost is existent.
        LIC requests re-transmitting of the packet lost.
ELSE
        . Inform LIC that the reception is completed.
END IF
```

6.3.1.4.11. OnWatchDogEvent

Identifier	CMDSData. OnWatchDogEvent

Processing Logic

```
If It doesn't receive the data from LIC at the appointed time.
        Stop the failure data collecting work of LIC.
        Inform the defective work to Main Process.
END IF
```

6.3.1.4.12. UpdateVersion

Identifier	CUnitVersion. UpdateVersion

Processing Logic

```
Read the information to manage the version information.
WHILE version management lists.
        Renew the version information of the relevant device.
END WHILE
```

6.3.1.4.13. GetUnitName

Identifier	CUnitVersion. GetUnitName

Processing Logic

```
Read a name of the device.

WHILE device name lists.
        If the device codes are the same.
                Return the device name.
        END IF
END WHILE
```

6.3.1.4.14. InitialSerialPort

Identifier	CSerialPort.InitialSerialPort

Processing Logic

Initialize the communication port information.

Create the communication port list available.

6.3.1.4.15. CreateSerialPortThread

Identifier	CSerialPort.CreateSerialPortThread
Processing Logic	
Initialize the transmission buffer initialize the reception buffer Creat LIC Rx Thread. Creat LIC Tx Thread.	

6.3.1.4.16. RemoveSerialPortThread

Identifier	CSerialPort. RemoveSerialPortThread
Processing Logic	
Delete LIC Rx Thread Delete LIC Tx Thread.	

6.3.1.4.17. SerailPortRxThreadFunc

Identifier	CSerialPort.SerialPortRxThreadFunc
Processing Logic	

Initialize LIC reception buffer.
 	
Until WHILE program is completed.
  If STX and ETX codes are existent.
  If the data received ir ASCII.
  If BBC is normal from STX to ETX.
  CALL OnReceiveLicSerialData
  END IF
  END IF
  END IF
END WHILE	

6.3.1.4.18. SerailPortTxThreadFunc

Identifier	CSerialPort.SerialPortTxThreadFunc
Processing Logic	

Initialize Car Line transmission buffer.
 	
Until WHILE program is finished.
  If the data in the buffer to transmit is existent.
  Create CHECK SUM of the data to transmit.
  Change CHAR data to BYTE data.
  Transmit the data to CAR LINE.
  END IF
END WHILE	

6.3.1.4.19. FTPData

Identifier	CSharedMemory InitSharedMemory.
Processing Logic	

Create the shared memory.
 	
If the shared memory is existent.
  Open the existing memory.
END IF
 	
If the shared memory creation and Open work are successful.
  Store the memory information created.
END IF	

6.3.1.4.20. SendProcessMessage

Identifier	CSharedMemory. SendProcessMemssage
Processing Logic	

If the shared memory is created.
  Record the data to transmit on the shared memory.
  FTP Process event occurs.
END IF	

6.3.1.4.21. ThreadSharedMemroy

Identifier	CSharedMemory. ThreadSharedMemroy
Processing Logic	

Repeat til WHILE program is finished.
  If Sub Process event occurs.
  Transmit the recorded data in the shared memory to FTP Process.
  END IF
END WHILE	

6.3.1.4.22. LoadServerList

Identifier	CFTPServer. LoadServerList
Processing Logic	

Delete the lists stored of ServerList
 	
If the lists of ServerList are existent.
  WHILE ServerList lists.
  Register the server on ServerList.
  END WHILE
END IF	

6.3.1.4.23. SerachServer

Identifier	CFTPServer. SearchServer
Processing Logic	
CALL ClearServer	

```
WHILE Server lists.
        If DNS address and the registered address are the same.
                Store the Server information found.
                break WHILE
        END IF
END WHILE
```

6.3.1.4.24. GetServerState

Identifier	CFTPServer. GetServerState
Processing Logic	
Return the FTP Server information found.	

6.3.1.4.25. ClearServer

Identifier	CFTPServer. ClearServer
Processing Logic	
Delete the FTP Server information found.	

6.3.1.4.26. InitConfigurationFtpUload

Identifier	CFTPDataDlg. InitConfigurationFtpUload
Processing Logic	
Bring the lists information to upload from the registry	

6.3.1.4.27. WriteConfigurationFtpUpload

Identifier	CFTPDataDlg. WriteConfigurationFtpUpload
Processing Logic	
Store the lists information to upload to the registry.	

6.3.1.4.28. InitConfigurationFtpDownload

Identifier	CFTPDataDlg. InitConfigurationFtpDownload
Processing Logic	
Bring the Download lists Item information from the registry.	

6.3.1.4.29. WriteConfigurationFtpDownload

Identifier	CFTPDataDlg. WriteConfigurationFtpDownload
Processing Logic	
Store the Download lists information to the registry.	

6.3.1.4.30. InitialScript

Identifier	CFTPDataDlg. InitialScript
Processing Logic	
Initialize the Wayside Script information.	

6.3.1.4.31. OnCompletedScriptStep

Identifier	CFTPDataDlg. OnCompletedScriptStep

Processing Logic

```
If  Wayside work is in the progress.
        SELECT
                CASE UPLOAD_SCRIPT:
                        CALL OnUpalodScriptStepComplate
                CASE DOWNLAOD_SCRIPT
                        CALL OnDownloadScriptStepComplate
        END CASE
END IF
```

6.3.1.4.32. OnScriptNextSource

Identifier	CFTPDataDlg. OnScriptNextSource

Processing Logic

```
If the Script item to progress is existent.
        Set up the existing Item information.
        Move the Item information to the next lists.
END IF
```

6.3.1.4.33. OnCompletedScript

Identifier	CFTPDataDlg. OnCompletedScript

Processing Logic

```
SELECT Wayside Step
        CASE WAYSIDE_ON:
                Set up IP_RENEW Step.
        CASE PING:
                If FTP Sever responds
                        If FTP work step is Upload.
                                Set up UPLOAD_SCRIPT step.
                        ELSE
                                Set up DOWNLOAD_SCRIPT step.
                        END IF
                ELSE
                        Set-up IP_RENEW Step.
                END IF
        CASE IP_RENEW:
                Set-up PING Step.
        CASE UPLOAD_SCRIPT:
                Set-up DOWNLOAD_SCRIPT.
        CASE DOWNLOAD_SCRIPT:
                Complete Wayside work.
END CASE

The work with SELECT setting.
        Work is completed.
                End Wayside work.
        CASE PING :
```

```
                CALL Ping
        CASE IP_RENEW:
                Progress the IP renewing work of CNCS.
        CASE UPLOAD_SCRIPT:
                CALL OnUploadScript
        CASE DOWNLAOD_SCRIPT:
                CALLOnDownloadScript
END SELECT
```

6.3.1.4.34. OnUploadScript

Identifier	CFTPDataDlg. OnUploadScript

Processing Logic

```
SELECT Upload 작업 step
        CASE Step1:     Try FTP Server and Ping test.
        CASE Step2:     Try to connect with FTP Server.
        CASE Step3:     Progress the File Upload work.
        CASE Step4:     Disconnect with FTP Server.
        CASE Step5:     Complete the Download work.
END CASE
```

6.3.1.4.35. OnUpalodScriptStepComplate

Identifier	CFTPDataDlg. OnUpalodScriptStepComplate

Processing Logic

```
If it is disconnected with FTPServer.
        Set to progress Step4.
ELSE
        SELECT Download work step
                CASE Step1:
                        If Ping test is completed.
                                Set to progress Step2.
                        END IF,
                CASE Step2:
                        If the connection with FTP Server is normal.
                                Set to progress Step3.
                        END IF,
                CASE Step3
                        If the upload object is not existent.
                                Set to progress Step4.
                        END IF,:
                CASE Step4:
                CASE Step5:
                        Process the FTP Upload work as an error.
        END CASE

        CALL OnUploadScript
END IF
```

6.3.1.4.36. OnUploadScriptNextSource

Identifier	CFTPDataDlg. OnUploadScriptNextSource

Processing Logic

```
If the upload object is existent.
        Bring the object information to upload
        Move the object  to the next step.
ELSE
        The upload object is not existent.
END IF
```

6.3.1.4.37. OnDownloadScript

Identifier	CFTPDataDlg. OnDownloadScript

Processing Logic

```
SELECT Download Work step Bring the object information to upload
        CASE Step1:     Try FTP Server and Ping test.
        CASE Step2:     Try to connect with FTP Server.
        CASE Step3:     Progress the File Download work.
        CASE Step4:     Disconnect with FTP Server.
        CASE Step5:     Complete the Download work.
END CASE
```

6.3.1.4.38. OnDownloadScriptStepComplate

Identifier	CFTPDataDlg. OnDownloadScriptStepComplate

Processing Logic

```
If it is disconnected with FTPServer.
        Set to progress Step4.
ELSE
        SELECT Download Work step
                CASE Step1:
                        If Ping test is completed.
                                Set to progress Step2.
                        END IF,
                CASE Step2:
                        . If the connection with FTP Server is normal.
                                Set to progress Step3.
                        END IF,
                CASE Step3
                        IF the next download is not existent.
                                Set to progerss Step4.
                        END IF,:
                CASE Step4:
                CASE Step5:
                        Process the FTP Download work as an error.
        END CASE

        CALL OnDownloadScript
END IF
```

6.3.1.4.39. OnDownloadScriptNextSource

Identifier	CFTPDataDlg. OnDownloadScriptNextSource

Processing Logic

```
If the download object is existent.
        Set the download object information
        Move  the download object to the next step.
ELSE
        No download object.
END IF
```

6.3.1.4.40. Ping

Identifier	CPing. Ping

Processing Logic

```
Create the communication soket.
If the communication soket is normally created.
        Repeat as many as the number of FOR LOOP.
                CALL SendEchoRequest
                CALL WaitForEchoReply

                If the transmission work is normally completed.
                        CALL RecvEchoReply
                        If the response is at the appointed time.
                Ping test is normally completed.
                        ELSE
                                Ping test Time out occurs.
                        END IF
                ELSE
                        Host server error occurs.
                END IF
        NEXT FOR
END IF
```

6.3.1.4.41. RecvEchoReply

Identifier	CPing. RecvEchoReply

Processing Logic

Bring the test response data of FTP Server from the communication SOCKET.

6.3.1.4.42. SendEchoRequest

Identifier	CPing. SendEchoRequest

Processing Logic

Initialize PING test data.
Store the current time information.
Transmit the PING test data to FTP server to PING test.

6.3.1.4.43. WaitForEchoReply

Identifier	CPing. WaitForEchoReply

Processing Logic
Wait until FTP Server and the event of the communication SOCKET occur.

6.3.1.4.44. OnRemoteConnect

Identifier	CFTPThread. OnRemoteConnect
Processing Logic	
in case of not connecting with FPT Server. Try to connect with FTP Server. END IF	

6.3.1.4.45. CreateDirectory

Identifier	CFTPThread. CreateDirectory
Processing Logic	
in case of connection with FTP Server. Create the route directory appointed on FTP Server. END IF	

6.3.1.4.46. SetCurrentDirectory

Identifier	CFTPThread. SetCurrentDirectory
Processing Logic	
If it is connecting with FTP Server. Move to the route appointed in FTP Server. END IF	

6.3.1.4.47. GetCurrentDirectory

Identifier	CFTPThread. GetCurrentDirectory
Processing Logic	
If it is connection with FTP Server. Return the existing work directory name of FTP Server. END IF	

6.3.1.4.48. OnUpload

Identifier	CFTPThread. OnUpload
Processing Logic	

If it is connecting with FTP Server.
Receive the file lists to upload on CNCS.
 	
WHILE upload file lists.
Upload the object files to a temporary route of FTP Server.
If upload is normally completed.
Move the uploaded files to the normal position.
END IF
END WHILE
END IF	

6.3.1.4.49. OnDownload

Identifier	CFTPThread. OnDownload

Processing Logic

```
If it is connecting with FTP Server.

        Receive the file lists to download.

        WHILE download file lists.
                If the object is a directory.
                        CALL OnDownload
                ELSE
                        a download route is not existent.
                                Create a directory to download.
                        END IF

                        Download the files appointed.
                END IF
        END WHILE

        If the download of the entire file is normally completed.
                Mark the DOWNLOAD completion.
        END IF
END IF
```

6.3.1.4.50. OnRemoteClose

Identifier	CFTPThread. OnRemoteClose

Processing Logic

```
If it is connecting with FTP Server.
        Disconnect with FTP Server.
END IF
```

6.3.1.5. PP

6.3.1.5.1. OnMediaNotify

Identifier	CPlayerTestDlg.OnMediaNotify

Processing Logic

```
in case of displaying the existing messages
        Erase the messages and return.
ELSE
         Erase the messages and play the next contents.
END IF
```

6.3.1.5.2. LoadPlayList

Identifier	CPlayerTestDlg. LoadPlayList

Processing Logic

The number of WHILE upload file lists.
If the play lists codec and the codec provided by the player are the same.
Add to the play lists.
Upload file lists number growth.
END IF
END WHILE

6.3.1.5.3. LoadSetupFile

Identifier	CPlayerTestDlg. LoadSetupFile
Processing Logic	
If the setting file is existent. Load the setting values. END IF	

6.3.1.5.4. OnSocket

Identifier	CPlayerTestDlg. OnSocket
Processing Logic	

If time is changed.
If it is the current time display mode.
Update the time.
END IF
END IF
If it is a trigger point valid.
CALL DisplayInfoScreen
ELSE
Play the content.
END IF	

6.3.1.5.5. OnOneshotEvent

Identifier	CPlayerTestDlg. OnOneshotEvent
Processing Logic	

If playback time of the still image is expired.
 	
END IF
If time of the passenger guide messages is expired.
If it is the last part of the passenger guide messages.
Play an ad content.
ELSE
CALL DisplayInfoScreen
 	
END IF	

6.3.1.5.6. OnUpdateText

Identifier	CPlayerTestDlg. OnUpdateText

Processing Logic

If it is not same as the counts of the text data loaded.
Output the string on the location relevant to the count and increase 1 count
ELSE
Output the entire screen.
END IF

6.3.1.5.7. LoadTrigPointFile

Identifier	CPlayerTestDlg. LoadTrigPointFile
Processing Logic	
If it is existent on the route by loading the trigger point files appointed If the return value of CALL LoadSCHInfo is TRUE CALL DisplayInfoScreen END IF	

6.3.1.5.8. DisplayInfoScreen

Identifier	CPlayerTestDlg. DisplayInfoScreen
Processing Logic	

If the size of trigger point play lists is not 0.
If the video is playing.
Stop the video.
END IF
If the list is a text.
CALL ParseStringAndGenerateBitmap
Start to display guide messages.
If the list is time.
이면 If the value of TimeValidityCheck is TRUE.
Display time.
END IF
END IF
ELSE
Replay the video stopped.
END IF	

6.3.1.5.9. OnResizeScr

Identifier	CPlayerTestDlg. OnResizeScr
Processing Logic	
Adjust the size of the screen by the value of Parameter.	

6.3.1.5.10. ParseStringAndGenerateBitmap

Identifier	CPlayerTestDlg. ParseStringAndGenerateBitmap
Processing Logic	
Parse the contents of m_XMLBuffer. Output the messages on Bitmap. Initialize the text counts. Store the location for the text to be displayed.	

6.3.1.5.11. InitBitmap

Identifier	CPlayerTestDlg. InitBitmap
Processing Logic	
Initialize the background color by parting the screen display bitmap with a destination part and a middle message part.	

6.3.1.5.12. GetTagType

Identifier	CPlayerTestDlg. GetTagType
Processing Logic	
If a sting is "FONT" Return TAG_FONT. If a sting is “B" Return TAG_BOLD If a sting is “I" Return TAG_ITALIC If a sting is “BR" Return TAG_BREAK If a sting is “UPTEXT" Return TAG_UP If a sting is “MIDDLETEXT " Return TAG_MIDDLE If a sting is “BOTTOMTEXT " Return TAG_BOTTOM	

6.3.1.5.13. GetAttrType

Identifier	CPlayerTestDlg. GetAttrType
Processing Logic	
If a tag is TAG_FONT and a sting is “COLOR” Return FONT_COLOR. If a sting is "SIZE" Return FONT_SIZE. “FACE” 이면 If a sting is “FACE” Return FONT_FACE. END IF ELSE IF TAG_UP,TAG_MIDDLE,TAG_BOTTOM If a tag is TAG_UP,TAG_MIDDLE,TAG_BOTTOM If a sting is ALIGN Return TEXT_ALIGN END IF END IF Return ATTR_NONE.	

6.3.1.5.14. TextDataFileLoad

Identifier	CPlayerTestDlg. TextDataFileLoad
Processing Logic	

If it is not the end of the file. Load on m_XMLBuffer END WHILE

6.3.1.5.15. UpdateTimeBitmap

Identifier	CPlayerTestDlg. UpdateTimeBitmap
Processing Logic	
Output time.	

6.3.1.5.16. TimeValidityCheck

Identifier	CPlayerTestDlg. TimeValidityCheck
Processing Logic	
If the day of time requested is less than 1 or greater than 31. Return FALSE. END IF If the month of time requested is less than 1 or greater than 12. Return FALSE. END IF If the time of time requested is greater than 23. FALSE Return FALSE. END IF If the minute of time requested is greater than 59. Return FALSE. END IF If the second of time requested is greater than 59. Return FALSE. END IF If the year of time requested is less than 8 or greater than 99. Return FALSE. END IF Return TRUE.	

6.3.1.6. SP

6.3.1.6.1. LoadSentenceFile

Identifier	CScrollSent. LoadSentenceFile
Processing Logic	
If pass is existent. Load a text file of the relevant pass. Return TRUE. END IF Return FALSE.	

6.3.1.6.2. DrawSentence

Identifier	CScrollSent. DrawSentence

Processing Logic
Create pass by reading a sentence Get the size of the pass. If the size of the pass is bigger than the screen. Set the left- scroll. Create the bitmap. If the size of the pass is not bigger than the screen. Set the up-scroll Create the bipmap. END IF

6.3.1.6.3. ScrollLongSentence

Identifier	CScrollSent. ScrollLongSentence
Processing Logic	
If the location of the bitmap is not bigger than the width of the bitmap. Output the bitmap and increase the location of the bitmap. Return TRUE. END IF Return FALSE.	

6.3.1.6.4. ScrollShortSentence

Identifier	CScrollSent. ScrollShortSentence
Processing Logic	
If the location of the bitmap is not bigger than the height of the bitmap. Output the bitmap and increase the location of the bitmap. Return TRUE. END IF Return FALSE.	

6.3.1.6.5. ScrollStop

Identifier	CScrollSent. ScrollStop
Processing Logic	
Stop the timer. Hide the display area and transmit the screen control message to PP.	

6.3.1.6.6. ScrollStart

Identifier	CScrollSent. ScrollStart
Processing Logic	
If a trigger file is existent. Stop the timer Create the bipmap then. Show the display area and transmit the screen control message to PP. Get the timer stared. ELSE IF Stop the timer. END IF	

6.3.1.6.7. IsScrolling

Identifier	CScrollSent. IsScrolling
Processing Logic	
Judge if it is scrolling or not. Return TRUE. ELSE Return FALSE.	

6.3.1.6.8. InitPositionSetLimit

Identifier	CScrollSent. InitPositionSetLimit
Processing Logic	
Fine the size of the bitmap. According to the scroll type. If it is a short sentence. The entire scroll size is the height of the bitmap. Initialize the scroll count. If it is a long sentence. The entire scroll size is the width of the bitmap. Initialize the scroll count. END IF	

6.4. LEDC Description

6.4.1. Module Detailed Design

6.4.1.1. Main Process

ID	LEDC.MainProcess.F

Processing Logic	BEGIN Process
	INITIAL General IO
	INITIAL Variable Value
	INITIAL Timer and Interrupt
	INITIAL UART
	READ Device ID
	6.4.1.2. SELECT CASE Device ID
	6.4.1.3. **CASE1 FDI ID**
	INITIAL SYSTEM DEVICE FDI
	CASE2 SDI ID
	INITIAL SYSTEM DEVICE SDI
	CASE3 PII ID
	INITIAL SYSTEM DEVICE PII
	END CASE
	
	DO WHILE Infinitely
	READ UART Buf
	ANALYSIS Protocol
	IF Timer Scroll THEN Display Scroll PII
	END IF
	IF From PAC DATA THEN
	DO PAC Process
	ELSE IF From CNCS DATA THEN
	DO CNCS Process
	END IF
	END WHILE
	END Process

6.4.1.4. PAC Process

ID	LEDC.PACProcess.F
Processing Logic	**6.4.1.5. BEGIN Process** **6.4.1.6. SELECT CASE Device ID** 6.4.1.7. **CASE1 FDI ID** DO FDI Display Process **CASE2** SDI ID DO SDI Display Process **CASE3** PII ID DO PII Display Process END CASE END Process

6.4.1.8. FDI Display Process

ID	LEDC.FDIDisplayProcess.F
Processing Logic	BEGIN Process READ STORAGE FDI Station Image DISPLAY FDI Station READ STORAGE FDI Logo Image DISPLAY FDI Logo END Process

6.4.1.9. SDI Display Process

ID	LEDC.SDIDisplayProcess.F
Processing Logic	BEGIN Process READ STORAGE SDI Station Image DISPLAY SDI Station READ STORAGE SDI Logo Image DISPLAY SDI Logo END Process

6.4.1.10. PII Display Process

ID	LEDC.PIIDisplayProcess.F
Processing Logic	BEGIN Process READ STORAGE PII Message CONVERT Message TO PII Image DISPLAY PII Image END Process

6.4.1.11. CNCS Process

ID	LEDC.CNCSProcess.F
Processing Logic	BEGIN Process SELECT CASE Device ID CASE1 FDI ID DO FDI Update Process **CASE2** SDI ID DO SDI Update Process **CASE3** PII ID DO PII Update Process END CASE END Process

6.4.1.12. FDI Update Process

ID	LEDC.FDIUpdateProcess.F
Processing Logic	BEGIN Process READ UART Buf CONVERT UART Buf TO FDI Station Image WRITE STORAGE FDI Station Image END Process

6.4.1.13. SDI Update Process

ID	LEDC.SDIUpdateProcess.F
Processing Logic	BEGIN Process READ UART Buf CONVERT UART Buf TO SDI Station Image WRITE STORAGE SDI Station Image END Process

6.4.1.14. PII Update Process

ID	LEDC.PIIUpdateProcess.F
Processing Logic	BEGIN Process READ UART Buf CONVERT UART Buf TO PII Message WRITE STORAGE PII Message END Process

6.4.1.15. Timer Process

ID	LEDC.TimerProcess.F

Processing Logic	BEGIN INTERRUPT Process
	IF Scroll Timer Value > 0 THEN
	  SUBTRACT Scroll Timer Value
	ELSE
	  INITIAL Scroll Timer Value
	END IF
	END Process

6.4.1.16. Serial interrupt Process

ID	LEDC.SerialInterruptProcess.F

Processing Logic	BEGIN INTERRUPT Process
	IF UART RX Interrupt THEN
	  WRITE STRUCTURE UART Buf.RX
	ELSE IF UART TX Interrupt
	  TRANSMITTER UART STRUCTURE UART Buf.TX
	END IF
	END Process

6.4.2. Data Detailed Design

6.4.2.1. Image Storage Data

Struct/ File name	Image Storage Data	
process	write	CNCSProcess
	read	PACProcess
Filed name	data type	description

FDI Image Data	1MBYTE	Image data gathered for FDI device.
SDI Image Data	1MBYTE	Image data gathered for SDI device.
PII Image Data	1MBYTE	Image data gathered for PII device.

6.4.2.2. FDI Image Data

Struct/ File name	FDI Image Data	
process	write	FDIUpdateProcess
	read	FDIDisplayProcess
Filed name	data type	description
FDI APStation	512 KBYTE	Image data of last train station which to be used on actual FDI display in FDIDisplayProcess
FDI UPStation	512 KBYTE	Image data of last train station downloaded from FDIUpdateProcess to update, it's copied for an image of last train station to display in FDI once download is complete.

6.4.2.3. SDI Image Data

Struct/ File name	SDI Image Data	
process	write	SDIUpdateProcess
	read	SDIDisplayProcess
Filed name	data type	description
SDI APStation	512 KBYTE	**6.4.2.4. Image data of last train station which to be used on actual SDI display in SDIDisplayProcess**
SDI UPStation	512 KBYTE	Image data of last train station downloaded from SDIUpdateProcess to update, it's copied for an image of last train station to display in SDI once download is complete.

6.4.2.5. PII Image Data

Struct/ File name	PII Image Data	
process	write	PIIUpdateProcess
	read	PIIDisplayProcess
Filed name	data type	description
PII APMessage	512 KBYTE	Message data which to be used on actual PIII display in PIIDisplayProcess
PII UPMessage	512 KBYTE	Image data of last train station downloaded from PIIUpdateProcess to update, it's copied for an image of last train station to display in PII once download is complete.

6.4.2.6. Serial Buf Data

Struct/ File name	Serial Buf Data	
process	write	MainProcess ,SerialInterruptProcess
	read	MainProcess, SerialInterruptProcess
Filed name	data type	description
PAC Buf Data	BYTE [100]	It saves protocol data from PAC Unit.
CNCS Buf Data	BYTE [670]	It saves protocol data from CNCS unit.

6.4.2.7. PAC Buf Data

Struct/ File name	PAC Buf Data	
process	write	PACProcess, SerialInterruptProcess
	read	PACProcess, SerialInterruptProcess
Filed name	data type	description
cPACBuf	BYTE [100]	It saves protocol data fro, PAC Unit.

6.4.2.8. CNCS Buf Data

Struct /File name	CNCS Buf Data	
process	write	CNCSProcess ,SerialInterruptProcess
	read	CNCSProcess ,SerialInterruptProcess
Filed name	data type	description
cCNCSBuf	BYTE [670]	It saves protocol data from CNCS Unit.

6.4.2.9. Timer Parameter

Struct/ File name	Timer Parameter	
process	write	TimerProcess
	read	MainProcess, TimerProcess
Filed name	data type	description
wSTimer	WORD	It is a variable to save image scroll time of PII device.

6.5. LIC

6.5.1. Module Detailed Design

6.5.1.1. PACDATA

6.5.1.1.1. user_WithPacRs485Loop Interface

Identifier	user_WithPacRs485Loop
Processing Logic	
If it is PAC DATA. Then extract status information of each device. END IF	

6.5.1.1.2. Make1ByteBcc Interface

Identifier	Make1ByteBcc
Processing Logic	
If the test of DATA BCC is normal. Then check device status information. Delete ELSE DATA.	

6.5.1.2. MDSDATA.

6.5.1.2.1. user_LonWorkLoop Interface.

Identifier	user_LonWorkLoop
Processing Logic	
If MDS device status time is requested. Then transmit GPS time data. If device status information is requested. Then transmit device status DATA.	

6.5.1.2.2. user_LonWorkRead Interface.

Identifier	User_LonWorkRead.
Processing Logic	
If CNCS failure data requests are received. Request the failure data to MDS. END IF If MDS failure data is received. MDS Algorism is right. Memory 에 Write. Writhe on the memory. Delete failure requests and data. END IF	

6.5.1.3. CNCSDATA

6.5.1.3.1. User_WithCncsLoop Interface.

Identifier	User.WithCncsLoop
Processing Logic	
If MDS failure records are being received. If the failure recoreds are existent in the memory. If it is not the last block. Transmit DATA to CNCS. If it is the last block. End the failure record requests. END IF END IF END IF	

6.5.1.3.2. Make1ByteBcc Interface

Identifier	Make1ByteBcc
Processing Logic	
If the test of DATA BCC is normal. Then use GPS time information. Delete DATA.	

6.5.1.4. MAIN

6.5.1.4.1. Main Interface

Identifier	Main
Processing Logic	
If communication is received. Process MDSDATA. Process CNCSDATA. Process PACDATA. If MDS failure data is received. Process Memory Process. END IF	